



ENSEA

Beyond Engineering

OPERATING SYSTEMS

3IS – Spécialité Informatique et Systèmes

Slides booklet

Christophe BARÈS
2025

Contents

Introduction	5
Processes	19
Memory Management	31
File Systems	43
Signals & Pipes (IPC 1)	55
System V IPC (IPC 2)	65
Modules	73

3IS – Operating Systems – Introduction

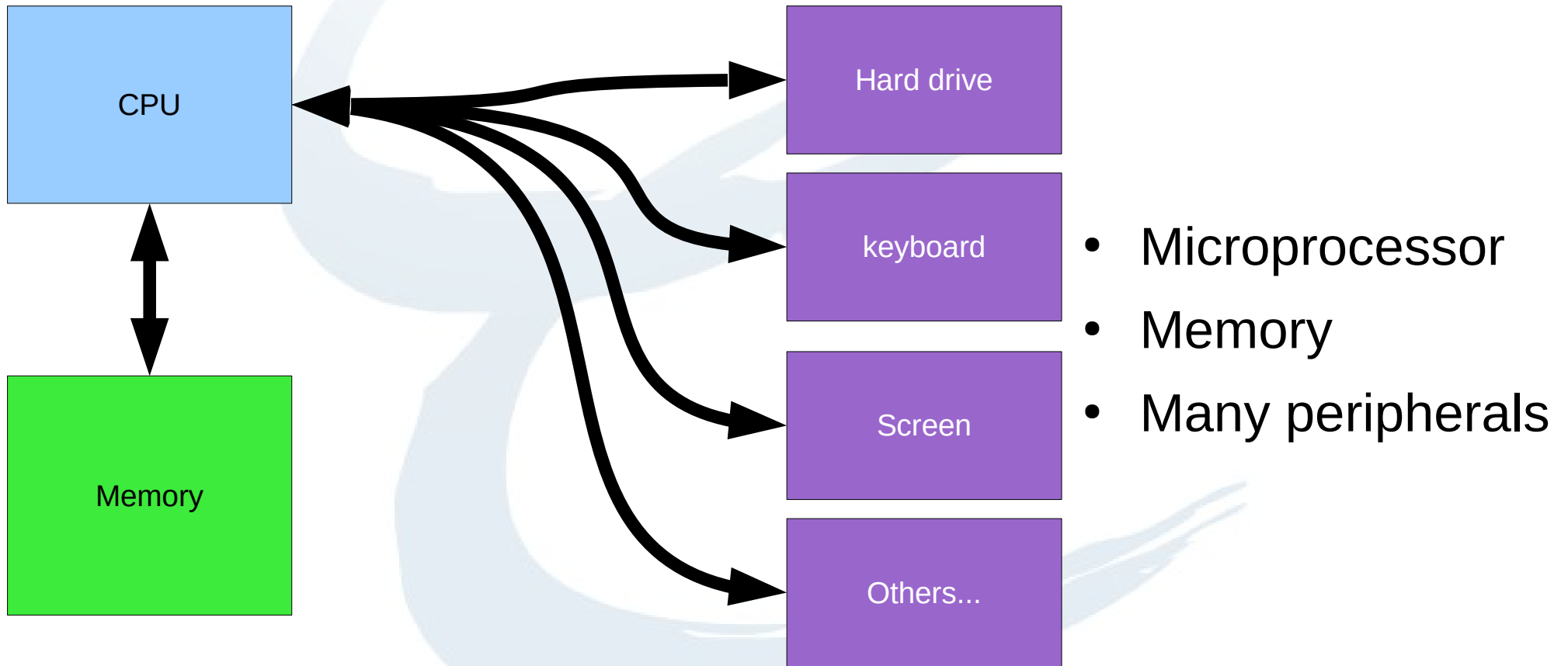
2024 – Christophe Barès
christophe.bares@ensea.fr

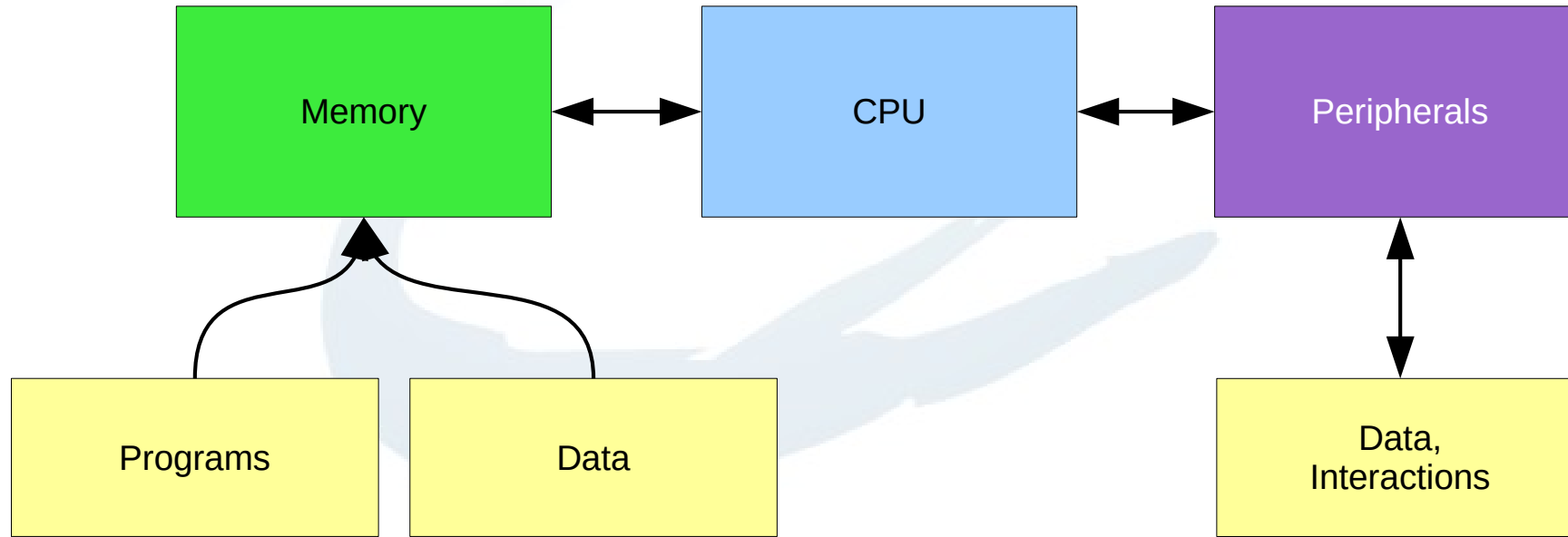
Contributions: David Picard,
Arnaud Revel, Mickaël Maillard

1. Introduction

- A very simple computer
- Goals of an operating system
 - Hardware management
 - Task management
- OS families
- Features
 - Multitasking, multiuser
 - Architecture dependances
- Unix and Linux history

A very simple computer





- Memory :
 - Programs
 - Data
- Peripherals :
 - Data
 - Interactions (high priority data)

Programs

```
int main(int argc, char **argv)
{
    int a, b, c;
    float *A, *B;

    /* check input */
    if(argc < 3)
    {
        printf("usage: dotprod <a> <b>\n");
        return -1 ;
    }

    /* read a from args */
    a = atoi(argv[1]);
    b = atoi(argv[2]);
}
```

- Instructions
- Data

Programs

```
int main(int argc, char **argv)
{
    int a, b, c;
    float *A, *B;

    /* check input */
    if(argc < 3)
    {
        printf("usage: dotprod <a> <b>\n");
        return -1 ;
    }

    /* read a from args */
    a = atoi(argv[1]);
    b = atoi(argv[2]);
}
```

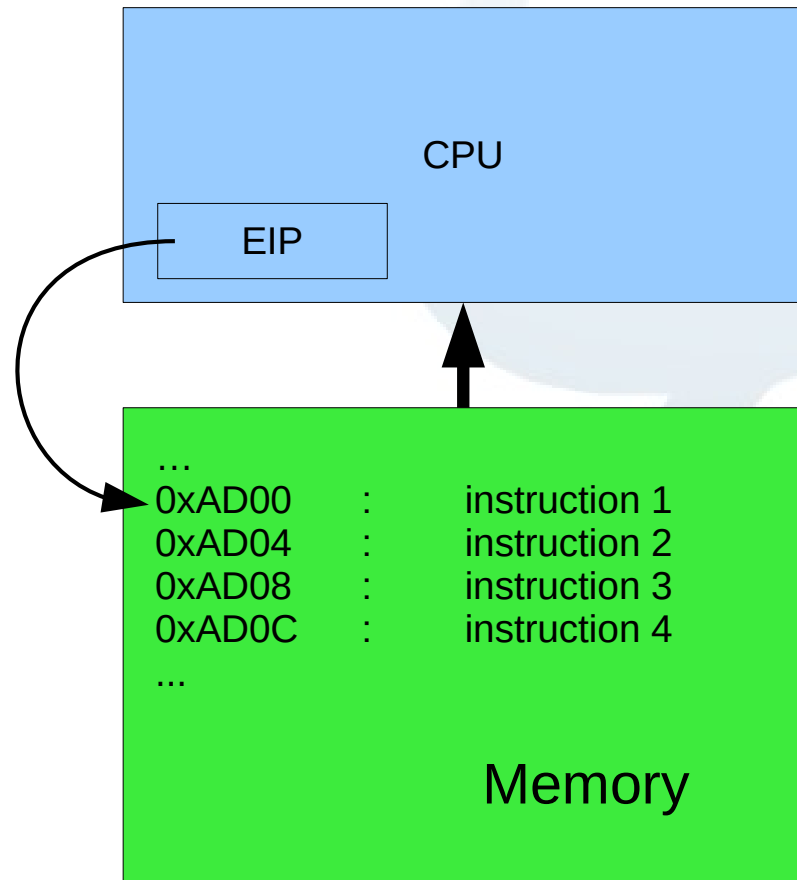
- Instructions
- Data

Memory

Data

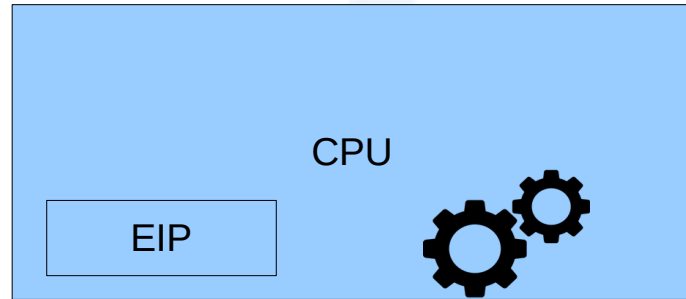
Instructions

Running a program

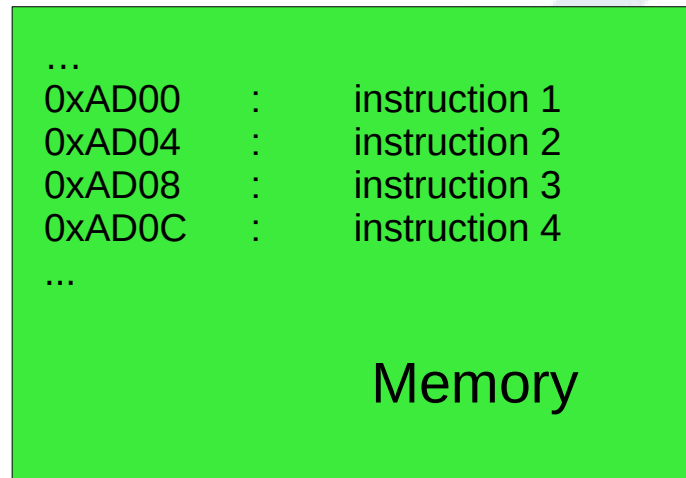


- Read instruction from mem at address contained in EIP ⁽¹⁾ register (Fetch)

Running a program

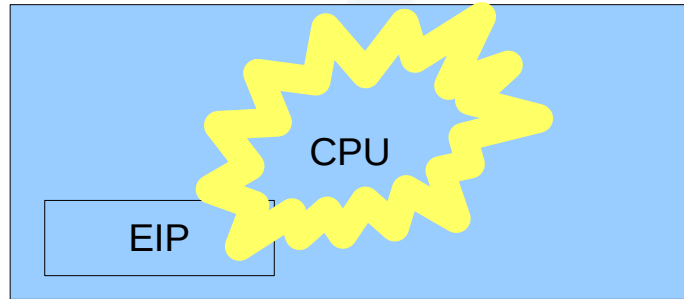


- Read instruction from mem at address contained in EIP ⁽¹⁾ register (Fetch)
- Decode instruction

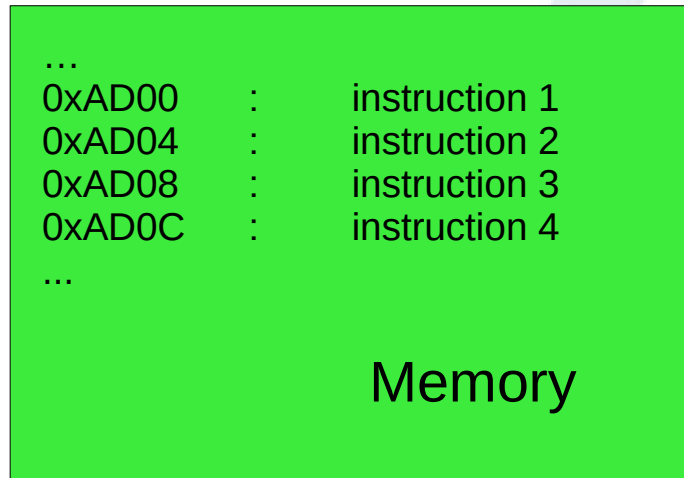


⁽¹⁾ Extended Instruction Pointer

Running a program

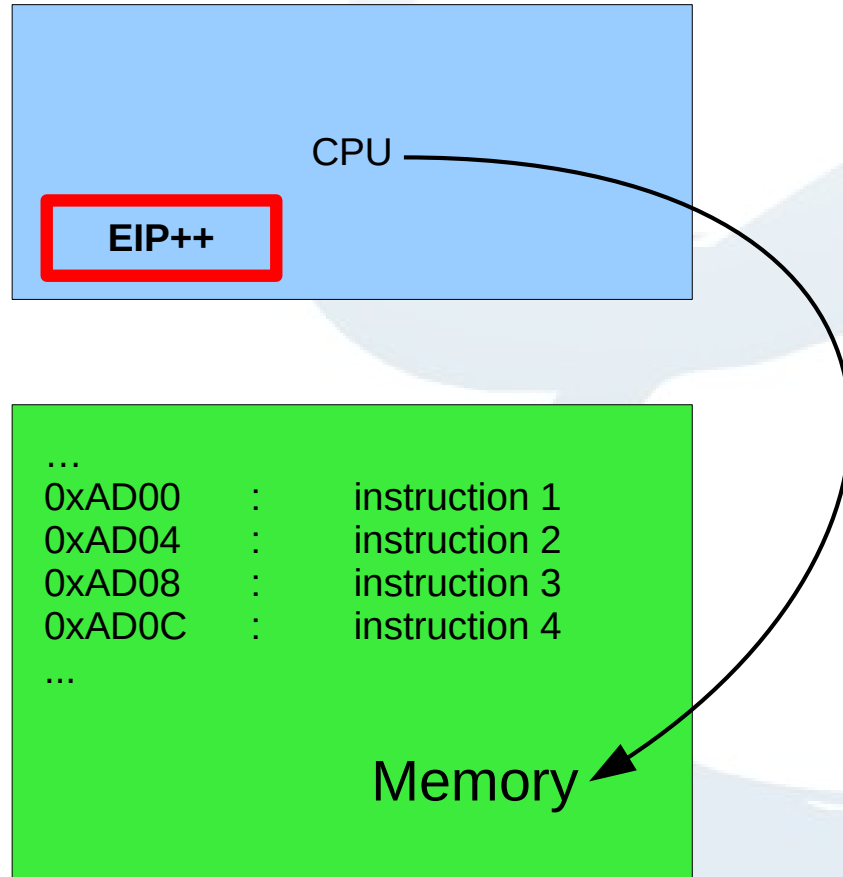


- Read instruction from mem at address contained in EIP ⁽¹⁾ register (Fetch)
- Decode instruction
- Execution



⁽¹⁾ Extended Instruction Pointer

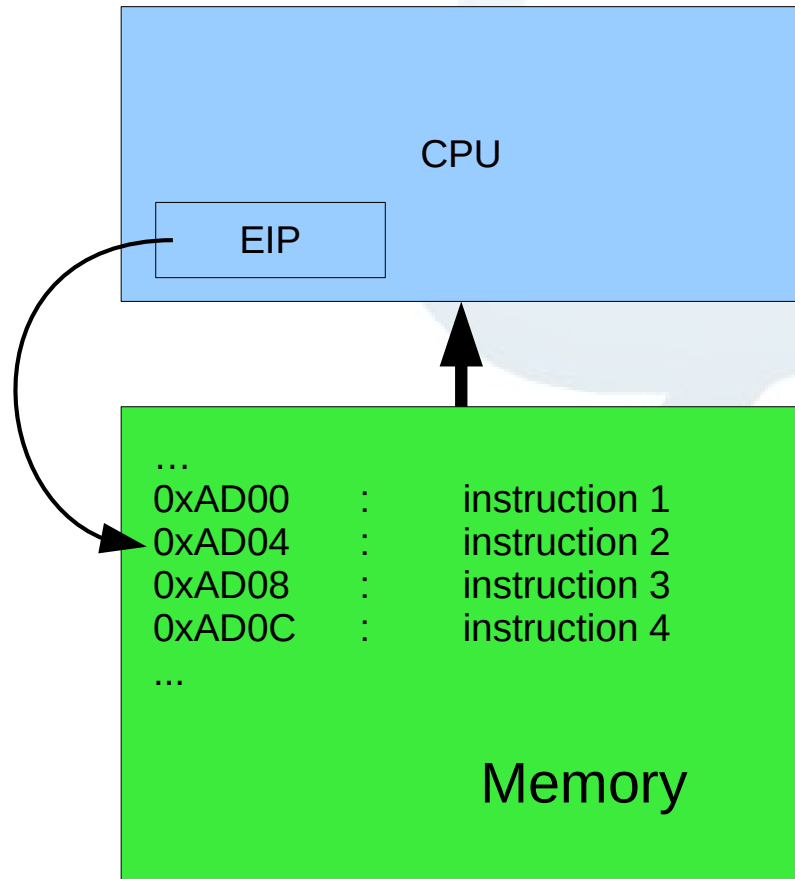
Running a program



- Read instruction from mem at address contained in EIP⁽¹⁾ register (Fetch)
- Decode instruction
- Execution
- Write Back & EIP++

⁽¹⁾ Extended Instruction Pointer

Running a program



- Read instruction from mem at address contained in EIP ⁽¹⁾ register (Fetch)
- Decode instruction
- Execution
- Write Back & EIP++

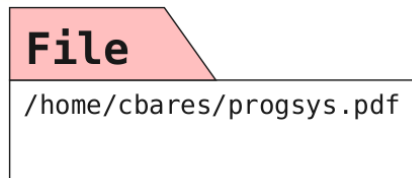
=> Next cycle...

Interaction between programs, memory and peripherals

- Memory:
 - Read data
 - Write data
 - Run instructions depending on data in memory
 - Run other program stored in memory
 - ...
- Peripherals:
 - Read data (eg Keyboard)
 - Write data (eg Screen)
 - Run instructions in case of interaction (eg power off button)
 - ...

Operating system roles

- Manage programs (tasks)
 - Start, stop, running, etc
 - Program's integrity, data security
 - Communication between programs
- Manage memory
 - Allocation
 - Permissions, security
- Manage hardware
 - Interface layer between programs and hardware
 - Handle hardware event
(eg: interrupt signals)

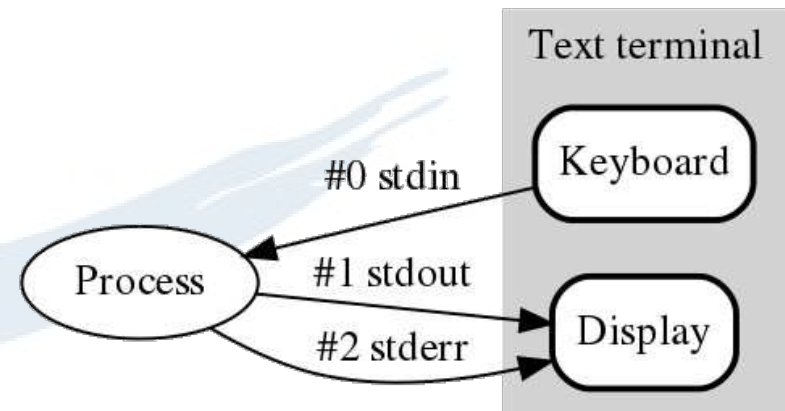


Managing Tasks

Execution context

- When a task is running, the OS setups its context
 - Process ID attribution (pid)
 - CPU Registers (stack pointer, instruction pointer...)
 - Loading libraries into memory
 - Access to files (file descriptors)
 - Environment variables
 - Command line arguments (char **argv)
 - Threads table
 - ...

Thread →



Managing Tasks

Thread →

Context switching

On multi-tasks OS

- Switching from one process to another:
 - Save the old context (CPU Register, address space)
 - Process scheduling (choose new thread to run)
 - **Restore the new context** (CPU Register, address space)
 - Start the new thread in its context
- Securing executions:
 - Program security (bugs)
 - Data security (unauthorized access)

Managing Tasks

Thread →

Communication between programs

OS provides several communication channels:
IPC (InterProcess Communication)

- Signals
- Shared memory
- Pipes
- Sockets
- Semaphores
- Message queue

Managing Memory

Communication between programs

OS uses Address Space to:

- Allocation
- Permissions & isolation
- Memory mapping
- Swapping to disk
- Shared memory: library & data



Managing Hardware

- Accessing hardware
 - Read data stored on hard drive
 - Access to the keys pressed on a keyboard
 - Write some characters on the screen
 - Print a document
 - Communicate through network

Peripherals are very varied

File

/home/cbares/progsys.pdf

The OS acts as an **unified access interface**
between programs and peripherals
=> Simplicity gain for program writing

Managing Hardware

Interruption management

- Hardware
 - Clock step
 - End of data transfer
 - Read/write head positioning
 - Emergency stop button
- Software
 - End of timers
 - Task commutation
 - Exception raise (division by zero, register overflow, segfault...)

File

/home/cbares/progsys.pdf

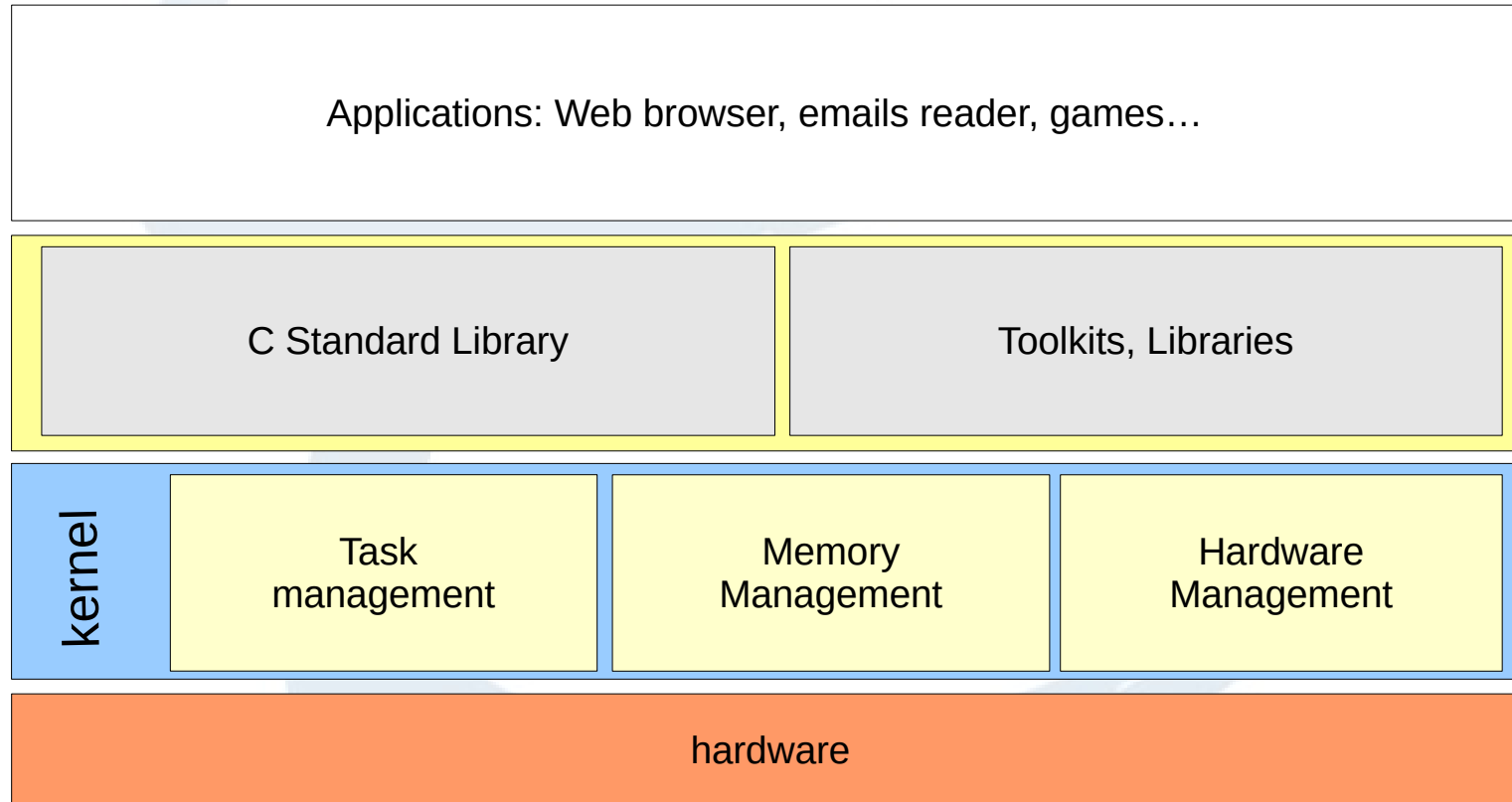
Managing Hardware

Securing critical sections

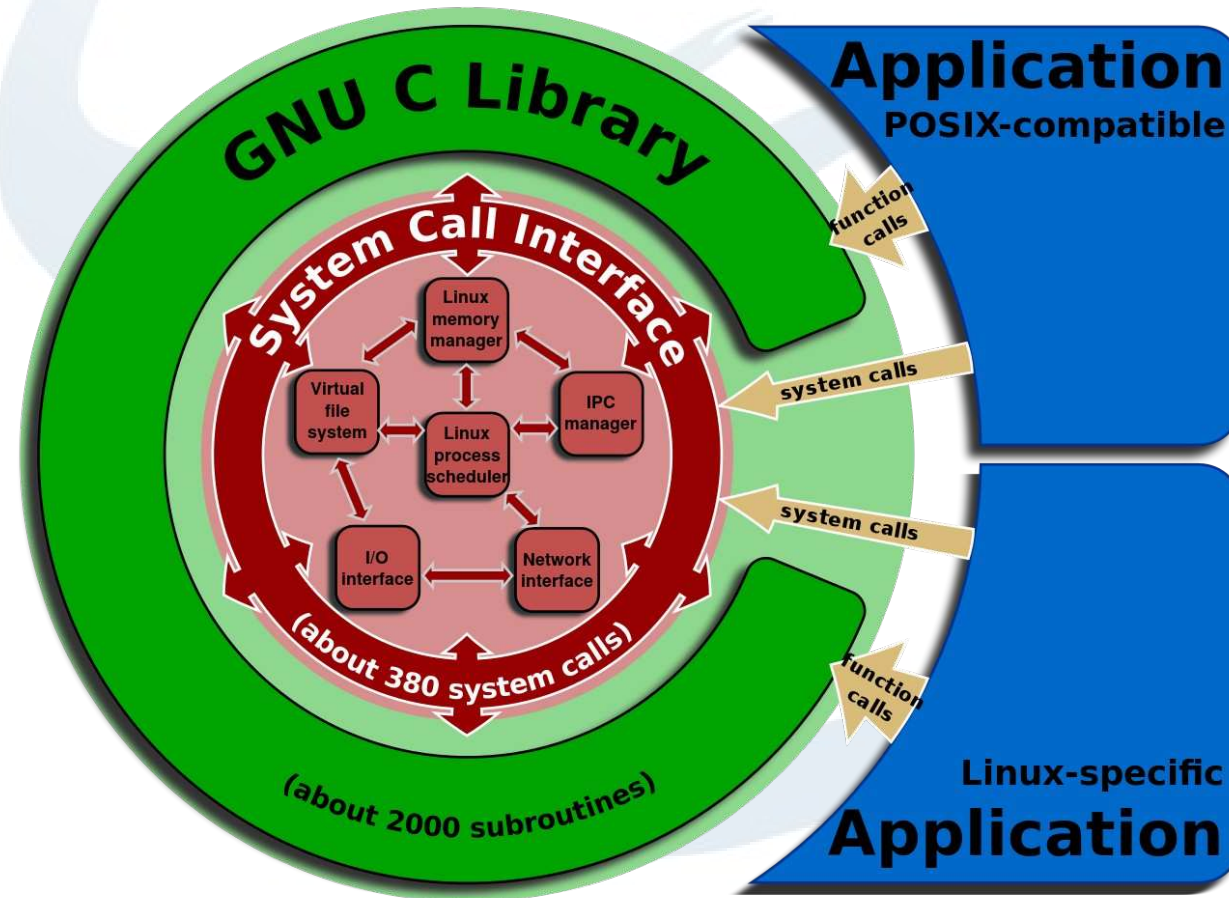
- The OS manages how requests to peripherals are handled and scheduled
- For example, 2 programs printing a file should not end with an entanglement of text coming out of the printer

OS Anatomy

Operating System



OS Anatomy: Linux Case



OS families

By use :

- **Desktop:** reactivity, many different programs, low constraints
- **Real time:** predictability, few known programs, strict constraints
 - Industries, finances
 - Automotive, aeronautic
- **Embedded:** very few known programs, very constrained (limited memory, low power consumption...)
 - Automotive, aeronautic
 - Robotic
 - Sensors network
 - Healthcare

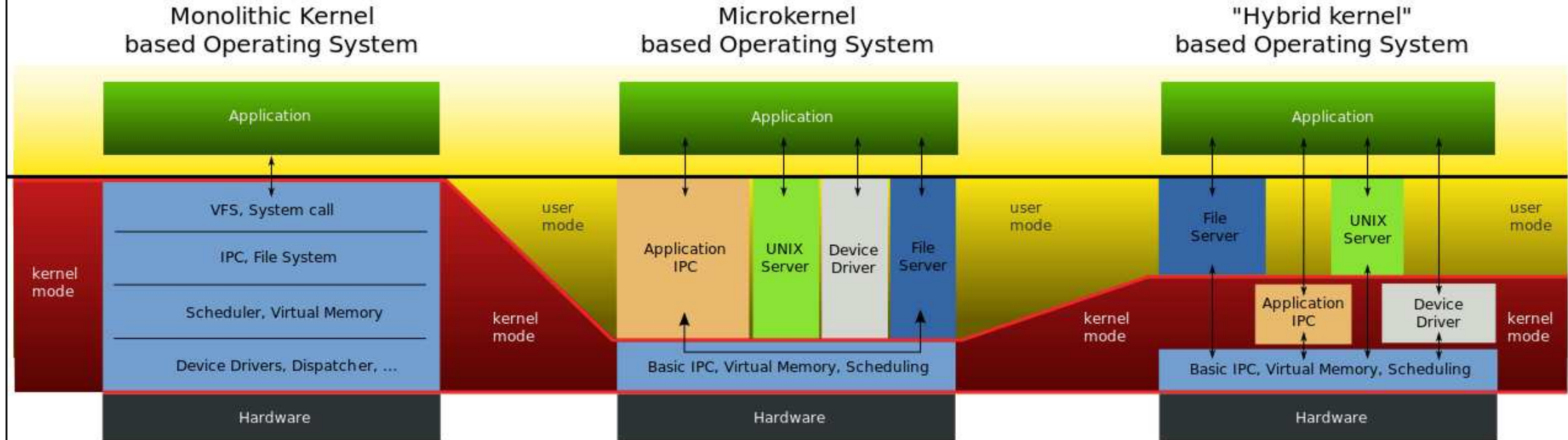
OS families

Family	Capabilities	Jobs	Example
Mainframe	I/O capacity	Batch, transaction processing, timesharing	OS/390, Linux
Server	Share hard/soft-ware resources among users	Print service, file service, Web service	Solaris, Linux, Windows Server
Personal	Reactivity, user-friendly	Provide a large number of programs	Linux, Windows, Mac
Handheld	Reactivity, user-friendly	Telephony, multimedia, games	Palm OS, Android, IOS
Embedded	No untrusted software	Small number of functions for one device	QNX, VxWorks
Sensor Node	Networks low energy	Transmit data	TinyOS, RIOT
Real-Time	Predictability	Meeting deadlines services	VxWorks, FreeRTOS
Smart Card	Small size, powered (reader, induction) & limited memory	Electronic payment	Java Card, Multos, Camille

Kernel types

- Monolithic: everything is in the kernel
 - task management
 - device drivers
 - memory management
 - network protocols, IPC...
- Ex: Linux, Solaris, BSD
- Microkernel: only basics are done by the kernel
 - task management
 - memory management
 - basic IPC
- Other things are done by programs called services
- Ex: Mach, L4, GNU/Hurd, MacOS X (hybrid)

Comparison



Modular kernels

- Monolithic, but
 - Possibility to add/remove functionality while running (ex: device driver)
 - Smaller (only necessary stuff is running)
 - Less safe (need to validate all drivers that will be loaded)
- Available in Linux since version 1.2 (march 1995)

Download Vanilla Kernel: <https://www.kernel.org/>

Explore Linux sources: <https://elixir.bootlin.com/>

Common OS features

- Multitasking:
 - Several programs can run at the same time
- Multi-user:
 - Several users can use the OS at the same time
- Multi-architecture:
 - OS can be run on a wide variety of architectures

Multitasking

- Several tasks can run at the same time:
 - Handle task creation and destruction
 - Handle access to resources in acceptable time
 - Scheduling (for CPU access)
 - Critical resources
 - Data integrity (permissions, rights and so on)

Multi-user

- Several users on the same system:
 - Handle critical resources
 - Integrity of users data
 - Handle communication between users (IPC)

Multi-arch

- Different hardware configuration
 - Independence of programs to the architecture (eg POSIX norm, ...)
 - Being able to run on several different arch (eg power, x86, arm, sparc, ...)

same source code

=>

Several hardware architectures

History

- **69 : Ken Thomson & Denis Ritchie**
 - First version of UNIX
 - PDP7/9; kernel 16Ko; Process 8Ko, Files 64Ko
- **72 : Kernighan & Ritchie: C Programming language**
- **73 : UNIX rewritten in C**
 - Process management
 - File management
 - Generic I/O

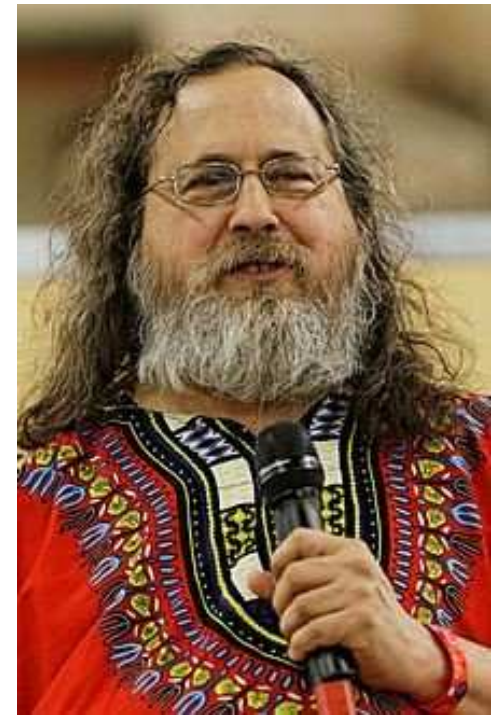


History

- 75 : First porting of UNIX
- 76 : 8/16 bits microprocessors
- 77 : **Thomson** at Berkeley Univ
 - 500 installations
- 79 : Porting to VAX and IBM
- 82 : Sell by ATT
 - Unix-based and Unix-like (100 000 install.)

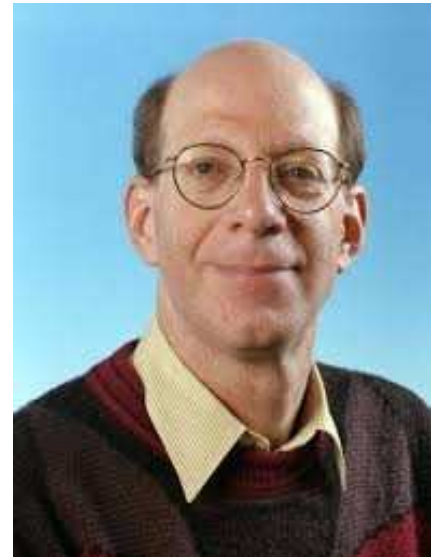
History

- 83 : Projet GNU par **Richard Stallman**
 - 84 : GNU C Compiler (GCC)
 - 85 : GNU Manifest, birth of FSF
 - 87 : GNU C Library (glibc)
 - 88-89 : GNU Bash (shell)
 - 89 : GPL version 1
 - 90 : Starting of GNU/Hurd (micro-kernel for GNU)



History

- 84 : Normalization efforts
 - DEC (Ultrix)
 - Gould (UTX)
 - HP (HP-UX)
- 85 : MINIX by **A. Tanenbaum** (μ kernel)
- 88 : standard desktop system
 - SUN/BULL/IBM...
- 89 : First free BSD (K. Thompson)



History

- Sep 91 : **Linux 0.01**
by Linus Tordvalds
(inspired by Minix)
- Oct 91 : Linux 0.03 (bash & gcc)
- Dec 91 : Linux 0.10 (first external contrib)
- **Jan 92 : Linux 0.12 (virtual mem, GPL
licenced)**
- Mar 92 : linux 0.95 (init/login, X)



History

- **93 : NetBSD and FreeBSD**
- **Mar 94 : Linux 1.0 (production stable, features comparable to UNIX)**
- Mar 95 : Linux 1.2 (several architectures, loadable modules)
- Jul 96 : Linux 2.0 (Multi processor, Tux)
- Jan 99 : Linux 2.2



History

- **99 : MacOSX (hybrid kernel)**
- 01 : Linux 2.4
- **03 : Linux 2.6 (preemptible kernel)**
- 07 : Linux 2.6.23 (CFS scheduler)
- 08 : Linux 2.6.26 (Kgdb)
- **21 Jul 2011 : Linux 3.0.0 (13M sloc)**
- **12 Dec 2016: Linux 4.9 (15M sloc)**
- Lastest: 6.10.10 (2024-09-12) (last thursday!)

Linux features

- Monolithic kernel (like most Unices)
- Modules : functionality can be added or removed (only SVR4.2 and Solaris can do this too)
- Kernel threads : some kernel features are independents
- Native multithreading support
- Preemptible kernel: a process can be interrupted in kernel mode
- Multi-CPU support (currently up to 8192 on x86_64)
<https://elixir.bootlin.com/linux/v6.0.2/source/arch/x86/Kconfig#L986>
- Many filesystems
- Many supported arch (26 + 9 with older versions)
- Small & compact (thanks to modules)
- Efficient
- Free (Free as in free speech, not as in free beer)

Distributions

- Complete OS based on Linux kernel:
 - Linux kernel
 - C library and compiler (mostly gcc)
 - Bootloader (grub, lilo...)
 - Init system to start the system (networking, printing, user interfaces...)
 - Shell (bash, dash, ksh...)
 - Package manager (apt, rpm, portage...)
 - Package repository loaded with verified programs
 - Package update policy

Distribution families

- Deb: Debian, Ubuntu and derived
 - .deb packages (`apt-get install ...`)
 - Enormous distributions come from Debian
 - Free software oriented
- RPM: RedHat, Fedora, SuSE and derived
 - .rpm packages (`yum install ...`)
 - RedHat : enterprise (\$\$), Fedora (free)
- Slackware
 - The Great Old One (packages *.tgz)
- Sources based: Gentoo, Linux From Scratch (LFS), ...
 - Compile all softwares from sources
 - Gentoo : portage package manager probably the best ever
 - LFS: compilation howtos in pdf



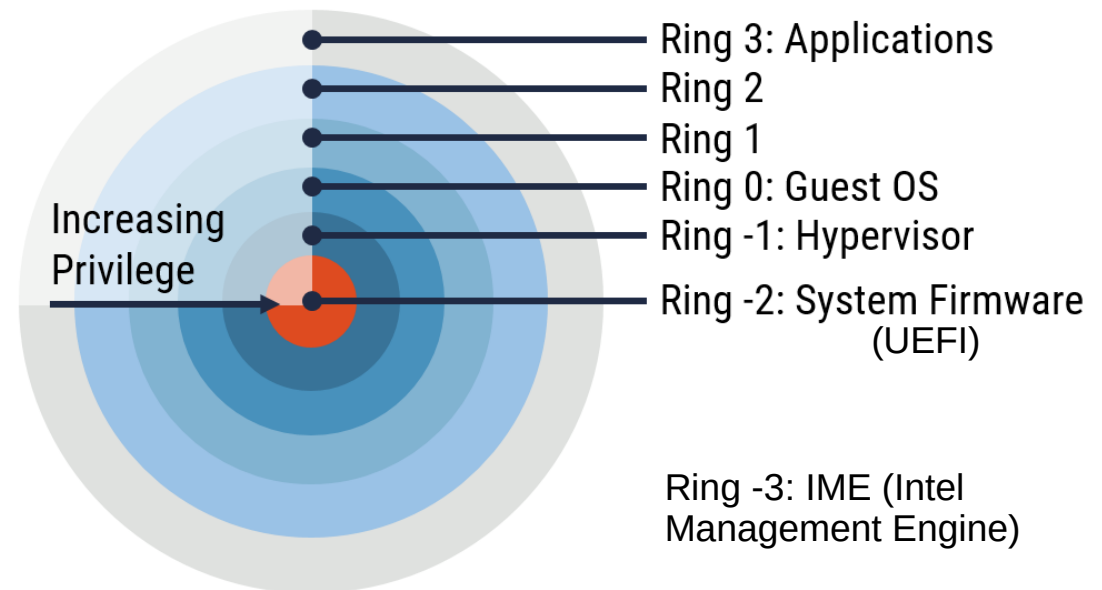
Fundamental features

- **Process** : instance of a running program and its associated context
- **Files** : in Unix world, everything is files: devices, documents, kernel interfaces...
- **Permissions** : files have different permission levels (user, group, other)
- **Reentrant** : several processes can be in kernel mode at the same time (no global variables, no side effects)

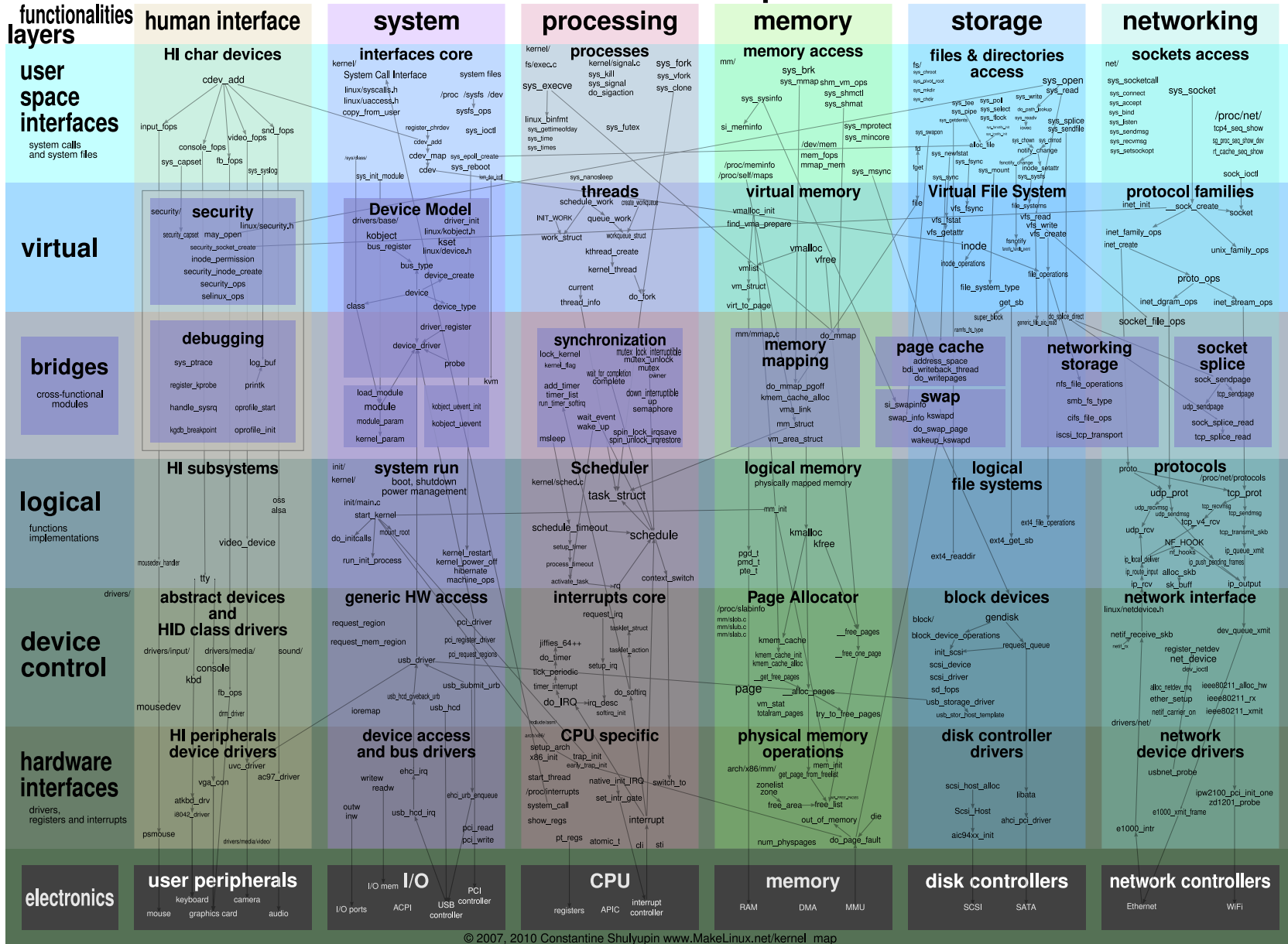
Fundamental concepts

Execution modes (hardware feature)

- User mode: instructions of the running program
 - Reserved memory zone
 - Specific context (stack pointer, eip...) = Ring 3
 - Restricted permissions
- Kernel mode (supervisor):
 - All memory zones
 - Specific context = Ring 0
 - All permissions
 - Accessed by interrupt (soft/hard)
- Hypervisor mode
 - Virtualisation = Ring -1



2.6.36



User/Kernel Mode

- Using the computer is divided into 2 execution modes:
 - *Kernel mode (Ring 0)*
 - *User mode (Ring 3)*
- Kernel mode is for... kernel programs (yes, it is!)
- User mode is for user programs
 - Administrator (root)
 - Services (printing, web server...)
 - Real users (mail, office, games, devel, ...)

Modes & Permissions

Memory is restricted to modes

- User mode applications cannot access to
 - Kernel memory zones
 - Memory zones of other applications
 - Most peripherals
- Kernel mode can access everything:
 - Memory
 - Devices
 - Instruction set

=> most processors have execution modes

Acess to kernel mode

Memory zones of the kernel are accessible only in kernel mode

- Not accessible in user mode
- Access to kernel mode by an interrupt
- System switch to privileged mode
- Kernel handles interrupt
- Back to user mode and continue user application

Switch to kernel mode

- Accessing kernel mode is done
 - By hardware **interrupts**
 - Switch to supervisor mode and execution of interrupt procedures
 - On abnormal execution: **exceptions**
 - System raises an exception, switch to supervisor mode and execution of exception procedures
 - When programs ask services by a **System Call**
 - Software interrupt raised by a calling application
 - Switch to supervisor mode and execution of procedures attached to the system call

Interruptions

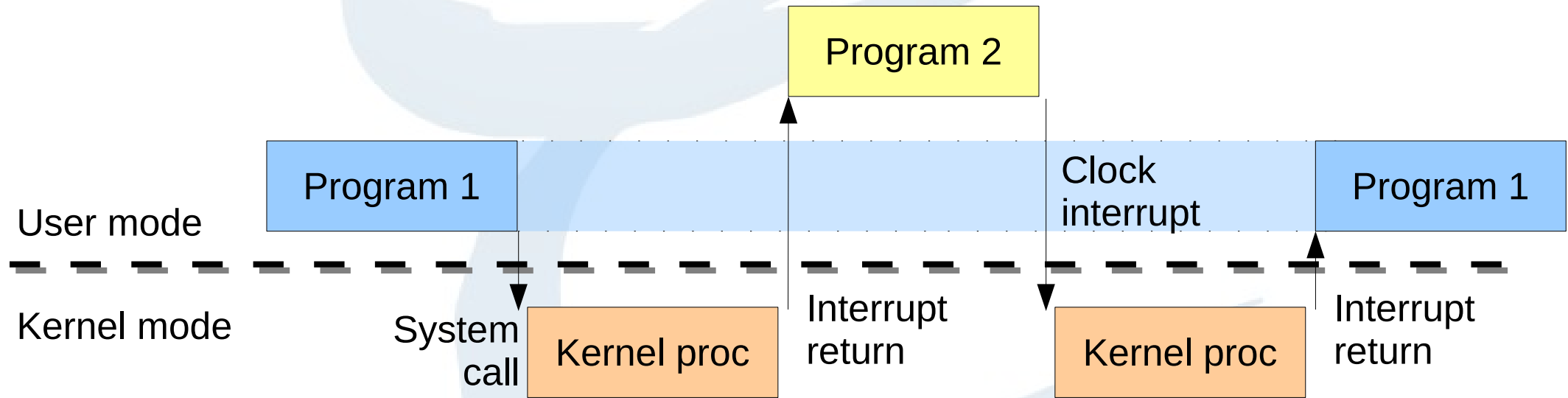
- Signal raised by
 - A hardware peripheral
 - Occurs at any time (asynchrone)
 - Procedure handling these are called drivers
 - Processor instruction (soft interrupt)
 - Special instruction allowing to switch to supervisor
 - Occurs at clock time (synchrone)
- Procedures related to interruptions are initialized at system boot
- Part of kernel code

Exceptions

- Processor responses to incorrect instructions
 - Interrupt raised by the processor
 - Signals errors in the running procedure (ex permission fault, calculation error)
 - Allow the kernel to handle errors
 - For example, allocate demanded resources (program asking for a non-allocated memory zone)

Interrupt handling

- When an interrupt occurs, the processor execute a procedure initialized at boot time



System calls

- Are functions that:
 - Are called in user mode
 - Are executed in supervisor mode
 - Return in user mode (in the calling program)
- The system call is asked by a unique software interruption (int 0x80)
 - Does the switch to supervisor
 - System calls are identified by numbers
 - The number of the system call is loaded in a register
 - A table allows to know which procedure to call depending on the number present in the register

Linux 0.01

- System calls are available in glibc
<https://elixir.bootlin.com/linux/0.01/source/include/unistd.h#L128>

```
#define __syscall0(type,name) \
type name(void) \
{ \
type __res; \
__asm__ volatile ("int $0x80" \
: "=a" (__res) \
: "0" (__NR_##name)); \
if (__res >= 0) \
return __res; \
errno = -__res; \
return -1; \
}
```

67 syscalls in linux 0.01:

<https://elixir.bootlin.com/linux/0.01/source/include/unistd.h#L60>

Kernel control path

- *kernel control path* is the sequence of instructions executed by the interrupt procedure
- Several kernel control path can be entangled (preemptible kernel)
 - Ex: a process asks for a non available resource, another process is executed and asks for a resource also
 - An interrupt is raised during a kcp

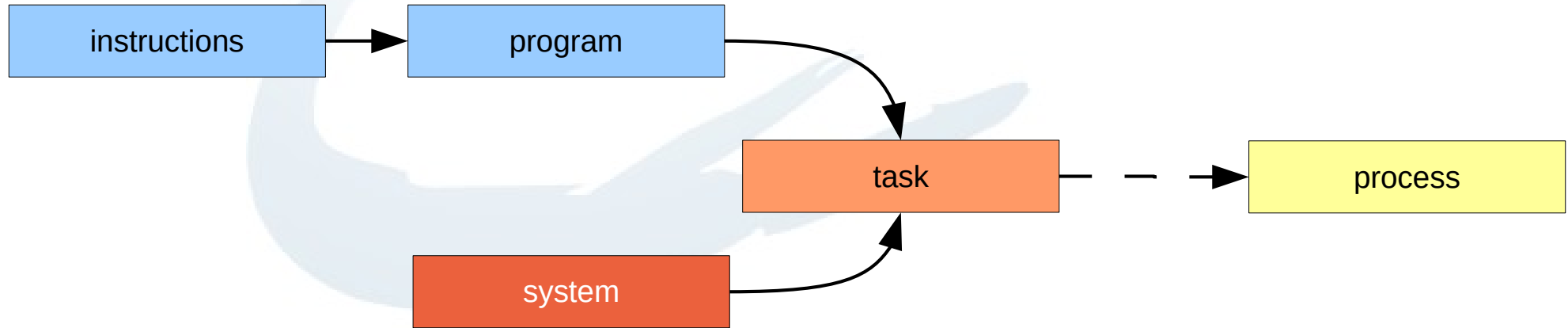
=> Need for a reentrant kernel

3IS – Operating Systems – Processes

2024 – Christophe Barès
christophe.bares@ensea.fr

Contributions: David Picard,
Arnaud Revel, Mickaël Maillard

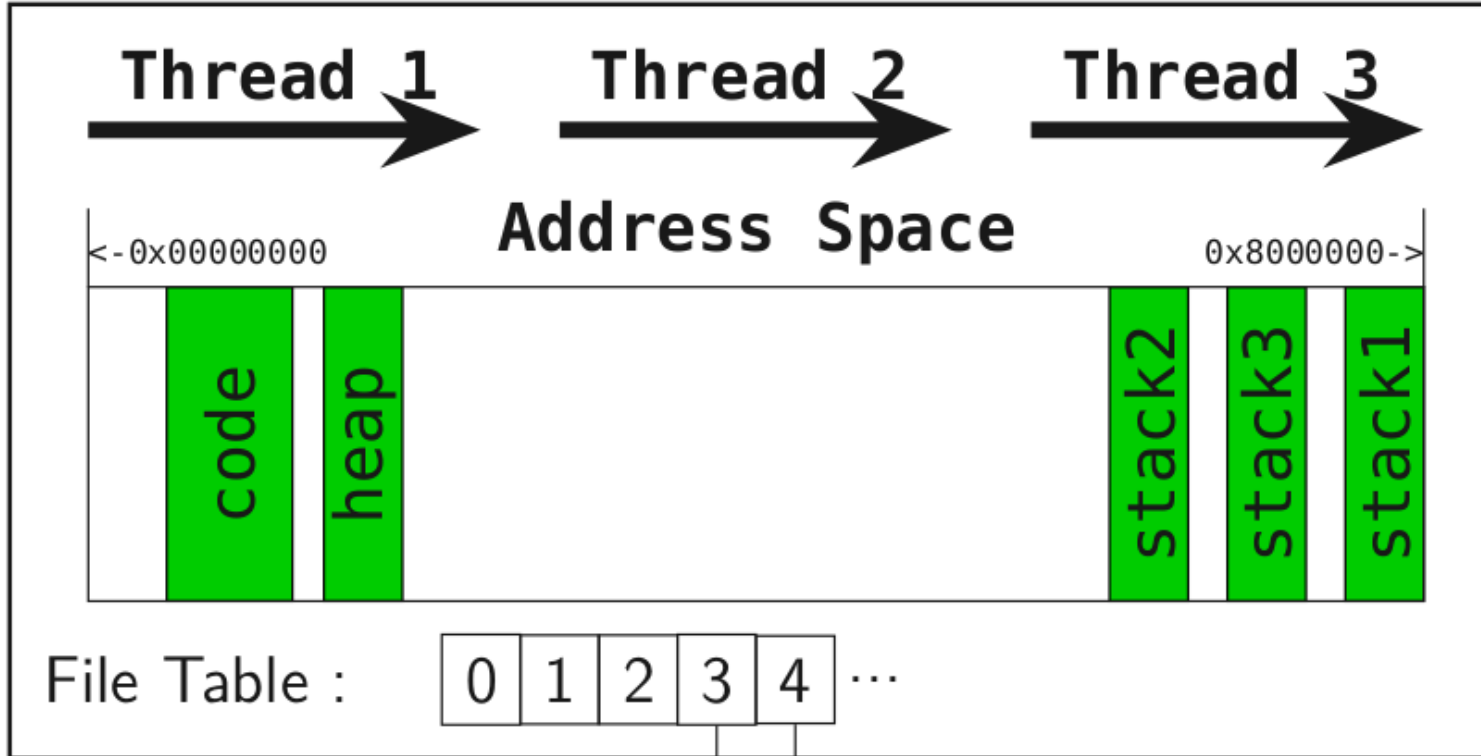
Definitions



- A **program** is a **sequence of instructions**
- A **task** is a **program** with all system structures necessary to its execution (**static**)
- A **process** is a running **task** (**dynamic**)

Abstraction²

Processus

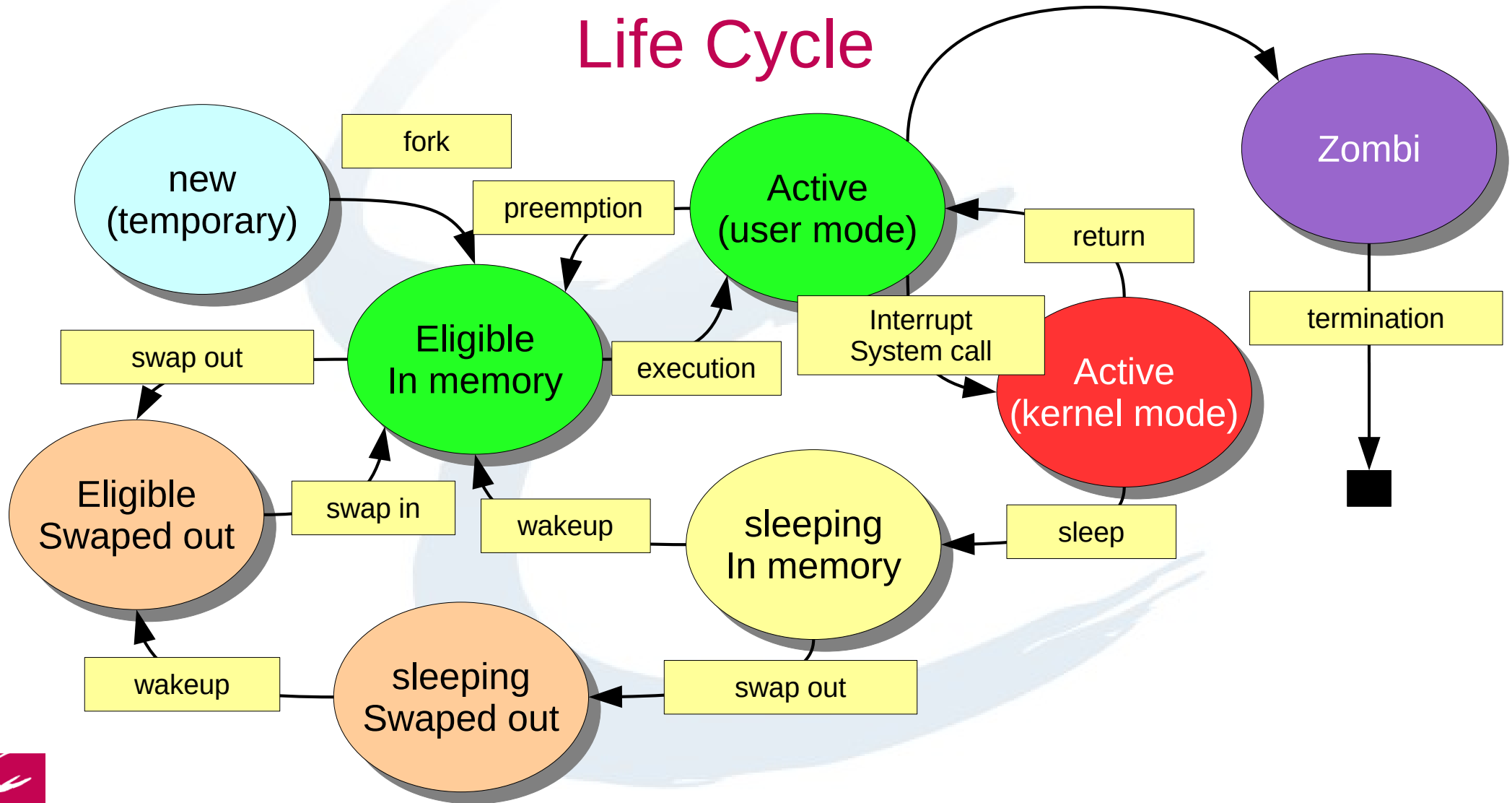


Process states

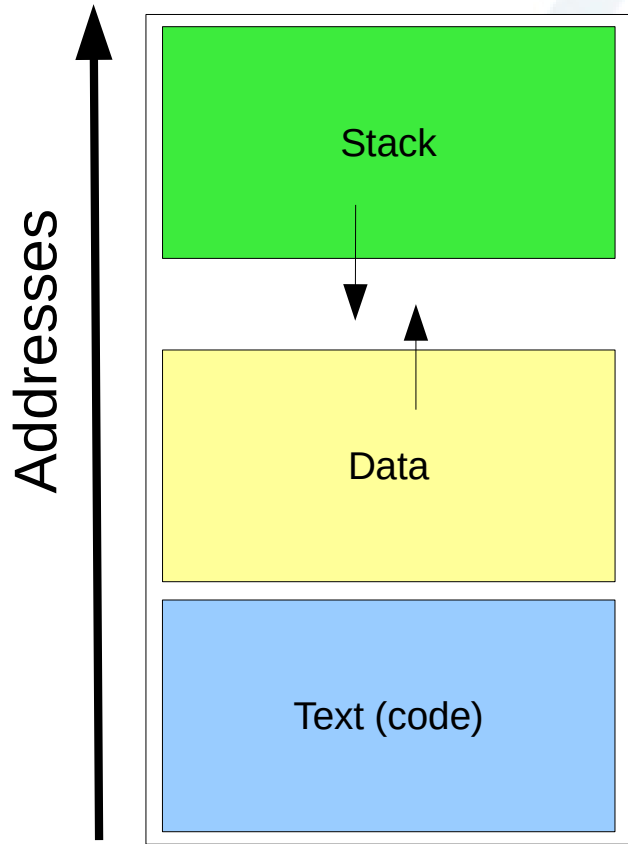
`include/linux/sched.h`

- `TASK_RUNNING` : running or scheduled for running.
- `TASK_INTERRUPTIBLE` : suspended (sleeping) and waiting for a condition. Can be interrupted.
- `TASK_UNINTERRUPTIBLE` : same as previous, but can't be interrupted.
- `EXIT_ZOMBIE` : terminated, but the parent process did not read termination information
- `EXIT_DEAD` : final state, being suppressed by the system.

Life Cycle



Process creation: fork()



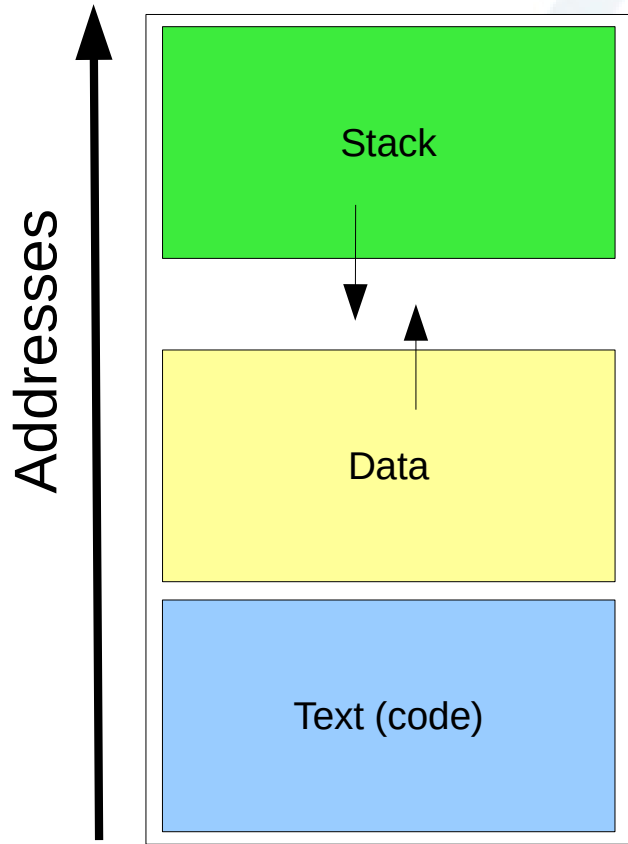
```
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>

int main(int argc, char **argv)
{
    int pid, status;

    pid = fork();

    if(pid != 0)
    { /* parent code */
        printf("I am the parent %d\n", getpid());
        printf("my child is %d\n", pid);
        wait(&status);
    }
    else
    { /* child code */
        printf("I am the child %d\n", getpid());
    }
    return 0;
}
```

Process creation: fork()



```
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>
```

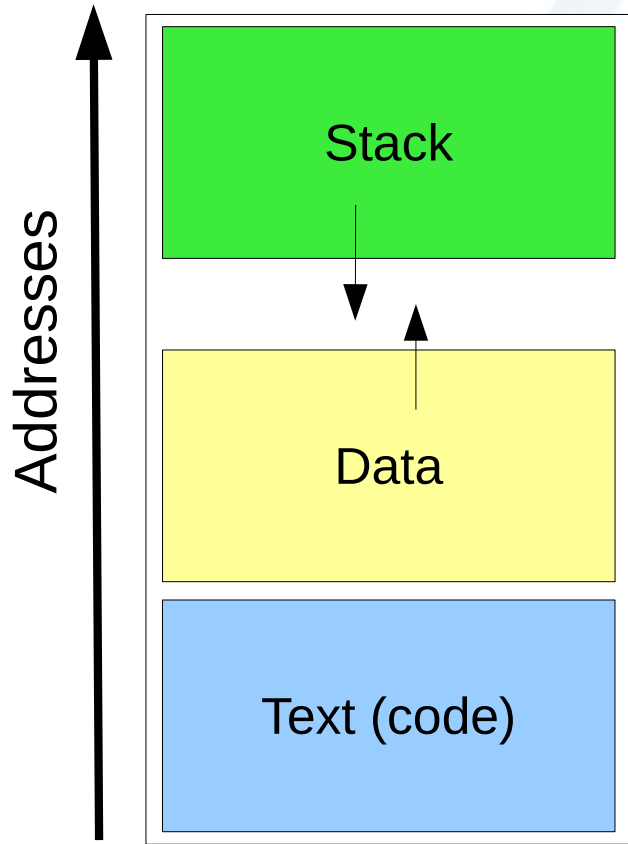
```
int main(int argc, char **argv)
{
    int pid, status;
```

```
    pid = fork();
```

```
    if(pid != 0)
    { /* parent code */
        printf("I am the parent %d\n",
               "my child is %d\n", getpid(), pid);
        wait(&status);
    }
    else
    { /* child code */
        printf("I am the child %d\n", getpid());
    }
    return 0;
}
```

PID :
Process identifier, unique
and attributed by the system.

Process creation: fork()



```
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>
```

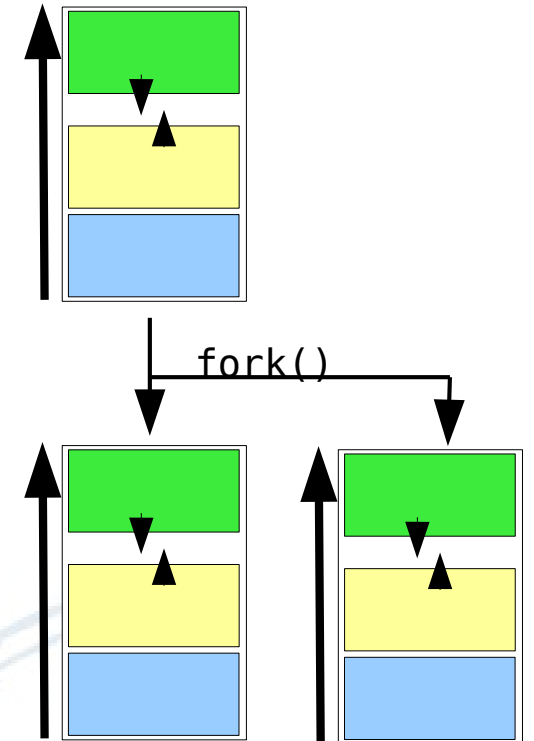
```
int main(int argc, char **argv)
{
    int pid, status;
```

```
    pid = fork();
```

```
    if(pid != 0)
    { /* parent code */
        printf("I am the parent %d\n", getpid());
        printf("my child is %d\n", pid);
        wait(&status);
    }
    else
    { /* child code */
        printf("I am the child %d\n", getpid());
    }
    return 0;
}
```

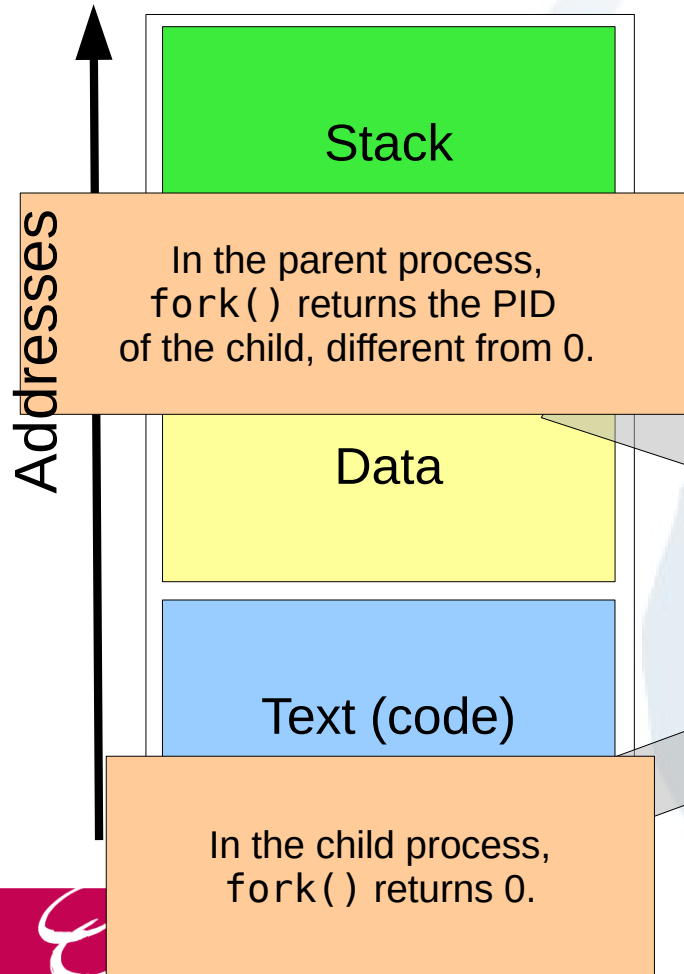

fork() system call

- Copy the current process
 - inherited:
 - Initialized variables
 - environment
 - open files
 - signals
 - GID (Group Identifier)
 - Not inherited:
 - PID (child has a new one)
 - PPID (Parent PID)
 - Timers, locks... (see man 2 fork)



It's like cloning in biology

Process creation: fork()



```
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>
```

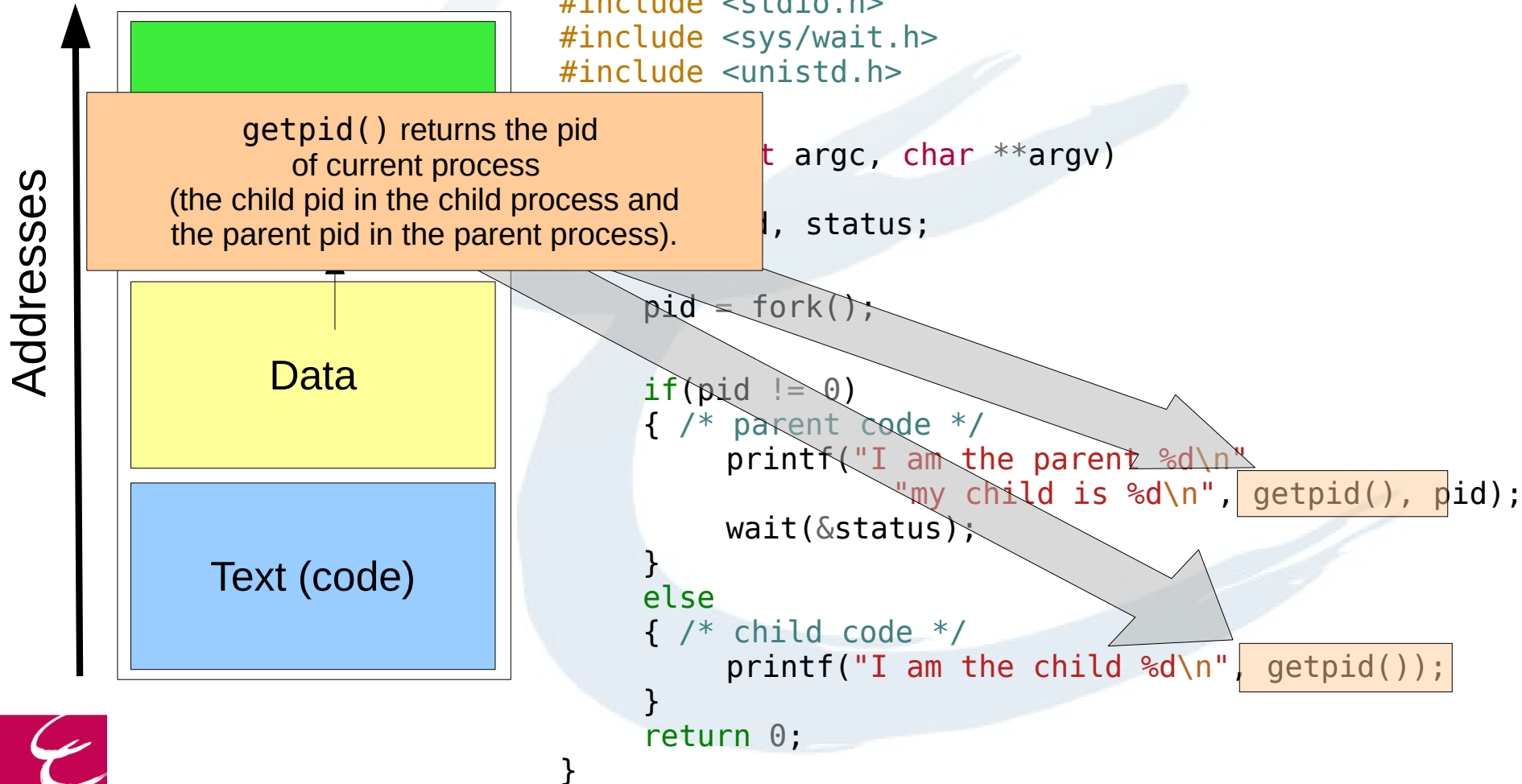
```
int main(int argc, char **argv)
{
    int pid, status;

    pid = fork();
```

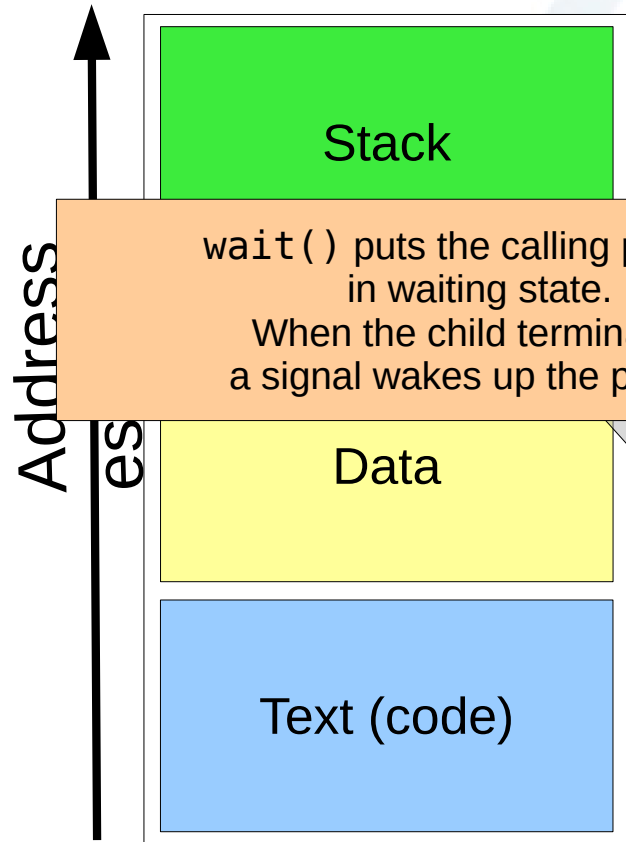
```
    if(pid != 0)
    { /* parent code */
        printf("I am the parent %d\n", getpid());
        printf("my child is %d\n", pid);
        wait(&status);
    }
```

```
    else
    { /* child code */
        printf("I am the child %d\n", getpid());
    }
    return 0;
}
```

Process creation: fork()



Process creation: fork()



wait() puts the calling process
in waiting state.
When the child terminates,
a signal wakes up the parent.

```
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>
```

```
int main(int argc, char **argv)
```

```
    pid, status;
```

```
    fork();
```

```
    if(pid != 0)
```

```
    { /* parent code */
```

```
        printf("I am the parent %d\n"
```

```
               "my child is %d\n", getpid(), pid);
```

```
        wait(&status);
```

```
    }
```

```
    else
```

```
    { /* child code */
```

```
        printf("I am the child %d\n", getpid());
```

```
    }
```

```
    return 0;
```

```
}
```

What is executed by each process?

parent

```
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>

int main(int argc, char **argv)
{
    int pid, status;

    pid = fork();

    printf("I am the parent %d\n",
           "my child is %d\n", getpid(), pid);
    wait(&status);
}

return 0;
```

child

```
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>

int main(int argc, char **argv)
{
    int pid, status;

    pid = fork();

    printf("I am the child %d\n", getpid());

    return 0;
}
```

What is executed by each process?

parent

```
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>

int main(int argc, char **argv)
{
    int pid, status;

    pid = fork();

    printf("I am the parent %d\n",
           "my child is %d\n", getpid(), pid);
    wait(&status);
}

return 0;
```

child

```
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>

int main(int argc, char **argv)
{
    int pid, status;

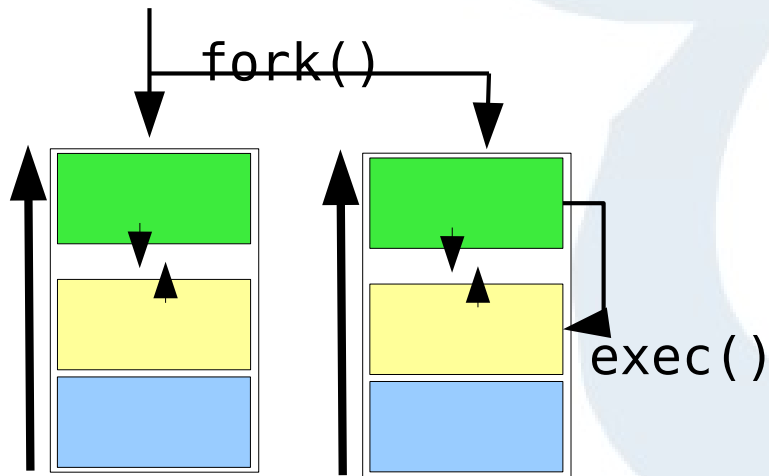
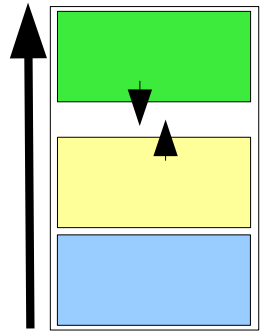
    pid = fork();

    printf("I am the child %d\n", getpid());
    return 0;
```

Code replacement: `exec()`

- `exec()` system call allows to replace the content of the `.text` memory zone (instructions) of a process
- `exec()` overwrite the running program
 - EIP is pointing to the beginning of the `.text` zone (main function)
 - To launch a new program, one only has to call `fork()` and then to call `exec()` to replace the child instructions with the desired ones.

Code replacement: exec()



```
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>

int main(int argc, char **argv)
{
    int pid, status;
    pid = fork();

    if(pid != 0)
    { /* parent process */
        wait(&status);
    } else { /* child process */
        printf("I am the child process %d\n",
               getpid());
        exec("/usr/bin/ls", "ls", NULL);
        printf("you will never see this!");
    }

    return 0;
}
```

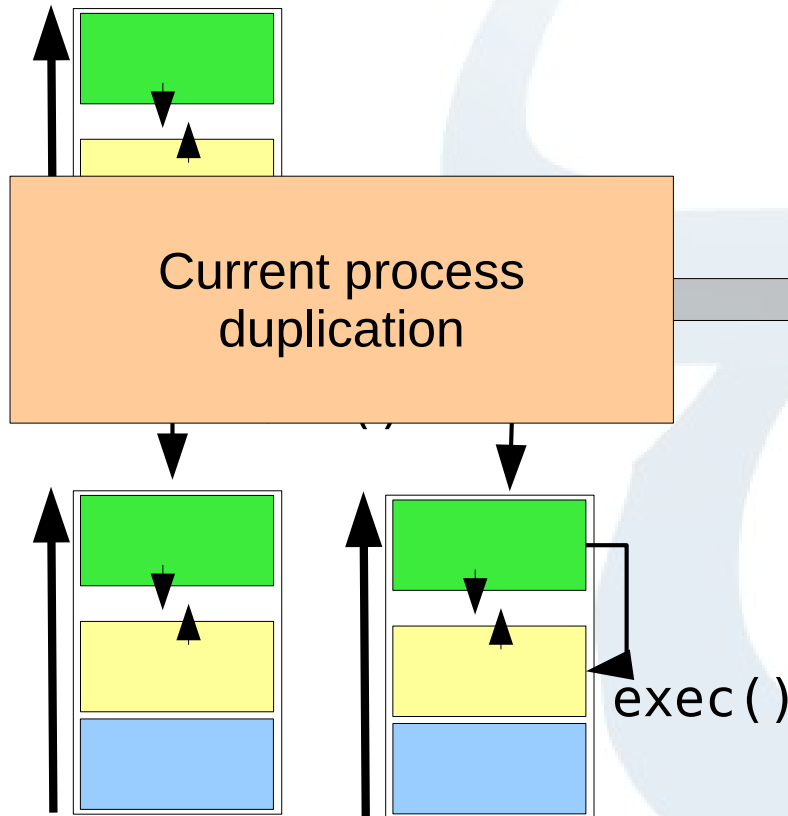

Code replacement: exec()

```
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>
```

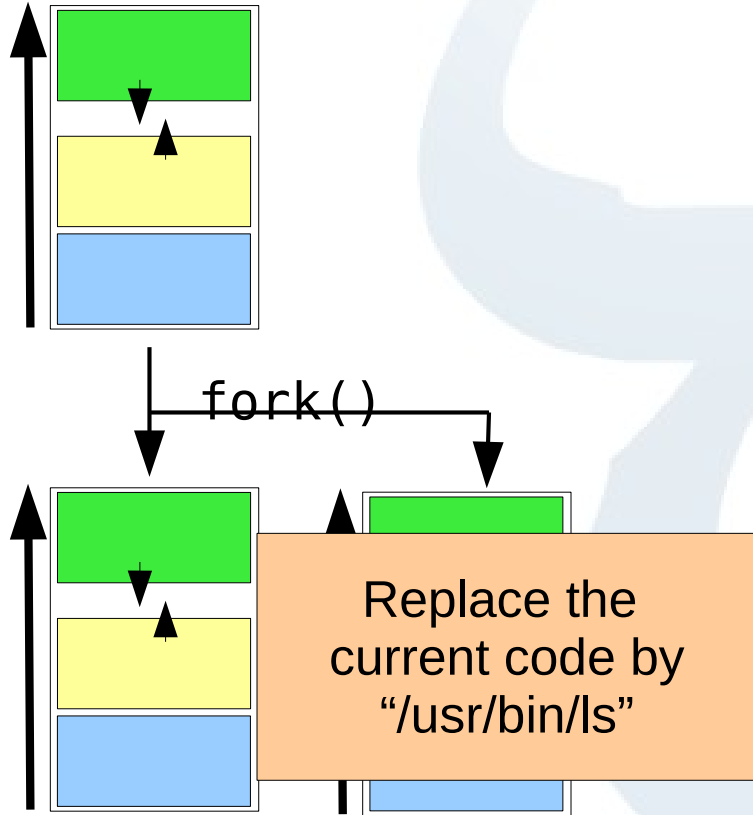
```
int main(int argc, char **argv)
{
    int pid, status;
    pid = fork();
```

```
    if(pid != 0)
    { /* parent process */
        wait(&status);
    } else { /* child process */
        printf("I am the child process %d\n",
               getpid());
        exec("/usr/bin/ls", "ls", NULL);
        printf("you will never see this!");
    }
```

```
    return 0;
}
```



Code replacement: exec()



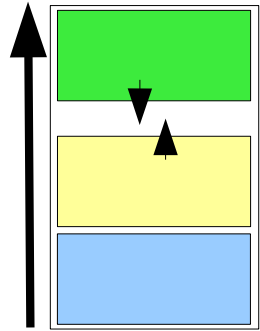
```
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>

int main(int argc, char **argv)
{
    int pid, status;
    pid = fork();

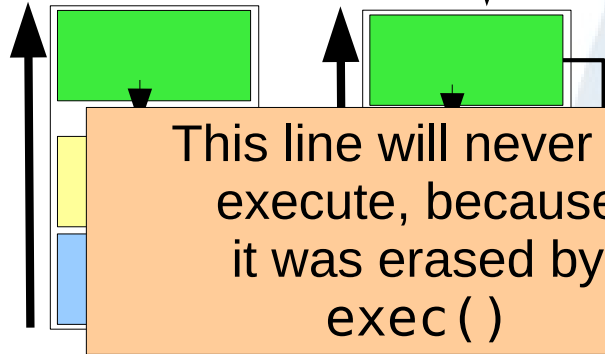
    if(pid != 0)
    { /* parent process */
        wait(&status);
    } else { /* child process */
        printf("I am the child process %d\n",
               getpid());
        exec("/usr/bin/ls", "ls", NULL);
        printf("you will never see this!");
    }

    return 0;
}
```

Code replacement: exec ()



fork ()



```
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>
```

```
int main(int argc, char **argv)
{
    int pid, status;
    pid = fork();
```

```
    if(pid != 0)
    { /* parent process */
        wait(&status);
    } else { /* child process */
        printf("I am the child process %d\n",
               getpid());
        exec("/usr/bin/ls", "ls", NULL);
        printf("you will never see this!");
    }
```

```
    return 0;
}
```

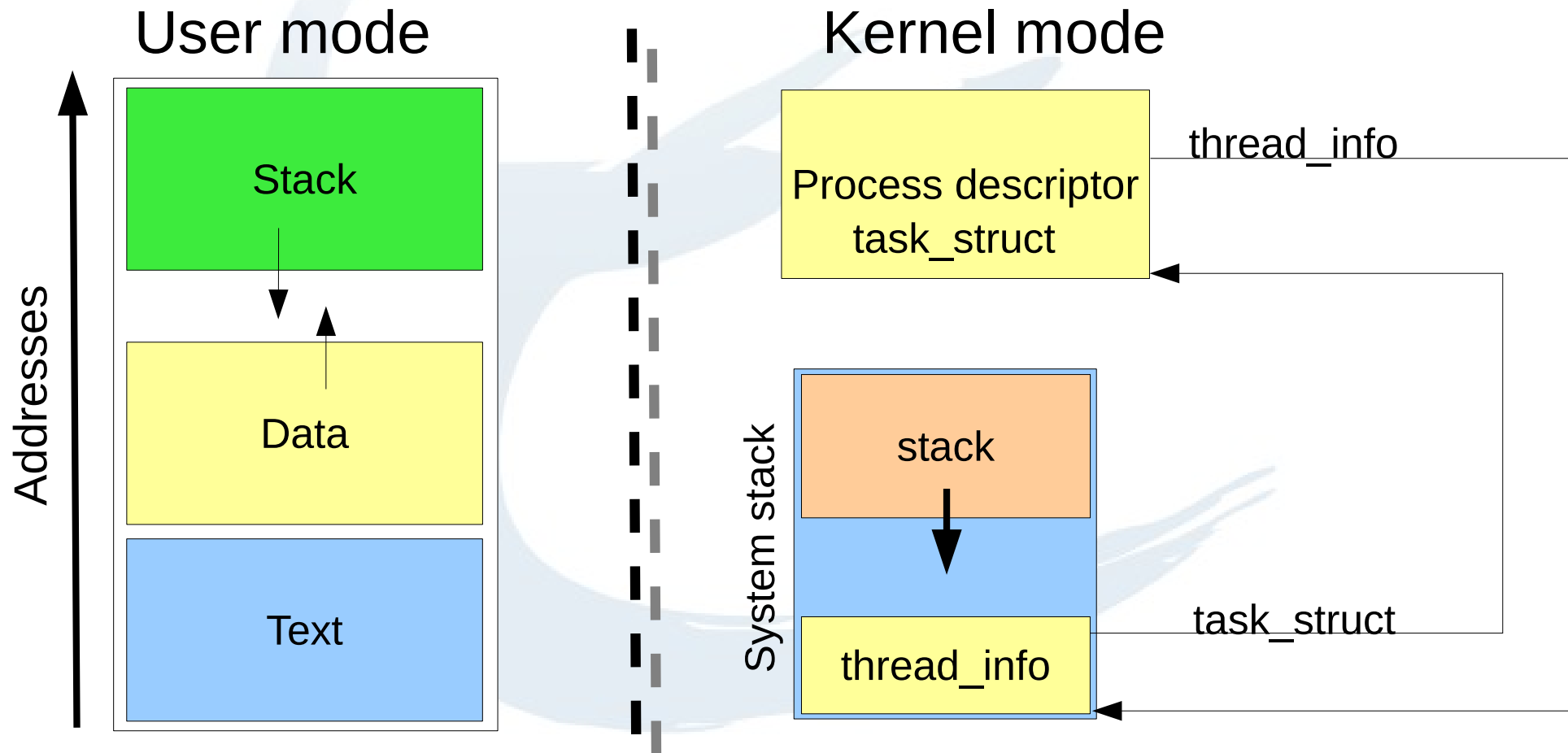
Associated structures

- each **process** is associated to:
 - A user context: process as seen by the user
 - A system context: process as seen by the kernel, in 2 parts:
 - Process descriptor
 - System stack

Process Control Block (task_struct in Linux)

- **task_struct** is C structure containing
 - Process information (pid, state, ...)
 - Information on parent/child relations
 - Information on open files
 - ...
- To manage processes, there is a double linked list of **task_struct** in the kernel
- There's also a hashtable of process descriptors indexed by pid

Structures



Process descriptor for 0.01 kernel kernel/sched.h

```
struct task_struct {
/* these are hardcoded - don't touch */
    long state;           /* -1 unrunnable, 0 runnable, >0 stopped */
    long counter;
    long priority;
    long signal;
    fn_ptr sig_restorer;
    fn_ptr sig_fn[32];
/* various fields */
    int exit_code;
    unsigned long end_code,end_data,brk,start_stack;
    long pid,father,pgrp,session,leader;
    unsigned short uid,euid,suid;
    unsigned short gid,egid,sgid;
    long alarm;
    long utime,stime,cutime,cstime,start_time;
    unsigned short used_math;
/* file system info */
    int tty;               /* -1 if no tty, so it must be signed */
    unsigned short umask;
    struct m_inode * pwd;
    struct m_inode * root;
    unsigned long close_on_exec;
    struct file * filp[NR_OPEN];
/* ldt for this task 0 - zero 1 - cs 2 - ds&ss */
    struct desc_struct0ss_struct tss;
};
```

Process descriptor for 0.01 kernel kernel/sched.h

```
struct tss_struct {
    long back_link; /* 16 high bits zero */
    long esp0;
    long ss0;        /* 16 high bits zero */
    long esp1;
    long ss1;        /* 16 high bits zero */
    long esp2;
    long ss2;        /* 16 high bits zero */
    long cr3;
    long eip;
    long eflags;
    long eax,ecx,edx,ebx;
    long esp;
    long ebp;
    long esi;
    long edi;
    long es;         /* 16 high bits zero */
    long cs;         /* 16 high bits zero */
    long ss;         /* 16 high bits zero */
    long ds;         /* 16 high bits zero */
    long fs;         /* 16 high bits zero */
    long gs;         /* 16 high bits zero */
    long ldt;        /* 16 high bits zero */
    long trace_bitmap; /* bits: trace 0, bitmap 16-31 */
    struct i387_struct i387;
};
```

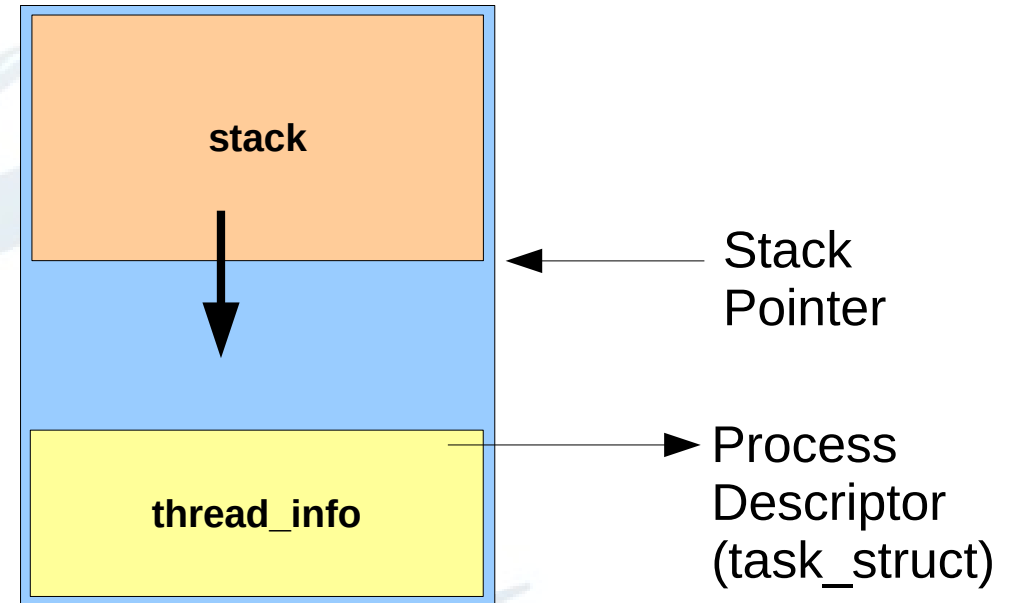

System stack

- Used to store the material context of a process
 - Everything set in the registers of the hardware relative to the current process
 - These registers cannot be shared among processes, so all states are saved in the system stack when a process switch occurs
 - When the process is scheduled, the context is restore from the system stack
 - Each process descriptor contains a pointer to the associated system stack

Union thread_union

- System stack on 2 pages (2 kB)
- A structure thread_info at the bottom
- thread_info contains a pointer to the Process Descriptor and some reserved space for well known registers

(for Linux<4)



```
union thread_union {  
    struct thread_info thread_info;  
    unsigned long stack[THREAD_SIZE/sizeof(long)];  
};
```

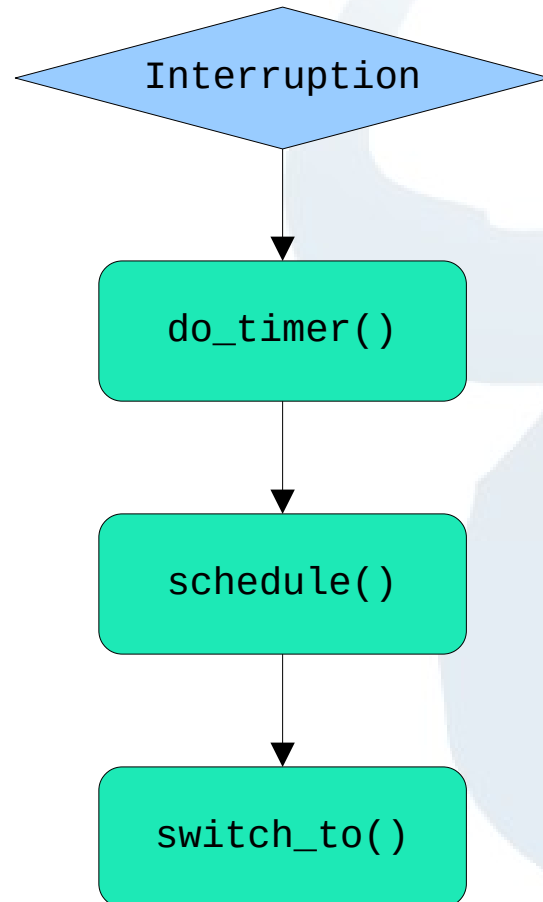
Scheduling

- Multitasking: several processes are running at the same time
- Simultaneity is simulated by time sharing
 - One running process is stopped to let another run
 - Time is split in quanta
 - Each quantum is attributed to a process
 - The attribution of quanta to processes is called “scheduling” and is performed by the “scheduler”

Scheduling

- Changing the current running process involves a **context switch**
 - In **cooperative multitasking**, this switch is initiated by the programs themselves (ex: windows pre 98, Mac Os 9)
 - In **preemptive multitasking**, (like linux) the switch is performed periodically by the kernel
- The switch is launched by interrupt 0 (clock interrupt), usually every 10 ms (HZ constant)
 - This interrupt calls the function `do_timer()` that calls `schedule()`
- Some processes are scheduled more often than other depending on some priority criteria
 - Priority field in `task_struct` , plus fine tuned algorithm

Scheduling



- Hardware Interrupt
- Clock Interrupt (int 0x00)
- System call (int 0x80)

Accumulate running time
Reduce current process counter
Call Scheduler

Among all TASK_RUNNING processes:
Elect the higher priority process if any
Or Idle and reset all processes counters

Switch to the new elected process

Function do_timer ()

/kernel/system_call.s

```
.align 2
_timer_interrupt:
    push %ds
    push %es
    push %fs
    pushl %edx
    pushl %ecx
    pushl %ebx
    pushl %eax
    movl $0x10,%eax
    mov %ax,%ds
    mov %ax,%es
    movl $0x17,%eax
    mov %ax,%fs
    incl _jiffies
    movb $0x20,%al
    outb %al,$0x20
    movl CS(%esp),%eax
    andl $3,%eax
    pushl %eax
    call _do_timer
    addl $4,%esp
    jmp ret_from_sys_call

    # save ds,es and put kernel data space
    # into them. %fs is used by _system_call

    # we save %eax,%ecx,%edx as gcc doesn't
    # save those across function calls. %ebx
    # is saved as we use that in ret_sys_call

    # EOI to interrupt controller #1

    # %eax is CPL (0 or 3, 0=supervisor)

    # 'do_timer(long CPL)' does everything from
    # task switching to accounting ...
```

```
void do_timer(long cpl)
{
    if (cpl)
        current->utime++;
    else
        current->stime++;
    if ((--current->counter)>0) return;
    current->counter=0;
    if (!cpl) return;
    schedule();
}
```

Jiffies: total count of clock interrupt

cpl: Current Privilege level (= ring)

Function schedule()

```
void schedule(void)
{
    int i,next,c;
    struct task_struct ** p;

    /* check alarm, wake up any interruptible tasks that have got a signal */
    for(p = &LAST_TASK ; p > &FIRST_TASK ; --p)
        if (*p) {
            if ((*p)->alarm && (*p)->alarm < jiffies) {
                (*p)->signal |= (1<<(SIGALRM-1));
                (*p)->alarm = 0;
            }
            if ((*p)->signal && (*p)->state==TASK_INTERRUPTIBLE)
                (*p)->state=TASK_RUNNING;
        }

    /* this is the scheduler proper: */
    while (1) {
        c = -1;
        next = 0;
        i = NR_TASKS;
        p = &task[NR_TASKS];
        while (--i) {
            if (!*--p)
                continue;
            if ((*p)->state == TASK_RUNNING && (*p)->counter > c)
                c = (*p)->counter, next = i;
        }
        if (c) break;
        for(p = &LAST_TASK ; p > &FIRST_TASK ; --p)
            if (*p)
                (*p)->counter = ((*p)->counter >> 1) +
                    (*p)->priority;

        switch_to(next);
    }
}
```

Context switch

- A running process needs the hardware to be in a specific state, called hardware context
- During a context switch, this hardware context is stored in kernel data structures
- Then, the context of the selected process is restored from these structures

Simplified macro switch_to

Let there be 2 processes **prev** and **next** :

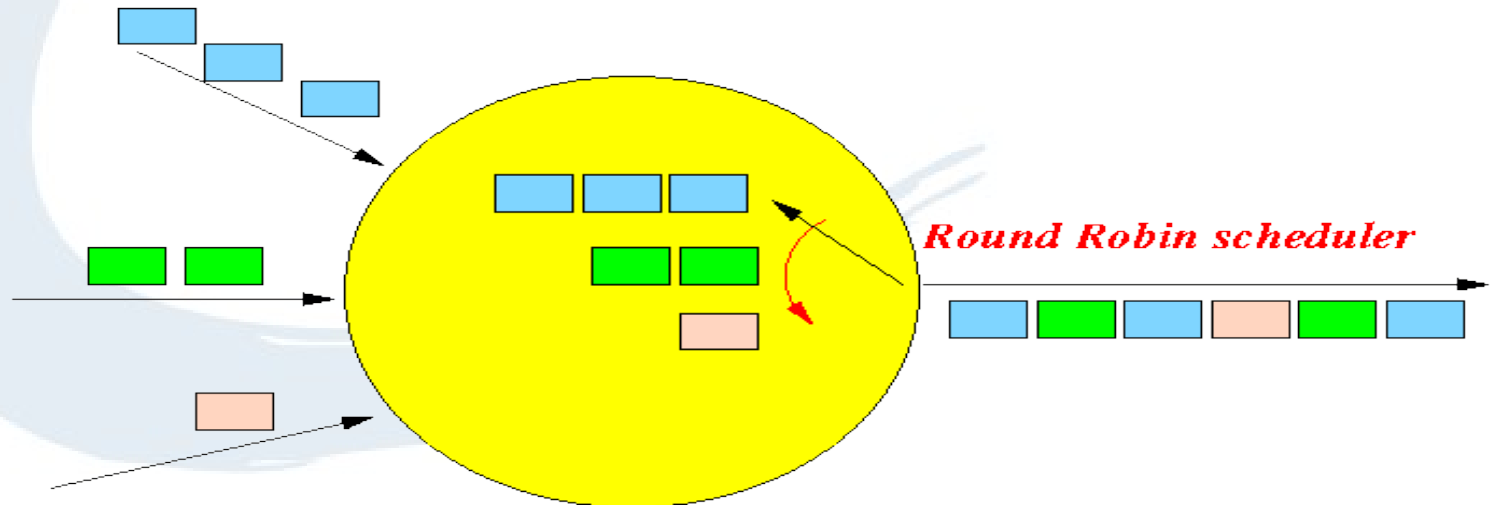
1. Store pointers to **prev** and **next** in 2 register
2. Store EBP (frame pointer) in the kernel stack of **prev**
3. Store ESP (stack pointer) in thread_info structure of **prev**
4. Load **next**->thread.esp in ESP (we now use the stack of **next**)
5. Store the address of label 1 in **prev**->thread.eip
6. Load **next**->thread.eip in EIP (instruction pointer) (we are now running **next**)
7. EIP contains the address of label 1, which is executed:
 - 1:
 - Load EBP from system stack
 - Load EFLAGS from system stack
8. Back to the running of **next**

Scheduling algorithms

- Linux kernel 0.01
 - `task_struct` defines 2 fields: `priority` and `counter`
 - Processes have a starting `priority` of 15
 - At each clock tick, `schedule` is called
 - `counter` is decremented
 - Process with the highest `counter` is scheduled

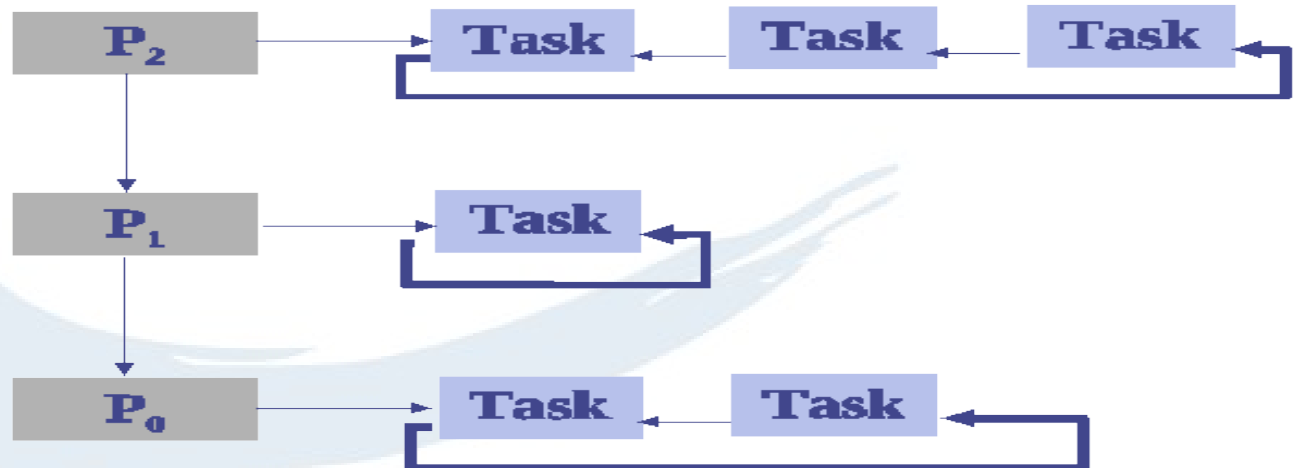
Round-robin

- All processes have the same priority
 - Stored in a circular list
 - The head process is scheduled
 - A quanta is assigned
 - Put at the end of the list
- Linux uses this scheduler for real-time processes



Priority based round-robin

- Processes are organized by priority lists
- Lists are selected based on priority
- A round-robin scheduling is applied to selected lists



$O(1)$ scheduler

- $O(1)$ scheduler used from experimental kernel 2.5.2 (jan 2002) until 2.6.22 (oct 2007)
- Developed by *Ingo Molnar* to address flaws and remarks in preceding scheduling algorithms
 - Time taken by schedule function should not depend on the number of process (avoid thrashing)
 - Reactivity should be better for interactive processes
 - Tend to “real-time”
 - Should take into account context
 - SMP (multi-processors)

$O(1)$ scheduler

- Different scheduling between
 - Real-time processes
 - Conventional processes
- Conventional processes split into 2 categories
 - Interactive processes
 - Background processes

$O(1)$ scheduler

Real time processes

- have highest priorities (0 to 99)
 - RT processes are always considered active
 - Hard scheduling regarding priority
- Inside a priority list, 2 types of scheduling
 - SCHED_FIFO: even if there are other processes in the list, the current process keeps running as long as he wants
 - SCHED_RR: Round robin scheduling in the list

$O(1)$ scheduler

Conventional processes

- Classification between interactive/batch is based on execution history (specifically the mean sleeping time)
 - A often sleeping process is prioritized by the scheduler
 - such a process is more likely to depend on I/O to peripherals, or is not an egoistic process consuming all available resources
- Complex heuristic to determine interactive/batch

Completely Fair Scheduler (CFS)

- Written to address the flaws in $O(1)$, in particular regarding SMP
 - $O(1)$ scheduler uses the mean sleeping time. However, sleeping process are more likely to generate preemptions and scheduling
 - Interactivity evaluation is biased when the system is heavily loaded. Not enough samples are available to have a correct estimate of interactiveness.
 - Attacks on the scheduler were possible

Principle

- CFS considers an ideal CPU where all processes can be executed at the same time, each one using a fraction of the computing resources
 - If there is only one process, it uses 100% of the processing power
 - For 2 processes, each would use 50%
- On a real processor, only one process can run
- During this time, other processes wait

Algorithm

- CSF tracks “fair play” disbalance in CPU allocation
- When a process is waiting for CPU, CFS computes the time it should have by dividing the waiting time by the number of waiting processes
- This value is the quantum the process has the right to use
- It is use to sort the processes in a red-black tree
- A static priority value is used to weight the resulting value
- The process with the highest fairness disbalance is scheduled

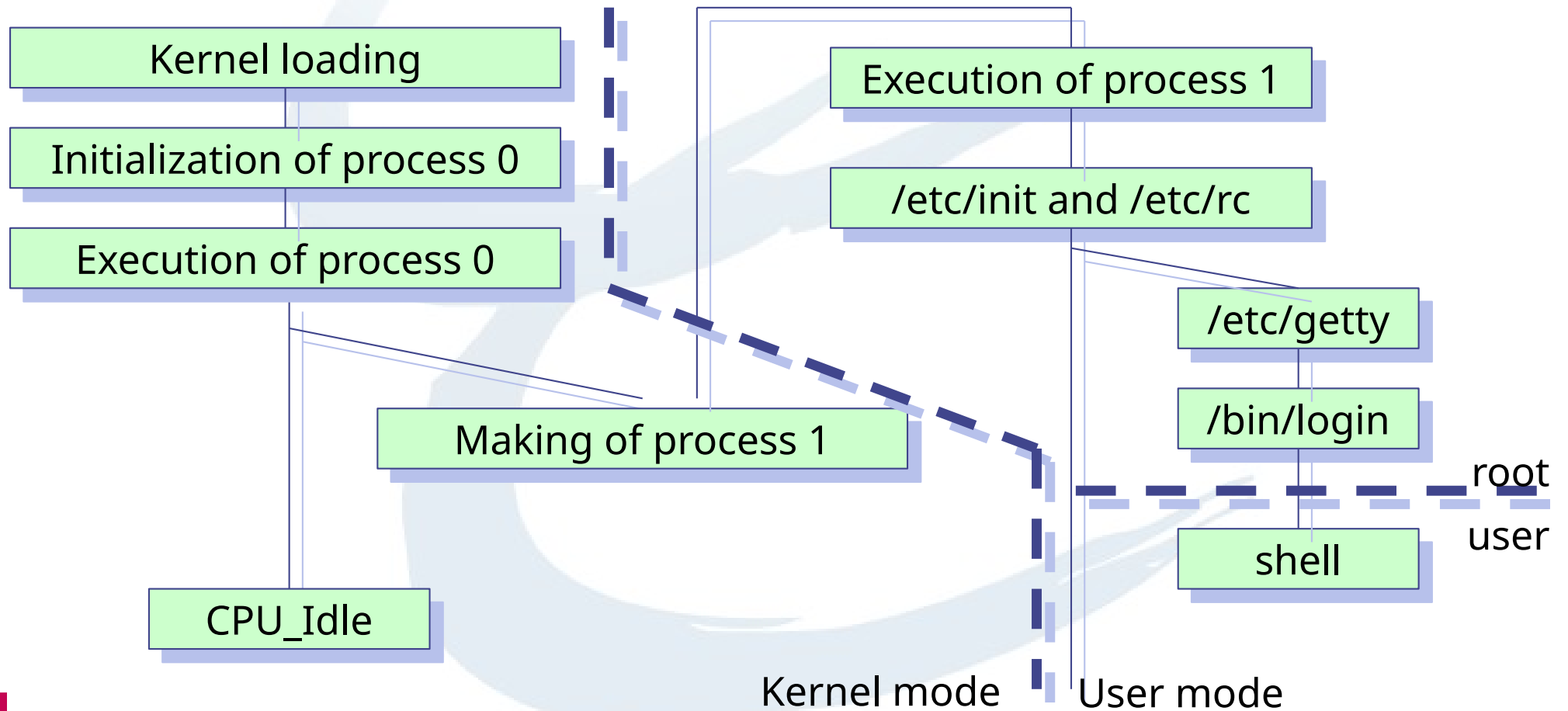
Scheduling modularization

- Since 2.6.23 scheduling is modularized
- Scheduling classes appear (`struct sched_class`)
- Each class has different process queues and scheduling policies
 - Principal scheduler (`kernel/sched.c`) forwards to implied modules
 - RT processes are handled by `kernel/sched_rt.c`
 - CFS is associated with types `SCHED_OTHER`, `SCHED_BATCH` handled by `kernel/sched_fair.c`
 - An idle module (`sched_idletask.c`) handles the idle task on each processor

Scheduling groups

- Starting from 2.6.24, fairness has been extended to users
- Let A and B be 2 users with respectively 2 and 48 processes
- Group scheduling extends fairness to users instead of processes
- In this example, each user benefits from 50% of CPU time: 50% for the 2 processes of A and 50% for the 48 processes of B
- Simple implementation: one CFS for groups, one CFS inside each group

init



start_kernel()

- `start_kernel()` is the function called at boot time
- Initialization of kernel structures (allocation of kernel data structure, memory allocators, schedulers, ...)
- Creates the process 0, called *swapper* (or *idle*), using function `kernel_thread()`
- Creates the process descriptor for *swapper*
- Creates then process 1, called *init*, using `kernel_thread()` by copying the *swapper*
- *init* invokes `exec()` to replace its code by the one of the startup program (often called *init* too)
- *init* becomes a standard user mode process
- Usually, *init* is able to launch all useful programs and services of the system (using `fork()` and `exec()`)

3IS – Operating System – Memory Management

2024 – Christophe Barès
christophe.bares@ensea.fr

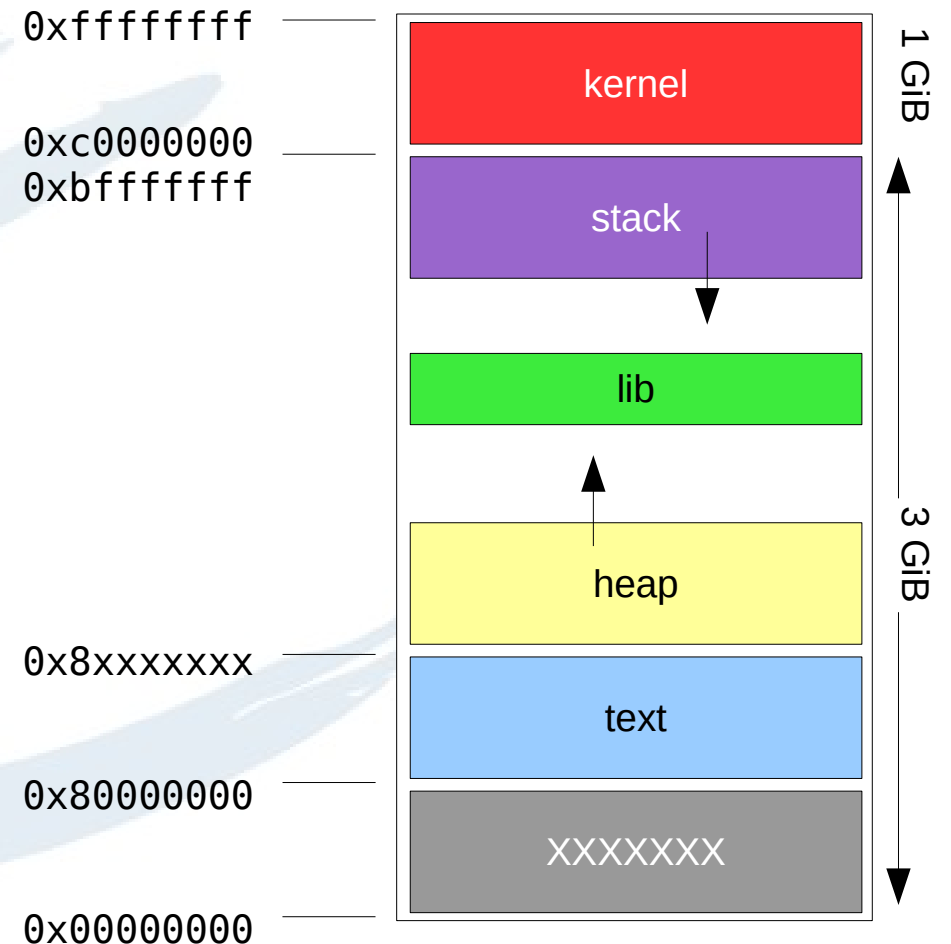
Contributions: David Picard,
Arnaud Revel, Mickaël Maillard

Process Address Space (PAS)

- Process address space is the whole memory as seen by the process
- Properties:
 - **Total**: if the address bus is 32 bits wide, then the process address space is $2^{32} = 4 \text{ GiB}$;
 - **Continuous**: every zone between $0x0...0$ and the highest address $0xF...F$ are accessible;
 - **Private**: except in explicit cases, only the process and the kernel have access to the memory.

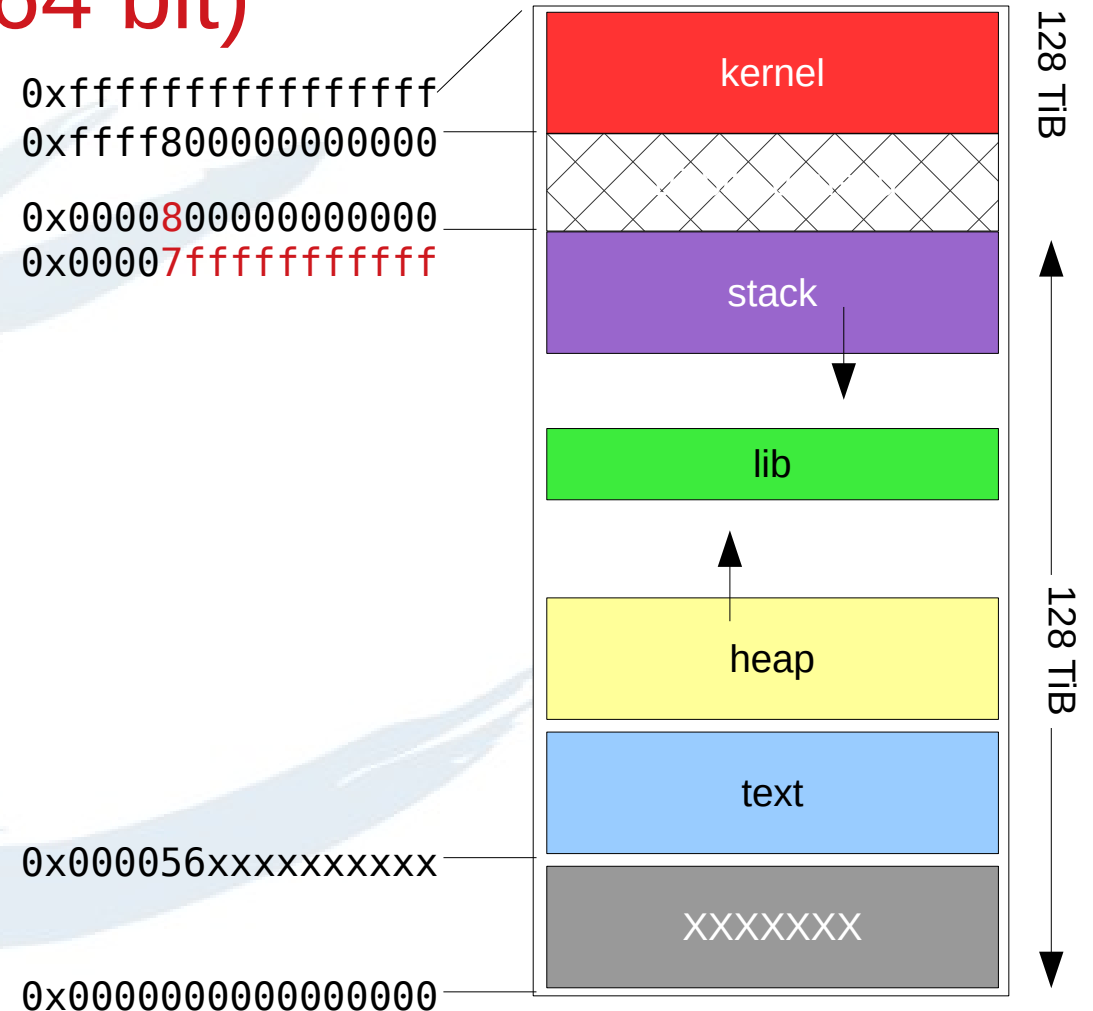
Slicing the process address space (Linux 32 bit)

- Addresses: 32 bits wide bus
- At least 5 zones
 - A reserved space
 - Code zone (often called text)
 - Data zone (heap)
 - Library zone
 - Stack
 - Kernel
- Kernel zone is only accessible by the kernel
- Code, Data and Stack are accessible in user mode



Slicing the process address space (Linux 64 bit)

- Addresses: 48 bits bus
- At least 5 zones
 - A reserved space
 - Code zone (often called text)
 - Data zone (heap)
 - Library zone
 - Stack
 - Kernel
- Kernel zone is only accessible by the kernel
- Code, Data and Stack are accessible in user mode

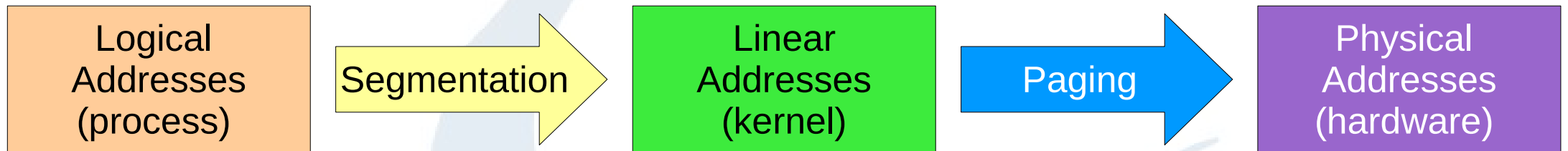


Several processes...

- For each process the addressing space has the same properties
- Unfortunately this is impossible to achieve:
 - **Total**: not all computer have 4 GiB of RAM (or even 256 TiB for x86-64)
 - **Private** and **continuous**: with several processes, it's either continuous or private.
- So in practice:
 - Kernel zone and reserved zone are shared among processes
 - The other parts are specific to each process

Virtual Memory

- Addresses in PAS are not real physical addresses
- 2 mechanisms:
 - Segmentation
 - Paging

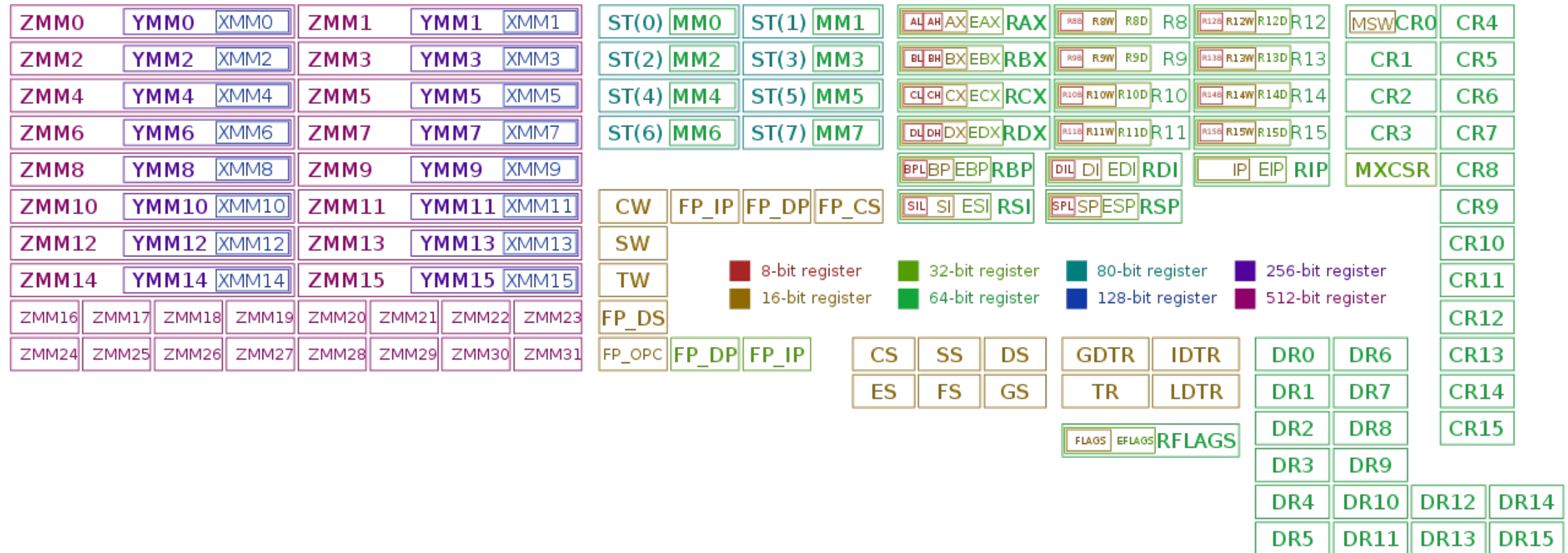


- logical addresses are the ones in PAS
- Physical addresses are the ones in memory chips

Segmentation

- Memory is divided in segments
- Any logical address has two parts:
 - a ***Id*** called ***segment selector***
 - a ***offset*** relative to the selected segment
- On x86, some 16 bits registers are dedicated to specific segments:
 - [cs]: Code Selector
 - [ss]: Stack Selector
 - [ds]: Data Selector
 - [tr]: Task Struct Selector (TSS) or Task Register
 - [es], [fs] & [gs] : extra selectors

X86_64 Registers



Segment descriptor

- Each segment is represented by a segment descriptor (8 Bytes)
- Descriptors are store in a table (either global, the *GDT*, or local, the *LDT*)
- There is only one GDT, whereas each process can have its own LDT (if necessary more segments)
- Segment descriptors contain:
 - The address of the first byte.
 - Size of the segment
 - Type of segment
 - Some right management information (the famous *Segmentation fault*)

=> pmap example

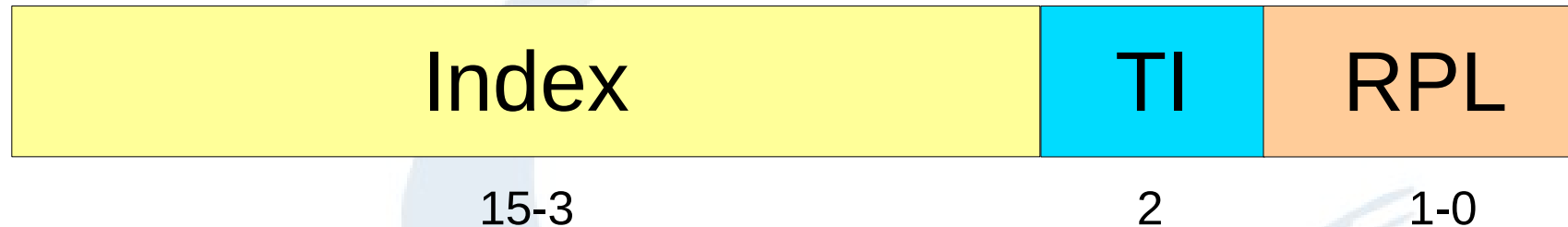
Types of descriptors

- Code segment descriptors:
 - Describe a code segment (GDT or LDT), used to fetch instructions
- Data segment descriptors:
 - Describe a data segment (GDT or LDT), used for common data (heap)
- Stack segment descriptors:
 - Describe a stack segment (GDT or LDT), used for stack pointers
- Task State Segment (TSS)
 - Describe a segment with information on a process context (Remember the `task_struct` of processes) only in GDT
- Local descriptor table descriptors
 - Describe a segment containing a LDT (only in GDT)

Segment selector

- 3 fields:
 - Index in the descriptor table (16 bits)
 - Flag specifying the GDT or LDT: Table Indicator (TI) (1bit)
 - Request Privilege Level (RPL) (2 bits = ring 0 to 3)

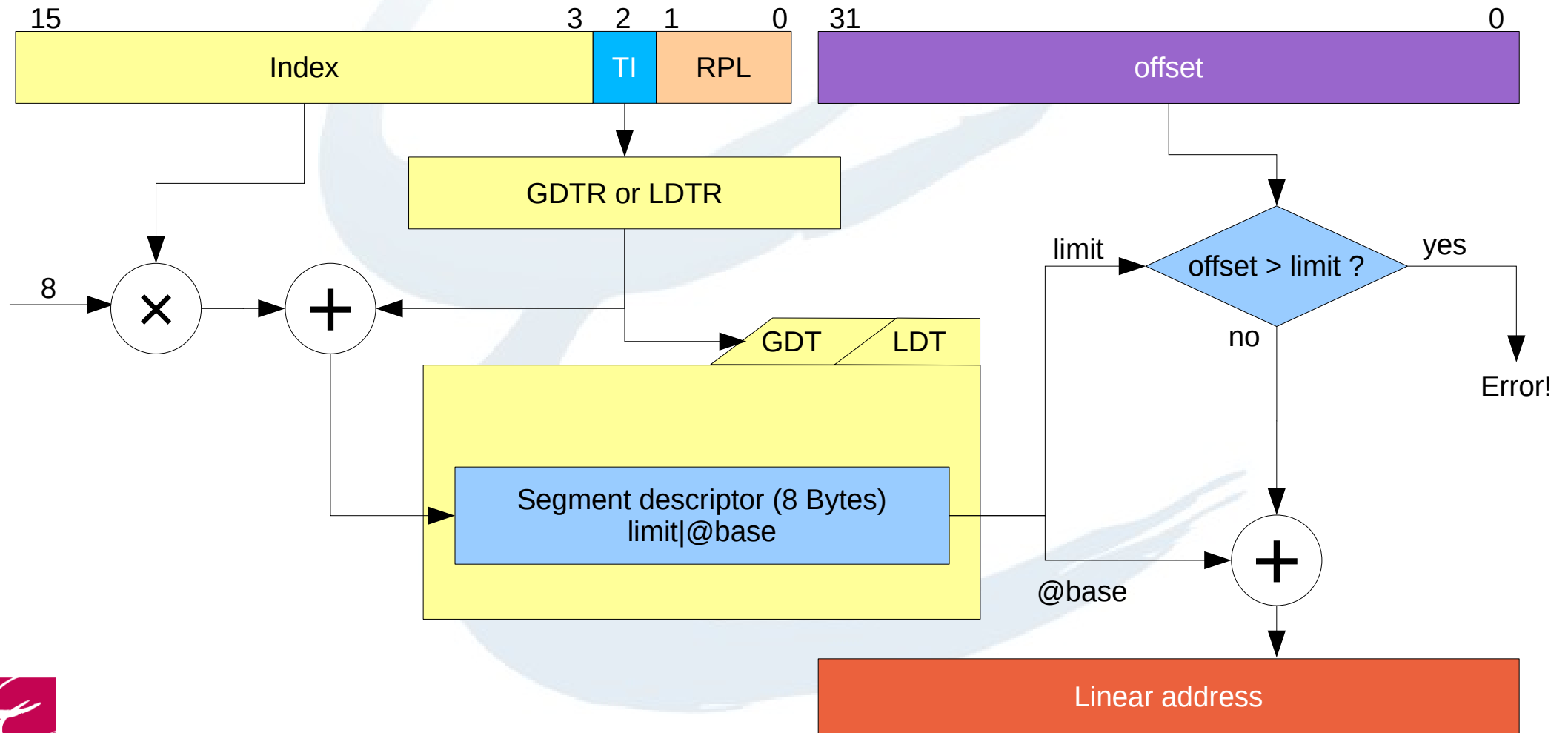
Segment selector



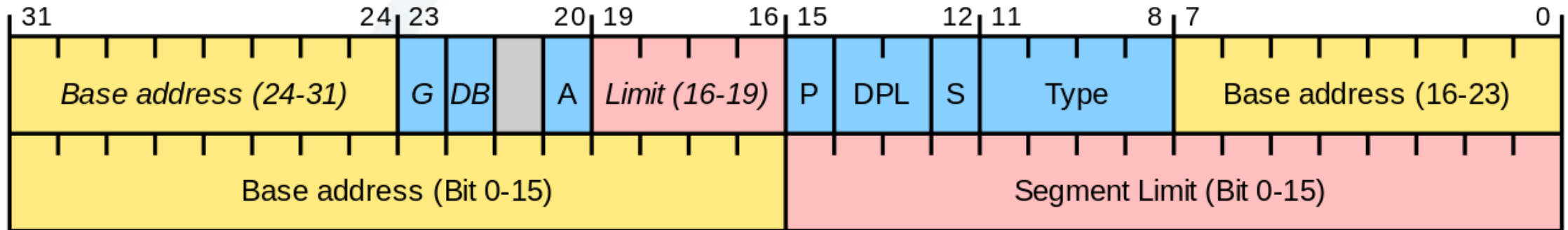
2 registers store the address of:

- GDT (GDTR)
- current LDT (LDTR)

Linear address computation



Segment descriptor

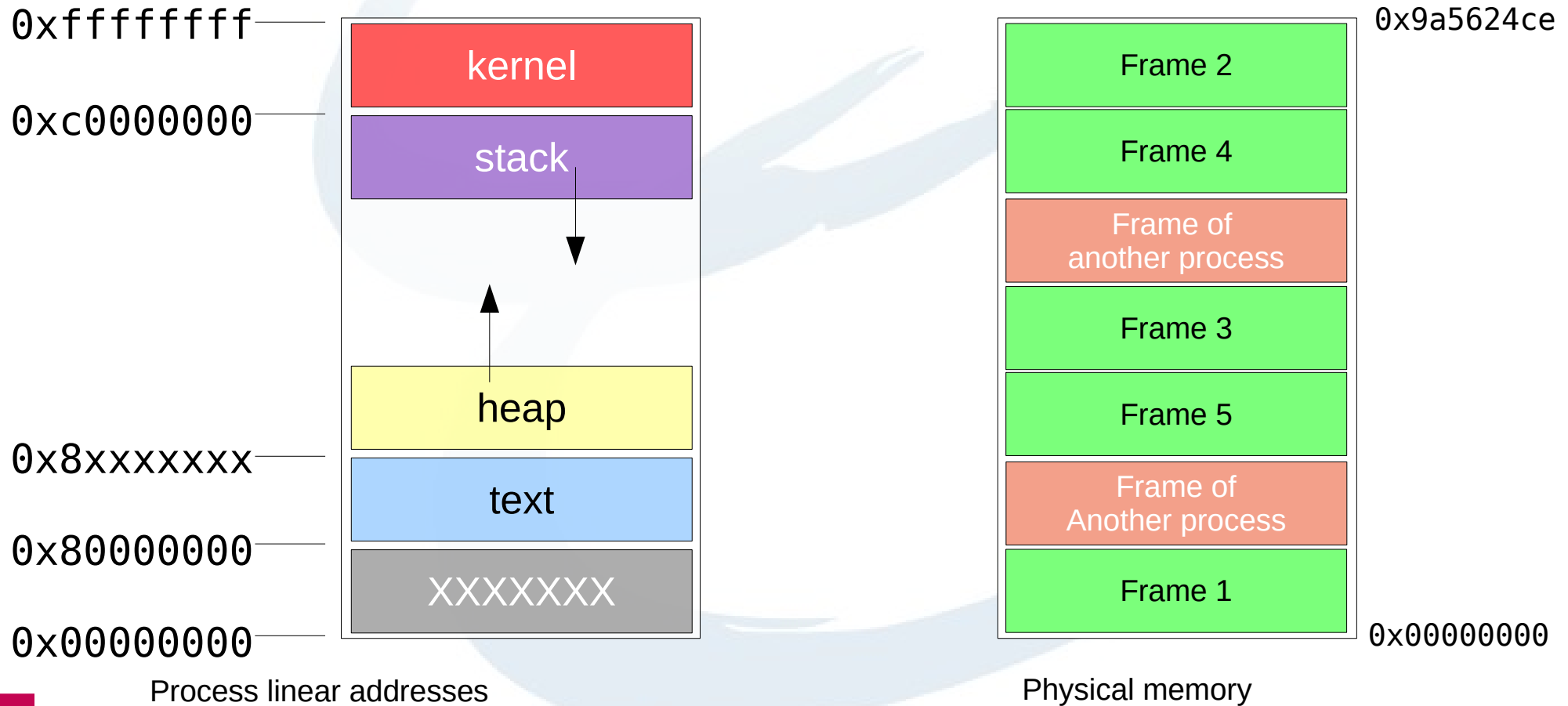


- Base Address: start of the segment (32 bits)
- Segment Limit: size of the segment (20 bits)
- G (Granularity): set if size is bytes, elsewhere in 4 kB pages
- D/B: Default (code) or Big (Data) segment
- L: Long , A: Available, P: Present
- Type (3 bits): Segment type : data/stack, code, TSS, LDT, GDT
- DPL: Descriptor Privilege Level, compared to RPL

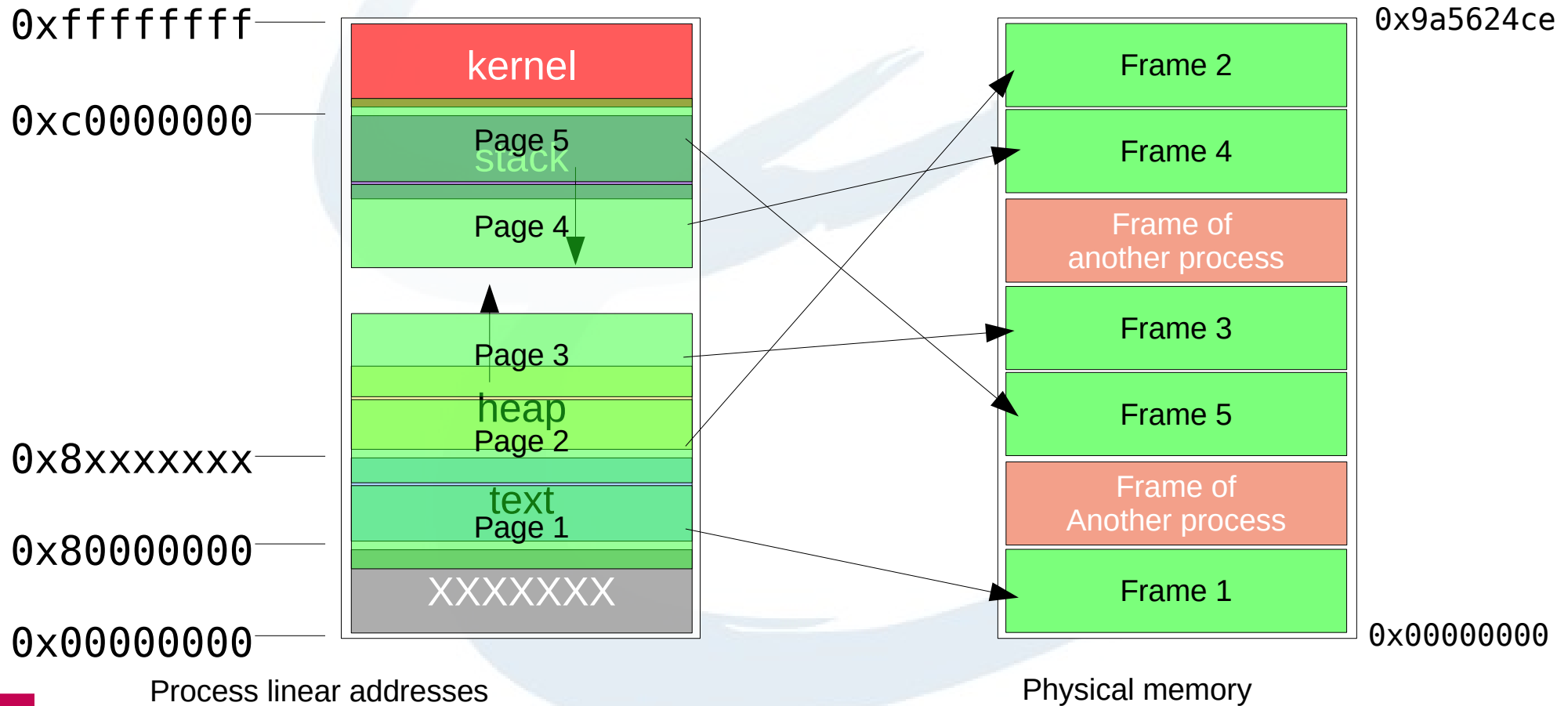
Paging

- Splitting linear addresses into pages
- Splitting the physical memory into page frames
- Map between pages and page frames
 - Non Linear mapping (addresses are not continuous)
 - Right management (*Page fault* exception)
 - **One mapping per process**

Paging



Paging

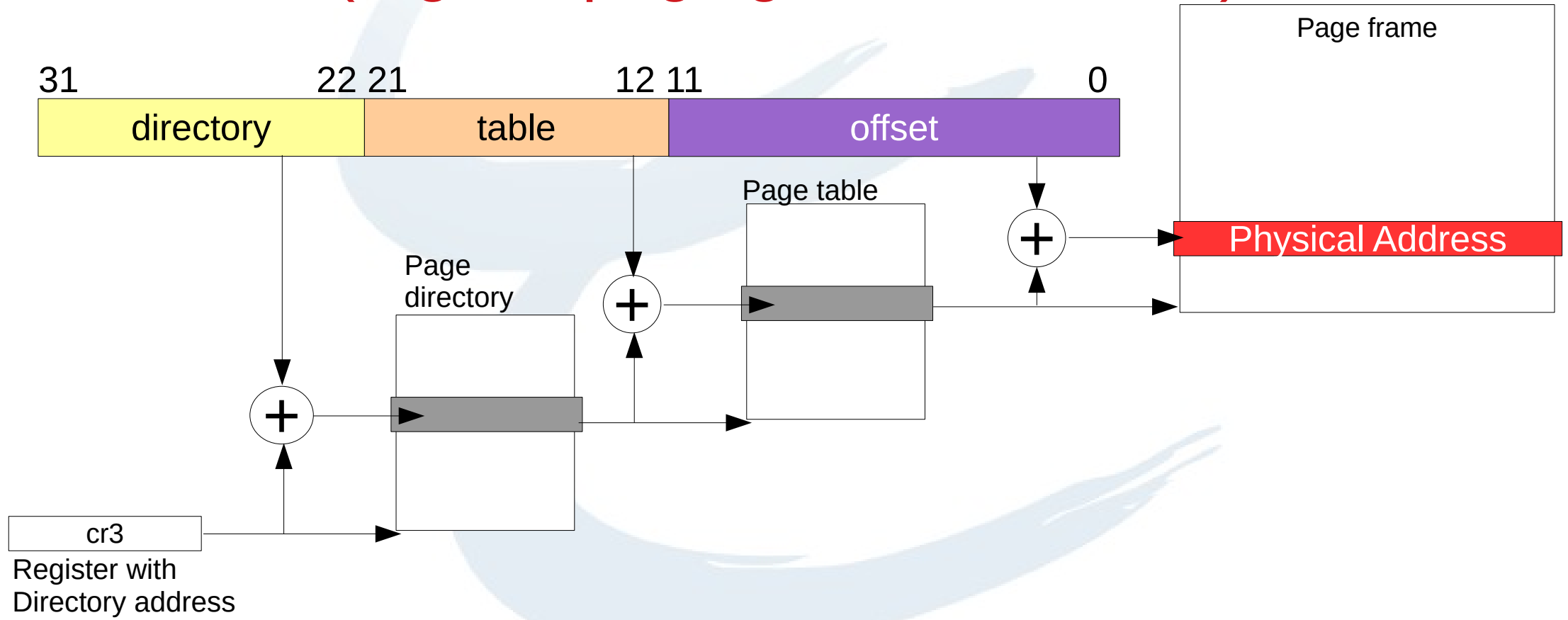


Linear addresses

- Split into 3 fields:
 - Directory index for selecting a directory of tables
 - Table index for selecting a table of pages
 - Offset in the page



Physical addresses computation (regular paging in 32 bits arch)



Directory and table structures

- flag **Present**: is the page in RAM (if 0 then *Page fault* exception)
- Address of a physical memory zone containing either a table (for directories) or a page (for tables)
- flag **Dirty**: page frame has been written
- flags **Read/Write**: right management on the page frame
- flag **User/Supervisor**: defines the level of permission to access the page

Paging in Linux on x84-64

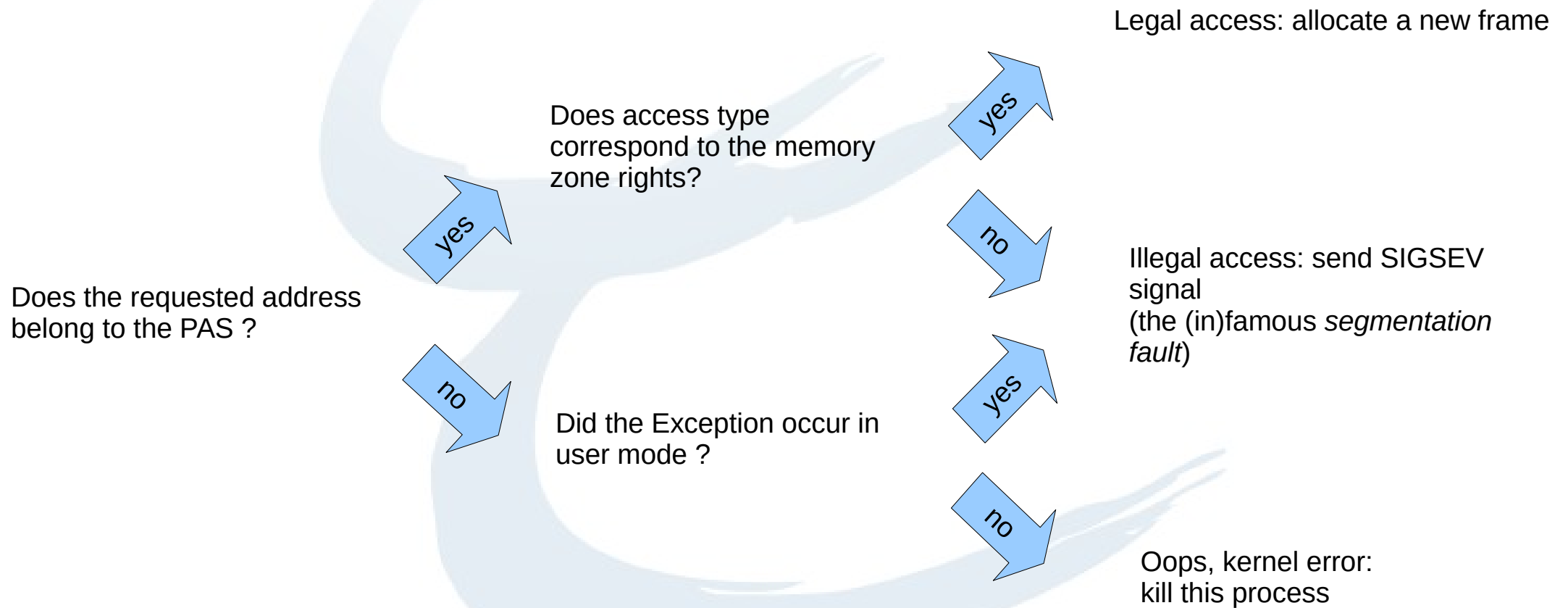
- 4 levels of indirection (3 directories, 1 table and the offset)
- Each process has its own private global directory and associated directories and tables
- During a context switch the address of the global directory is store in the `task_struct`
- The size of a page is fixed by the macro `PAGE_SIZE` (4096 = 4kB)
- The kernel uses its own page tables and maintain their allocation

Translation scenarios

During the translation of a linear address to a physical address, several things can happen:

- Table entry is valid: the address is replaced by the corresponding entry in the table (physical address), everything is ok
- Entry is not valid: we must find a free page frame, associate it with the requested page and push the association in the tables
- Entry is valid, but the read/write rights are not: throw an exception
- Entry is valid, but the context rights (user/supervisor) are not : throw an exception
- Entry is valid but the page frame is not in RAM: get the page frame from the disk (swap), find a free page frame in RAM, copy from the first to the second, update the association.

On *Page fault* Exception:



Hardware acceleration

- Modern processors have built-in hardware decoding circuits for segmentation and paging (MMU – memory management unit)
 - Fill in the register containing the addresses of vm structures (directories, tables, ...)
 - Activate segmentation or paging via a control register
 - The MMU does everything on its own...

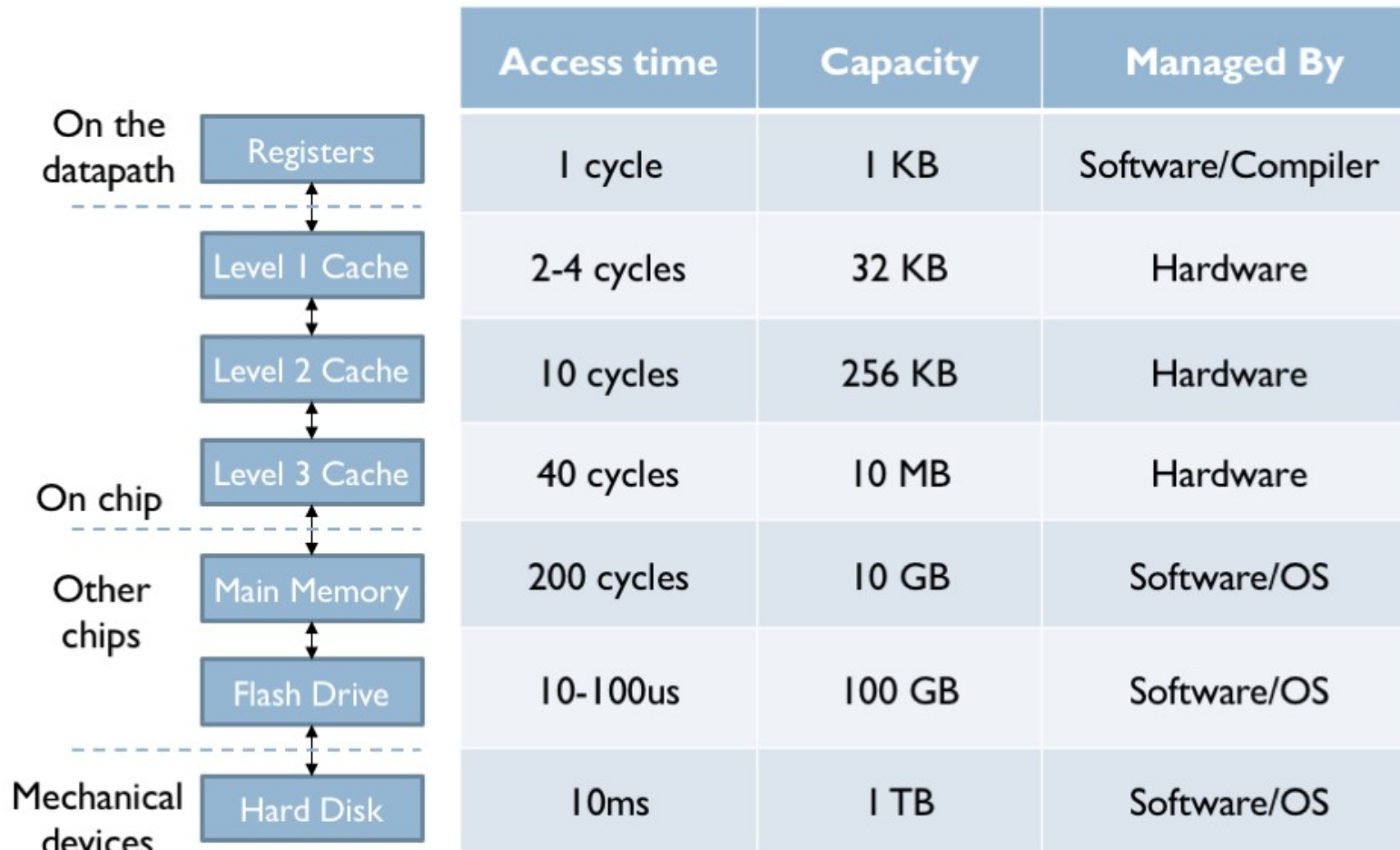
Virtual memory advantages

- Memory abstraction (independence from specific hardware implementations)
- Easy right management
- Hides memory fragmentation (simplifies a lot the job of programmers)
- All the complex memory management stuff is hidden in the OS
- Avoid reallocation a lot

Cache

- Accessing RAM is costly
- Processors have a small internal memory zone named cache
- Thanks to a hardware mechanism often used page are stored in this cache zone
- If a data is not in the cache, it is loaded into it (cache miss)

Typical Cache Hierarchy



Transition Look-aside Buffers

- TLBs accelerate the conversion from logical addresses to physical addresses
- Small buffers that store the mapping between logical and physical addresses
- Kernel handles the validity of the mapping in the TLBs (via `flush_tlb_*`), because only the kernel handles this mapping
- Whenever the register storing the address of the global directory register is changed, all TLBs are invalidated
- During process commutation, TLBs are thus cleared

Memory descriptors

- `struct mm_struct` : contains information on the address space of a process
- In the field `mm` of the process descriptor
- Double linked list of descriptors
- sharing `mm_struct` between several processes is possible (syscalls `clone()` and `fork()`)

mm_struct (mm_types.h)

```
struct mm_struct {  
    struct vm_area_struct * mmap;           /* list of VMAs */  
  
    ...  
  
    pgd_t * pgd;                           /* pages global directory */  
    atomic_t mm_users;                     /* How many users with user space? */  
    atomic_t mm_count;                     /* How many references to "struct mm_struct" (users count as 1) */  
    int map_count;                         /* number of VMAs */  
  
    ...  
  
    unsigned long total_vm, locked_vm, shared_vm, exec_vm;  
    unsigned long stack_vm, reserved_vm, def_flags, nr_ptes;  
    unsigned long start_code, end_code, start_data, end_data; /* code et data*/  
    unsigned long start_brk, brk, start_stack; /* tas et pile */  
    unsigned long arg_start, arg_end, env_start, env_end; /* arguments et environnement */  
    ...  
};
```

Memory areas

- Exists thanks to `vm_area_struct` structure
- Encodes an interval of linear addresses
- No overlap
- Fusion or splitting possible at allocation/reallocation if rights are the same on the 2 regions

vm_area_struct (mm_types.h)

```
/*
 * This struct defines a memory VMM memory area. There is one of these
 * per VM-area/task. A VM area is any part of the process virtual memory
 * space that has a special rule for the page-fault handlers (ie a shared
 * library, the executable area etc).
 */
struct vm_area_struct {
    struct mm_struct * vm_mm;        /* The address space we belong to. */
    unsigned long vm_start;          /* Our start address within vm_mm. */
    unsigned long vm_end;            /* The first byte after our end address
                                     within vm_mm. */

    /* linked list of VM areas per task, sorted by address */
    struct vm_area_struct *vm_next, *vm_prev;

    pgprot_t vm_page_prot;           /* Access permissions of this VMA. */
    unsigned long vm_flags;          /* Flags, see mm.h. */

    ...
    /* Function pointers to deal with this struct. */
    const struct vm_operations_struct *vm_ops;

    ...
};
```

Area operations

linux/mm.h

```
/*
 * These are the virtual MM functions - opening of an area, closing and
 * unmapping it (needed to keep files on disk up-to-date etc), pointer
 * to the functions called when a no-page or a wp-page exception occurs.
 */
struct vm_operations_struct {
    void (*open)(struct vm_area_struct * area);
    void (*close)(struct vm_area_struct * area);
    int (*fault)(struct vm_area_struct *vma, struct vm_fault *vmf);

    /* notification that a previously read-only page is about to become
     * writable, if an error is returned it will cause a SIGBUS */
    int (*page_mkwrite)(struct vm_area_struct *vma, struct vm_fault *vmf);
    ...
};
```

Area right management (mm.h)

```
/*
 * vm_flags in vm_area_struct, see mm_types.h.
 */

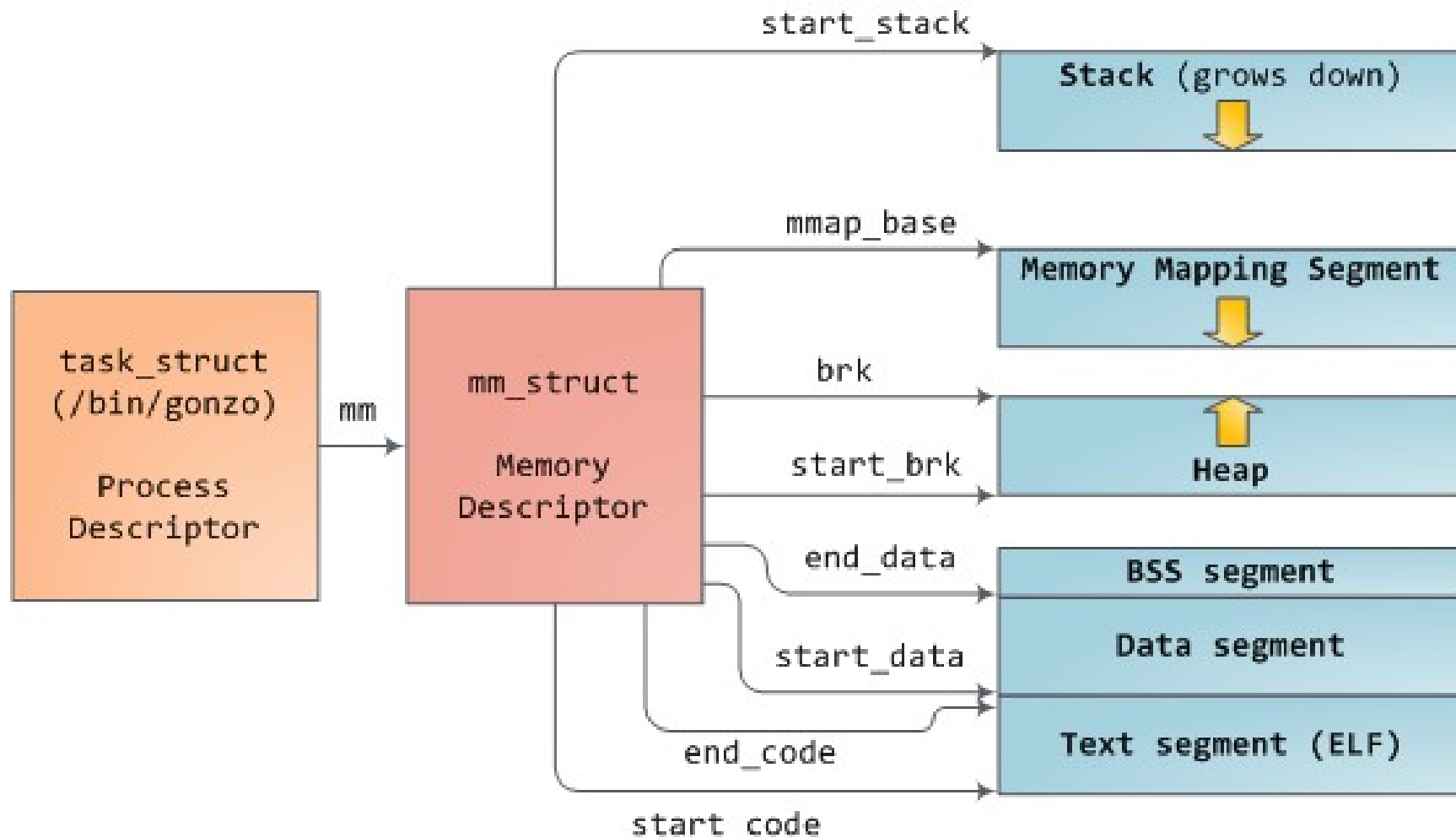
#define VM_READ      0x00000001  /* currently active flags */
#define VM_WRITE     0x00000002
#define VM_EXEC      0x00000004
#define VM_SHARED     0x00000008

/* mprotect() hardcodes VM_MAYREAD >> 4 == VM_READ, and so for r/w/x bits. */
#define VM_MAYREAD   0x00000010  /* limits for mprotect() etc */
#define VM_MAYWRITE  0x00000020
#define VM_MAYEXEC   0x00000040
#define VM_MAYSHARE  0x00000080

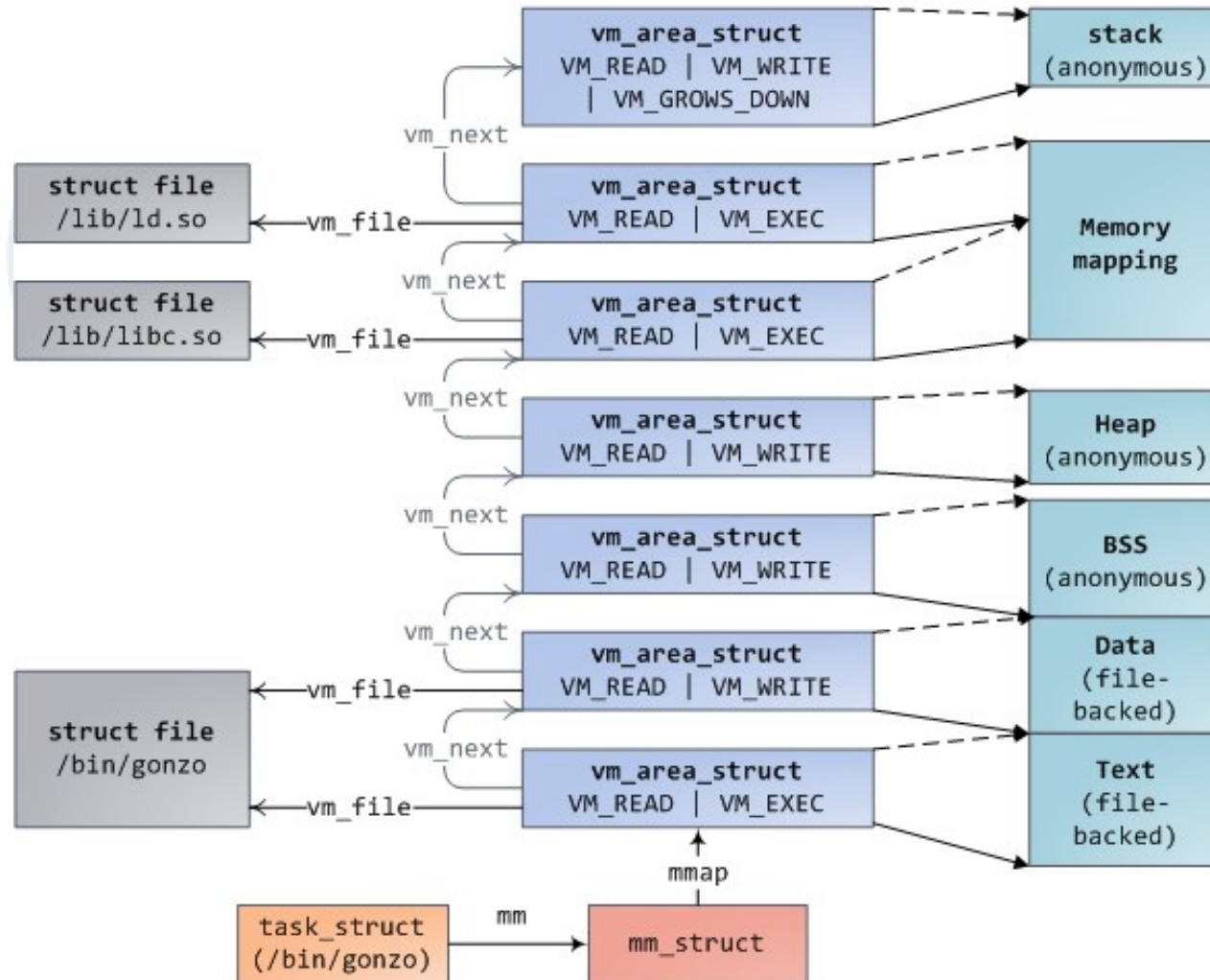
#define VM_GROWSDOWN 0x00000100  /* general info on the segment */

#define VM_DONTCOPY   0x00020000  /* Do not copy this vma on fork */
#define VM_DONTEXPAND 0x00040000  /* Cannot expand with mremap() */
#define VM_RESERVED   0x00080000  /* Count as reserved_vm like IO */
#define VM_ACCOUNT    0x00100000  /* Is a VM accounted object */
#define VM_NORESERVE  0x00200000  /* should the VM suppress accounting */
#define VM_HUGETLB    0x00400000  /* Huge TLB Page VM */
#define VM_NONLINEAR  0x00800000  /* Is non-linear (remap_file_pages) */
```


Memory Descriptor



-----> vm_end: first address **outside** virtual memory area
 -----> vm_start: first address **within** virtual memory area



Swap

- Sometime, the available RAM is not sufficient for storing all data (code and data)
- A major advantage of virtual memory is that page frames do not need to be in RAM
- A zone on the hard disk can be reserved to store some page frames, called swap
- The kernel handles the transfer between the RAM and the Swap
 - By putting in RAM most used pages
 - By handling the *page fault* exception when a requested page is not in RAM but in swap

Handling swap

- Sometimes, a process request a page in swap while RAM is occupied at 100%
 - The system chooses a page (hopefully one with low request rate)
 - Puts it in swap
 - Uses the freed page frame for the requested page
 - Updates the corresponding tables
- Sometimes the chosen page contains data that can safely be erased (cache data, for example)
 - In that case, the system simply erases its content
 - And updates the tables

Memory allocators

- As we have seen, the kernel handle the memory
- Whenever a process needs more memory, it asks the kernel via a system call:
 - `brk()` and `sbrk()`
- In fact the demand is done by calling a libc function:
 - `malloc()`, `calloc()`, `realloc()`...
 - Personal allocator

Managing the heap

- `malloc(size)`: allocate size
- `calloc(n,size)`: n elements of size size initialized to 0
- `realloc(ptr, size)`: change the size of a memory zone allocated with malloc or calloc
- `free(ptr)` deallocate the zone
- `brk(addr)` increase the size of the heap (`current->mm->brk`) until reaching address addr
- **sbrk**(incr) increase the size of the heap of size incr

`sys_brk(addr)` System call

- If `addr` is inside an existing Code memory area → out
- Align `addr` on a page (smallest allocable element)
- If decreasing the heap, then call `do_munmap()` → out
- Else, verify permissions
- `do_mmap()`

do_mmap()

- allocate a new memory area
 - Verify the request validity (size, max number of areas, rights...)
 - Invoke `get_unmapped_area()` to obtain a valid address
 - Compute permission flags of the new vm area
 - if `VM_SHARED` equals 0, then try to fuse with existing area
 - Allocate a new struct `vm_area_struct`
 - Fill the `vm_area_struct`
 - Insert in the tree of vm area structures
 - Return the new address

Kernel object allocation

- `kmem_cache_alloc()`
- Linux kernel allocator is called *slab allocator* and follows these principles:
 - Data type can influence memory allocation
 - Kernel functions ask often for areas of the same size (for example the size of a process descriptor)
 - Requests can be ordered by frequency
- The allocator has a cache system that allows faster allocation by reusing memory areas

3IS – Operating Systems – File Systems

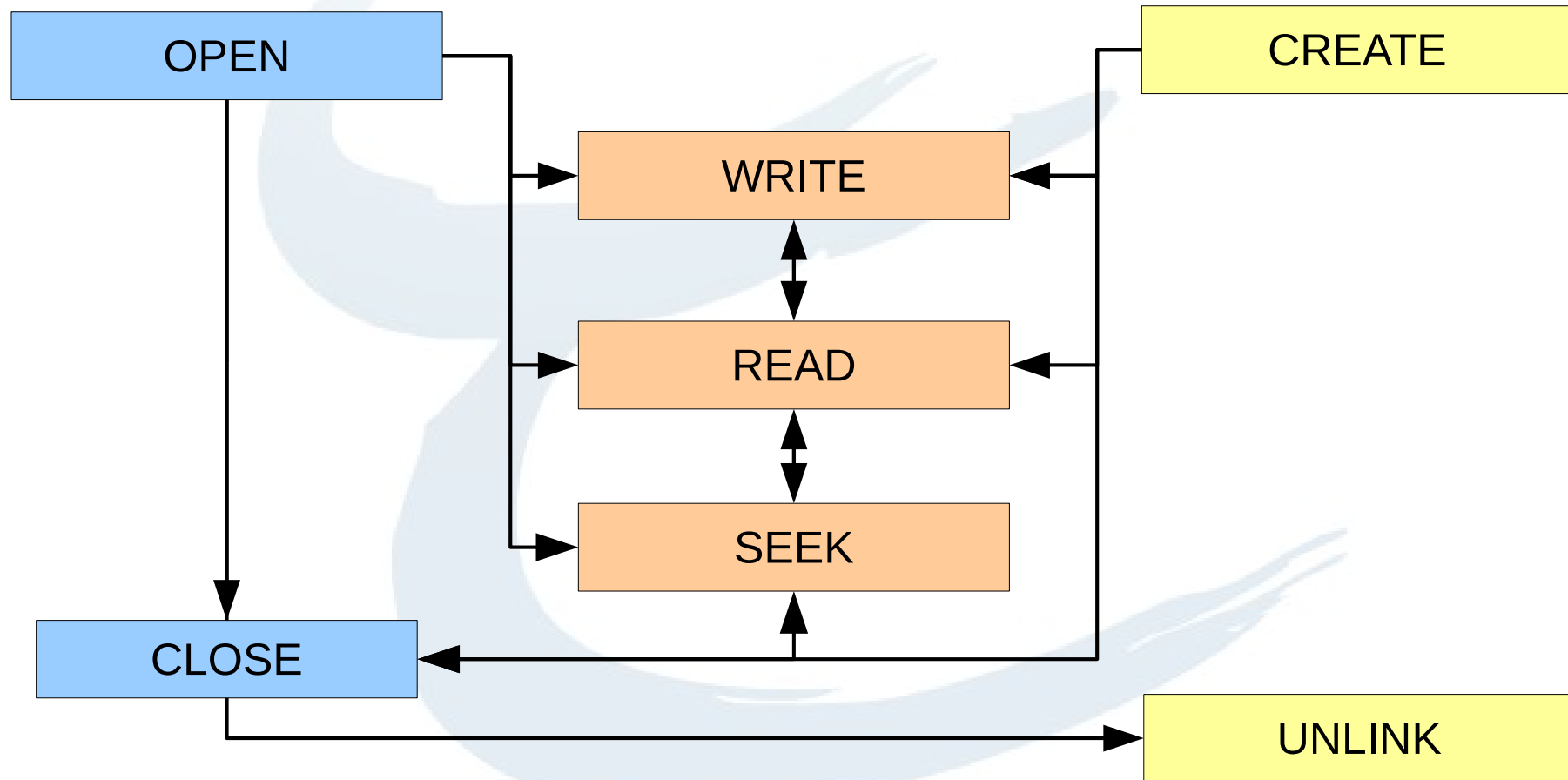
2024 – Christophe Barès
christophe.bares@ensea.fr

Contributions: David Picard,
Arnaud Revel, Mickaël Maillard

“File” concept

- Ordinary files (-)
 - Sequence of “random” bytes
- Directories (d)
 - Files containing other files
- Links
 - Hardware link (-): a files with several names
 - Symbolic link (l): an alias
- Special files: devices **/dev**
 - Character mode (c)
 - Block mode (b)
- Pipes and named pipes (p)
- Sockets (s)

Life cycle



Basic system calls

- Open: `open()`
- Release: `close()`
- Get data: `read()`
- Put data: `write()`
- Set head: `lseek()`
- Delete: `unlink()`
- Infos: `stat()`, `fstat()`, `lstat()`

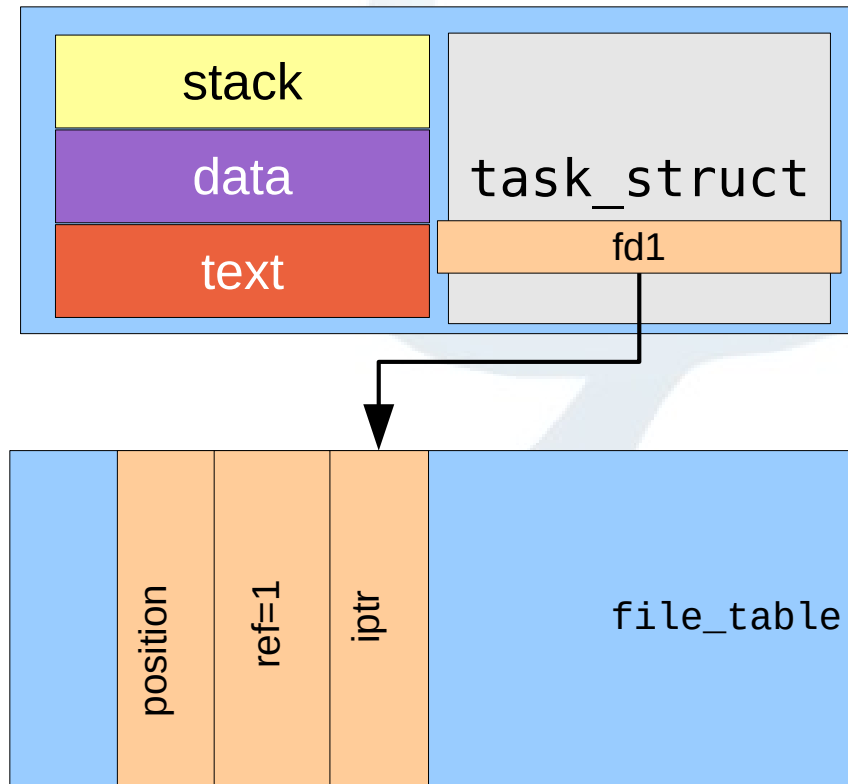
Struct file in Linux 0.01

include/linux/fs.h lines 112-118

<https://elixir.bootlin.com/linux/0.01/source/include/linux/fs.h#L112>

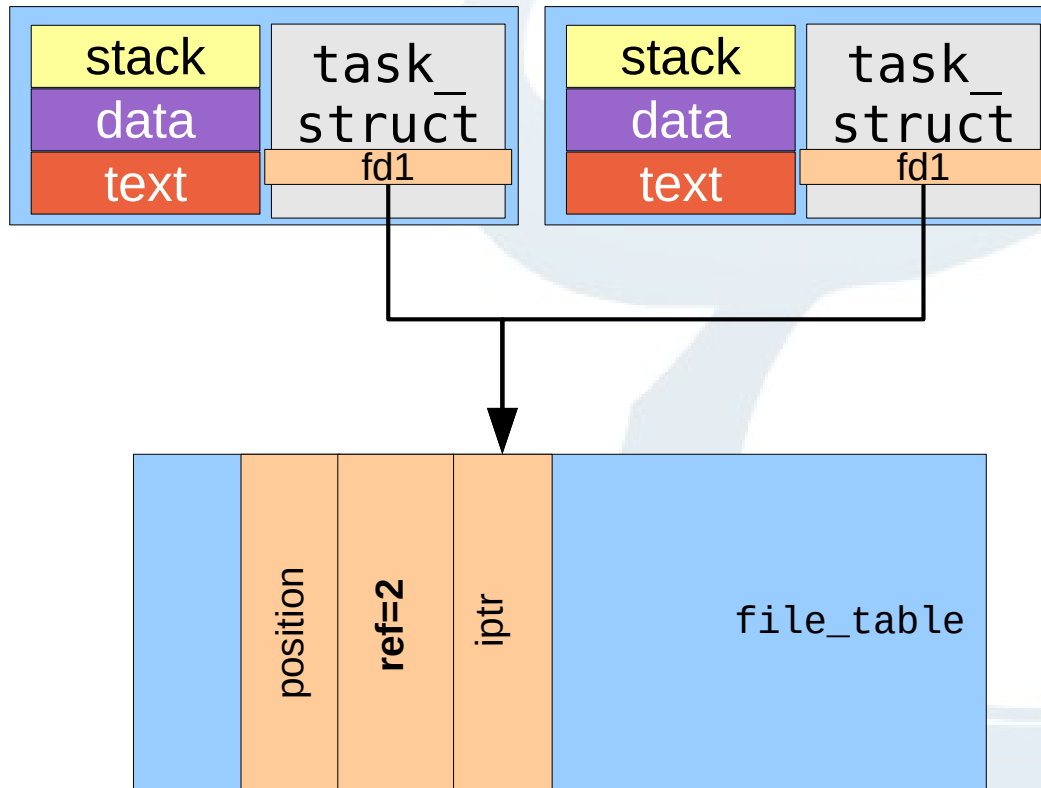
```
struct file {  
    unsigned short f_mode;    /* opening mode */  
    unsigned short f_flags;   /* bit field */  
    unsigned short f_count;   /* count of references */  
    struct m_inode * f_inode; /* associated inode */  
    off_t f_pos;              /* head position */  
};
```

task_struct



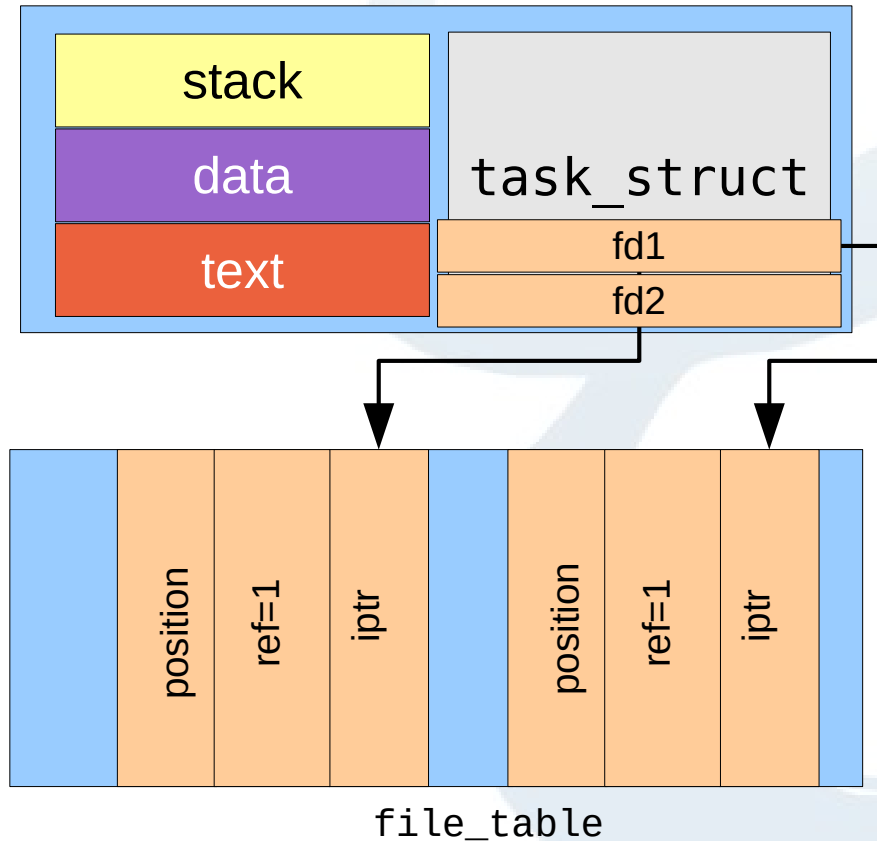
- A list of open file descriptors is stored in the `task_struct`
- A global list of open files is kept by the kernel in the `file_table`

fork()



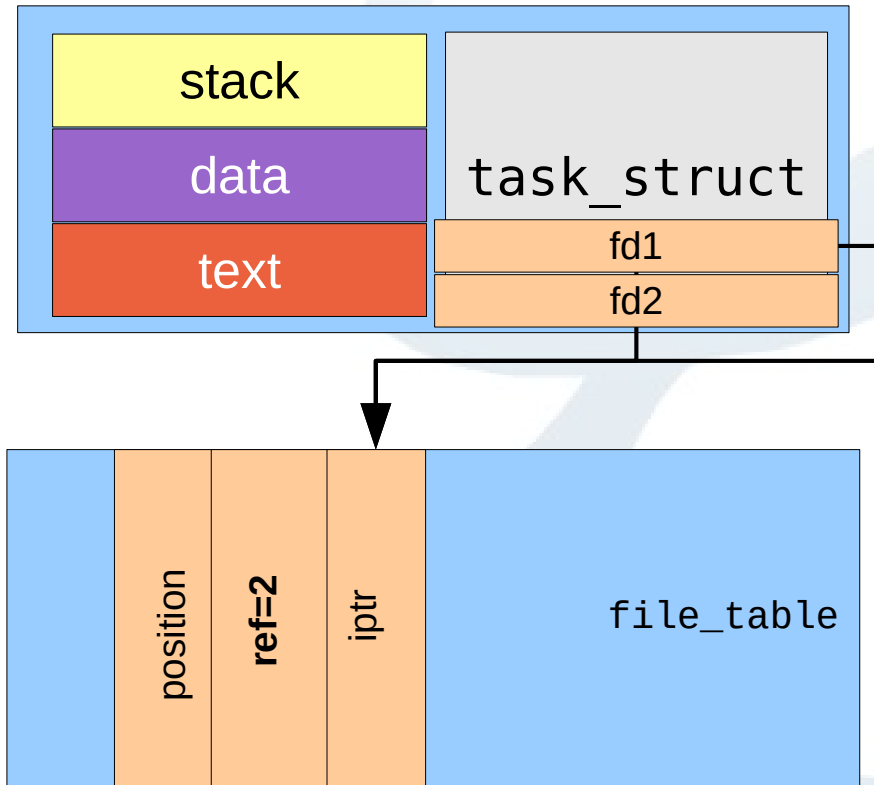
- `task_struct` is copied, file descriptor pointers also
- Reference count is increased in the list of open files

Open 2 times the same file



- A new descriptor is added to the task_struct
- A new entry is added to the file_table global list (different reference counters and positions)

dup ()



- Duplicates an entry in the `task_struct`
- Corresponding reference counter is increased
- Allows to change standard file descriptors (e.g. set fd 1 to a file)

Filesystem

- A filesystem is a **data structure** allowing to **store** and **organize** files and their corresponding data on mass storage units.
- One of the advantages of linux is to be able to use a lot of different filesystems
 - ext2/3/4, MINIX, NTFS, FAT16/32, JFS, Reiser3/4, UDF, HFS, XFS, btrfs, ...
- On unix operating systems, filesystems are organized with a tree structure

Hard disk drive

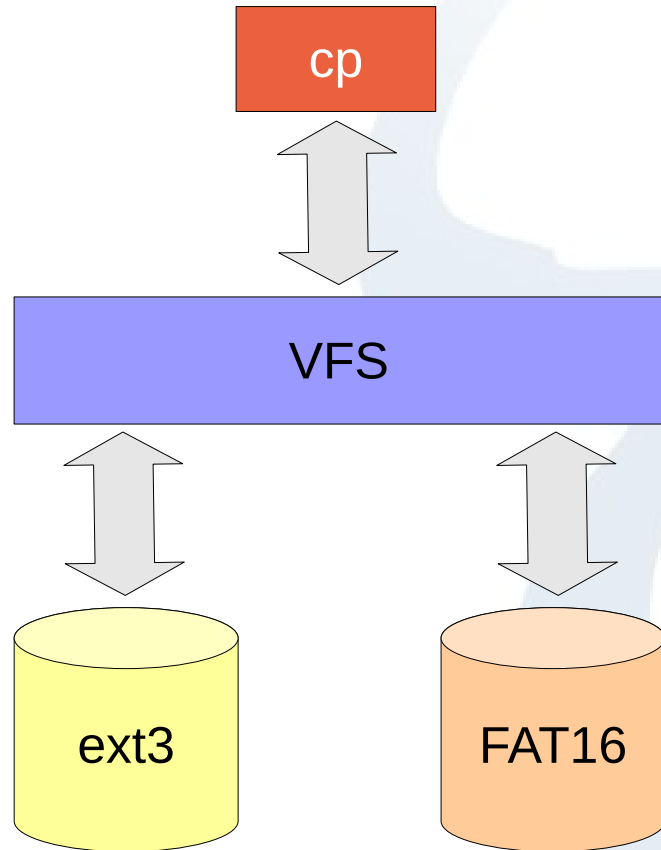


- A hard disk drive is a set of rotating **magnetic** disks
- At the beginning, the magnetic material is in a undetermined state
- It has to be **formatted**
 - The **low level** format sets block of 512 to 4k blocks in a state corresponding to what the hardware controller expects
 - The **high level** format creates some functional units corresponding to what the OS expects

FS specificity

- A FS corresponds to a specific organization of the available blocks on the disks
- Basic file operations (`open()`, `read()`, `write()`, etc) are thus depending on the FS
- In order to allow generic code, an hardware abstraction layer is developed: the **VFS** (*Virtual Filesystem Switch*)
- The VFS gets the standard requests and translates them to the right API corresponding to the FS where the file is

VFS



```
inf = open("/mnt/usbdisk/fichier.txt",  
           O_RDONLY, 0);
```

```
outf = open("/home/toto/fichier.txt",  
            O_WRONLY|O_CREAT|O_TRUNC, 0600);
```

```
do{  
    i = read(inf, buf, 4096);  
    write(outf, buf, i);  
} while(i);
```

```
close(outf);  
close(inf);
```

=> Switching from one file system
to another is **seamless**

FS supported by the VFS

- Disk FS
 - Linux (ext2/3/4, btrfs...)
 - Unix (sysv, UFS, MINIX...)
 - Proprietary (VFAT, NTFS, HFS, AFFS...)
 - Journalized (JFS, XFS...)
 - With snapshots (btrfs, ZFS...)
- Networking FS
 - To allow the access to distant FS as if it were local FS (NFS, CIFS, NCP, AFS, ...)
- Special FS
 - Virtual FS allowing to access specific resources as if it were normal files (/proc, /dev, ...)

Implementation

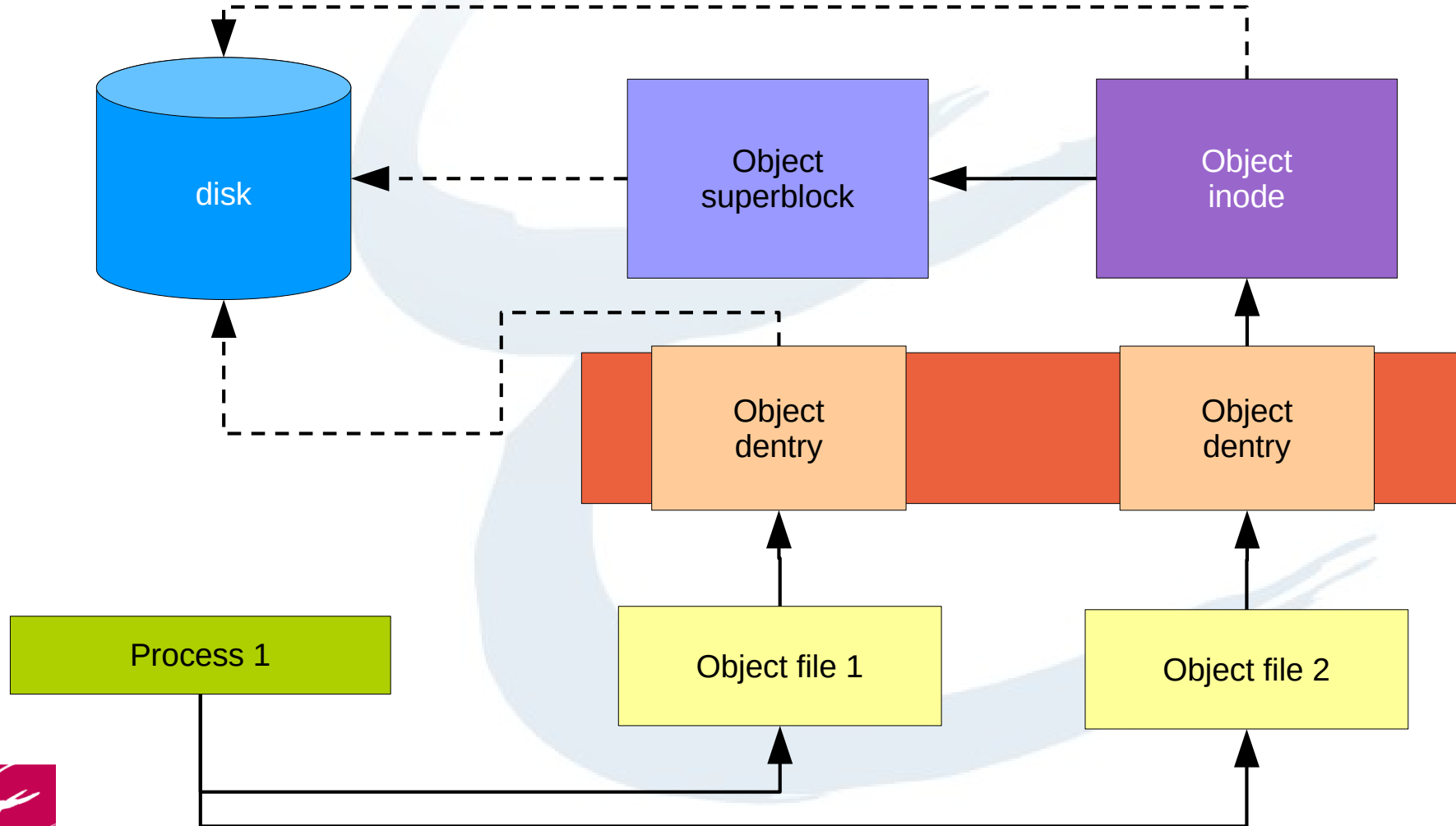
- field `f_op` added inside structure `file`
- `f_op` contains pointers to each FS specific function corresponding to basic file operations
- Functions for accessing file use (`read( )` for example) use the function pointed by the fields of the structure `f_op`

```
file->f_op->read(...);
```


VFS Object Model

- Developed in pure C for performances reasons
- Object **superblock**
 - Stores information on the FS itself (FS control block **on the disk**)
- Object **inode**
 - Contains information on a file (file control block **on the disk**), identified by a unique inode number. => “Mineral” information
- Object **dentry**
 - Object corresponding to a directory, and the corresponding **specific file of the FS** (this object stores the file name)
- Object **file**
 - Contains information on a file being used by the system and its related processes (**in memory only**). => “Living” object

Example



struct super_block

```
struct super_block {  
    struct list_head    s_list; // liste de superblock  
    dev_t               s_dev;  // identifiant de périphérique  
    ...  
    unsigned char       s_blocksize_bits; // taille block en bits  
    unsigned long       s_blocksize; //taille block en octets  
    loff_t              s_maxbytes; //taille max des fichiers  
    struct file_system_type *s_type; //type de fs  
    const struct super_operations *s_op; // pointeurs fonctions sb  
    ...  
    struct dentry       *s_root; // dentry de la racine  
    ...  
    int                 s_count; //compteur de références  
    ...  
    struct list_head    s_inodes; //list des inodes  
    ...  
    struct list_head    s_files; //liste de fichiers  
    ...  
    struct block_device *s_bdev; //pilote du périphérique  
    ...  
    struct list_head    s_instances; // list des sb du type de fs  
    ...  
    char                s_id[32]; // nom du périphérique  
    uuid_t              s_uuid;   // Universally Unique ID  
    ...  
    void               *s_fs_info; // info sur le fs  
}
```

struct super_block

- s_fs_info contains information on the FS
- Stored on the disk and copied into main memory (due to many access)
- s_op contains pointers to superblock functions, specific to the FS
 - Ex: `sb->s_op->read_inode(inode);`

struct inode

```
struct inode {  
    ...  
    struct list_head      i_sb_list; // liste des inodes du sb  
    struct hlist_head     i_dentry; //list des dentry qui font référence à l'inode  
    unsigned long         i_ino; // numéro d'inode  
    atomic_t              i_count; // compteur d'utilisation  
    unsigned int          i_nlink; //nombre de liens physique  
    kuid_t                i_uid; //id du propriétaire  
    kgid_t                i_gid; //id du groupe  
    dev_t                 i_rdev; //id du périphérique  
    u8                    i_blkbits; // taille du block en bits  
    loff_t                i_size; //taille en octet  
    struct timespec64     i_atime; //date d'accès  
    struct timespec64     i_mtime; //date de modification  
    struct timespec64     i_ctime; //date de création  
    blkcnt_t              i_blocks; //nombre de blocks contenant le fichier  
    unsigned short        i_bytes; //bompre d'octet utilisés dans le dernier block  
    umode_t               i_mode; //type de fichier et droits d'accès  
    const struct inode_operations *i_op; // routines sur les inodes  
    struct super_block     *i_sb; //pointeur vers le superblock  
    ...  
}
```

struct file

```
struct file {  
    struct list_head    f_list; //list de fichier du processus  
    struct dentry        *f_dentry; //répertoire contenant le fichier  
    ...  
    struct file_operations *f_op; //pointeurs de fonction  
    ...  
    atomic_t            f_count; //compteur d'ouverture  
    unsigned int         f_flags; // drapeaux comme lecture seule, append, etc  
    mode_t               f_mode; // mode : read ou write  
    int                  f_error; //erreur  
    loff_t                f_pos; //position dans le fichier  
    struct fown_struct    f_owner; // propriétaire du fichier  
    unsigned int          f_uid, f_gid; //id du propriétaire, du groupe  
    ...  
    size_t               f_maxcount;  
    ...  
    const struct file_operations *f_op; // voir diapo suivante  
};
```

struct file_operations

```
struct file_operations {  
    // Pointer to the owning module  
    struct module *owner;  
  
    // Change read/write position  
    loff_t (*llseek) (struct file *, loff_t, int);  
    // Read data, returns number of read bytes  
    ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);  
    // Write  
    ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);  
    // Device specific (non read or write) commands  
    int (*ioctl) (struct inode *, struct file *, unsigned int, unsigned long);  
    // The first operation on the driver, it can be left NULL but then the driver  
    // won't be notified  
    int (*open) (struct inode *, struct file *);  
    // Most of the times called before closing the files. Demands that pending  
    // operations be finished.  
    int (*flush) (struct file *);  
    // Implementation of file locking  
    int (*lock) (struct file *, int, struct file_lock *);  
  
    ...  
};
```

struct dentry

```
struct dentry {
    atomic_t          d_count;
    unsigned int      d_flags;      /* protected by d_lock */
    spinlock_t        d_lock;      /* per dentry lock */
    int               d_mounted;
    struct inode       *d_inode;    /* Where the name belongs to - NULL is
                                   * negative */

    struct hlist_node  d_hash;      /* lookup hash list */
    struct dentry      *d_parent;   /* parent directory */
    struct qstr        d_name;

    struct list_head   d_subdirs;   /* our children */
    struct list_head   d_alias;     /* inode alias list */
    unsigned long      d_time;      /* used by d_revalidate */
    const struct dentry_operations d_op;
    struct super_block *d_sb;       /* The root of the dentry tree */
    void              *d_fsdata;    /* fs-specific data */

    unsigned char      d_iname[DNAME_INLINE_LEN_MIN]; /* small names */
};
```

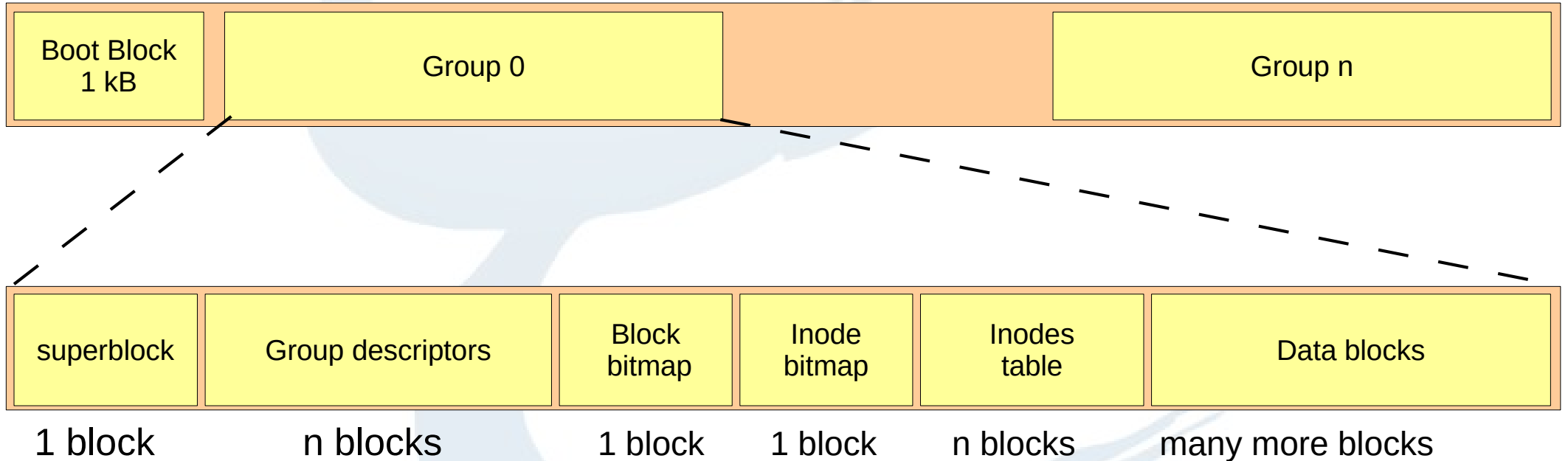

struct dentry

- Duplicated object (generic version in memory, specific version on the disk)
- One object for each directory accessed
 - Ex : /tmp/test creates 3 dentries : 1 for “/”, 1 for “tmp” in “/” and 1 for “test” in “tmp”
- Associates the directory with the corresponding inode
- Managed by a cache so as to minimize the creation of new dentries (minimizing disk access)

Filesystem ext2

- Inspired by the MINIX filesystem
- Defines an optimal block size (from 1kB to 4kB)
- Manages block by groups
 - Inodes and corresponding blocks of a same group are close together on the disk, reducing access time
- Anticipation allocation
 - ext2 reserves several blocks at creation time to anticipate further growing of the files
- Unchangeable files support (`$ chattr +i my_file`)
- Compatible with SysVR4 and BSD

Disk structures (ext2)



Example on ext4: dumpe2fs /dev/scd1

Filesystem volume name: <none>

Last mounted on: /

...

Filesystem state: clean

Errors behavior: Continue

Filesystem OS type: Linux

Inode count: 7815168

Block count: 31258368

Reserved block count: 1562918

Free blocks: 20478313

Free inodes: 6889700

First block: 0

Block size: 4096

Fragment size: 4096

Reserved GDT blocks: 1016

Blocks per group: 32768

Fragments per group: 32768

Inodes per group: 8192

Inode blocks per group: 512

Flex block group size: 16

Filesystem created: Sat Jan 23 17:08:47 2016

Last mount time: Mon Nov 26 21:56:20 2018

Last write time: Mon Nov 26 21:56:20 2018

Mount count: 644

Lifetime writes: 6250 GB

First inode: 11

Inode size: 256

...

Journal inode: 8

...

Groupe 0 : (Blocs 0-32767) csum 0x9c97 [ITABLE_ZEROED]

superbloc Primaire à 0, Descripteurs de groupes à 1-8

Blocs réservés GDT à 9-1024

Bitmap de blocs à 1025 (+1025)

Bitmap d'i-noeuds à 1041 (+1041)

Table d'i-noeuds à 1057-1568 (+1057)

20902 blocs libres, 0 i-noeuds libres, 1589 répertoires

Blocs libres : 9856-9983, 10808-10815, 10880-10943, 11456-11540\$

I-noeuds libres :

Groupe 1 : (Blocs 32768-65535) csum 0xc017 [ITABLE_ZEROED]

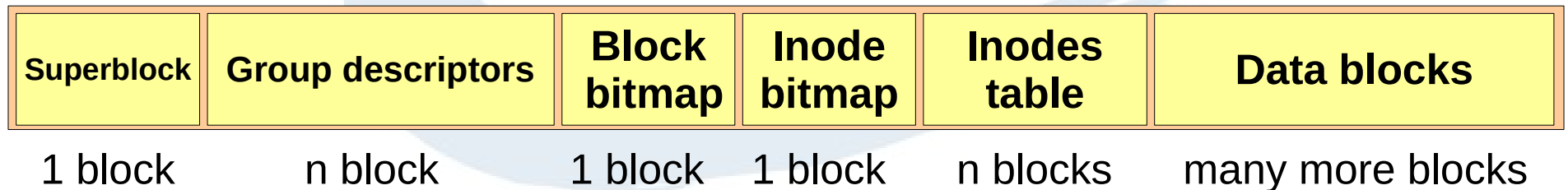
superbloc Secours à 32768, Descripteurs de groupes à 32769-32776

...

Groups (ext2)

Each group contains several blocks that are either:

- A copy of the superblock (the one of the group 0 is used, other are for redundancy)
- A copy of a group descriptor block (like for the superblock)
- A bitmap of allocated blocks in the group
- A bitmap of allocated inodes numbers in the group
- A table of used inodes
- Data blocks

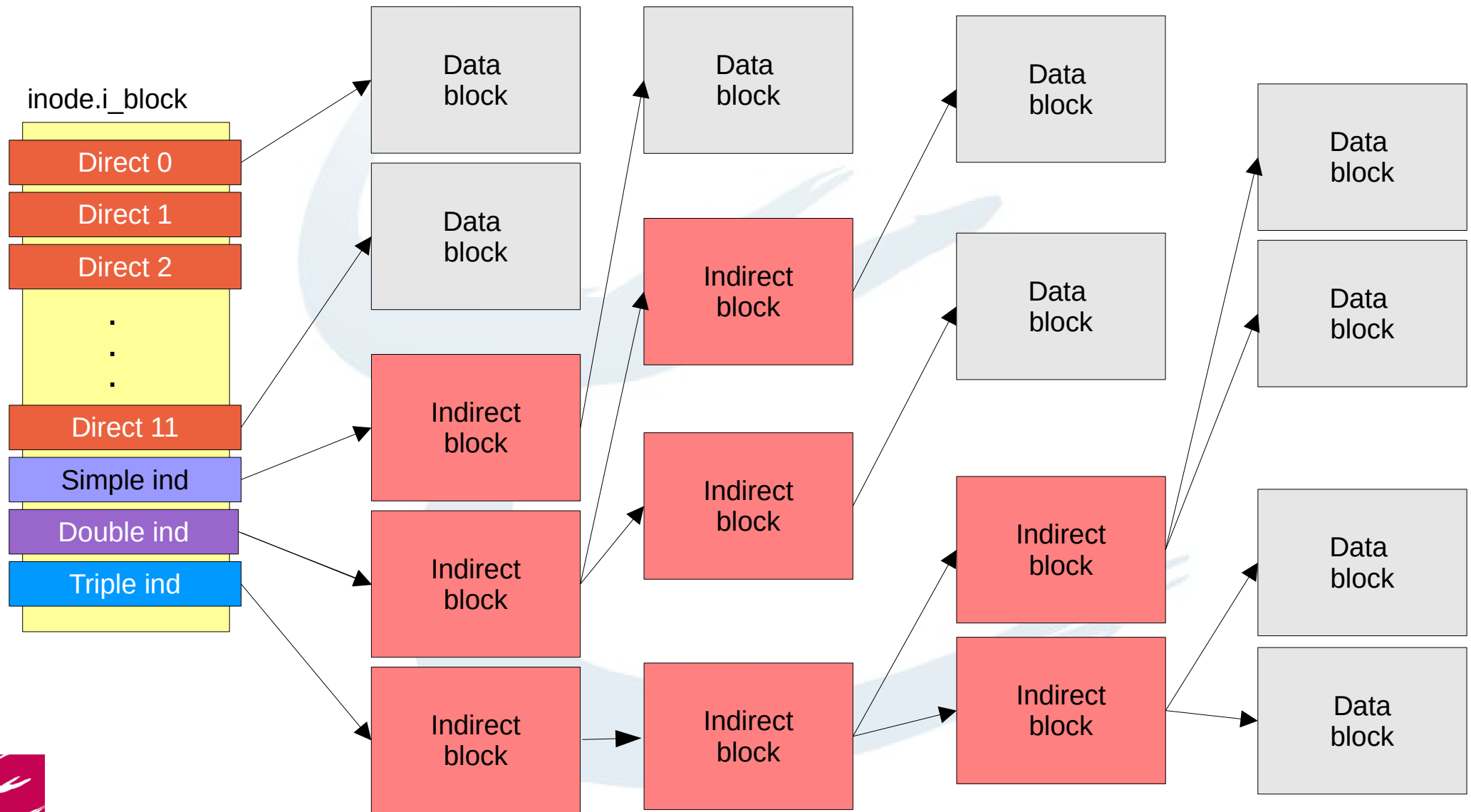


Inodes Table (ext2)

- Inode table is a set of blocks containing the inodes of the block
- Each inode is 128B (ext2)
 - 1kB blocks contain 8 inodes
 - 4kB blocks contain 32 inodes (defined by the FS structure)
- The total number of inodes is in the field `s_inode_per_group` of superblock
- The number of blocks used by the table depends on the size of a block and the number of inodes in the group

Inode

- An ext2 inode is very close to the virtual inode of the VFS
 - Type of file: regular, dir, link, char dev, block dev, pipe, socket
 - Contains the size of the file in bytes and in blocks
 - User id, group id, permissions
 - Access time, modification time
 - Hard link counter
 - Deletion time
- Contains a list of data blocks
 - `__le32[EXT2_N_BLOCKS=15] i_block;`
 - `i_blocks` is the 32bit block number
 - 12 first blocks are direct data
 - 13, 14, 15 are blocks of addresses of data blocks



i_block

- 15 block numbers (physical location on the disk)
 - First 12 are the first 12 data blocks of the file (block 0 to block 11)
 - 13th element is a block of simple indirection (block 12 to $11+b/4$)
 - 14th element is a block of double indirection (block $12+b/4$ to $11+b/4+b^2/16$)
 - 15th element is a block of triple indirection (block $12+b/4+b^2/16$ à $11+b/4+b^2/16+b^3/64$)
- Indirection blocks contain block numbers (physical location)
 - (block size / 4) blocks can be referenced (@32bit = 4B)
- Maximum number of blocks in a file:
 - $12+(b/4)+(b/4)^2+(b/4)^3$
 - With $b = 4096$, 48 kiB direct, 4 MiB simple indirection, 4 GiB double indirection and 4 TiB triple indirection

ext3

- Journaling filesystem
- Possibility to grow
- Directories with many files indexed by hash-tree
 - Better access time to files inside a directory
- Compatible ext2
 - Possibility to mount a ext2 filesystem, albeit without the advantages of ext3

Journaling

- Principle :
 - Write in the journal
 - Then write the data blocks
- If everything is fine
 - Journal is invalidate, we get the new file
- If crash during the writing of the data blocks
 - Copy the journal into the data blocks, get the new file
- If crash during the writing of the journal
 - Ignore the journal, keep the old file
- More robust, albeit slower

Journaling types

- Journal
 - Data and meta-data are written into the journal
 - More robust, slower
- Ordered
 - Only meta-data are written (number of blocks, etc)
 - Less robust (possible data loss for old file), default behavior
- Delayed
 - Only changes in meta-data are written, but not synchronization between journal and real data
 - Least robust, highest speed

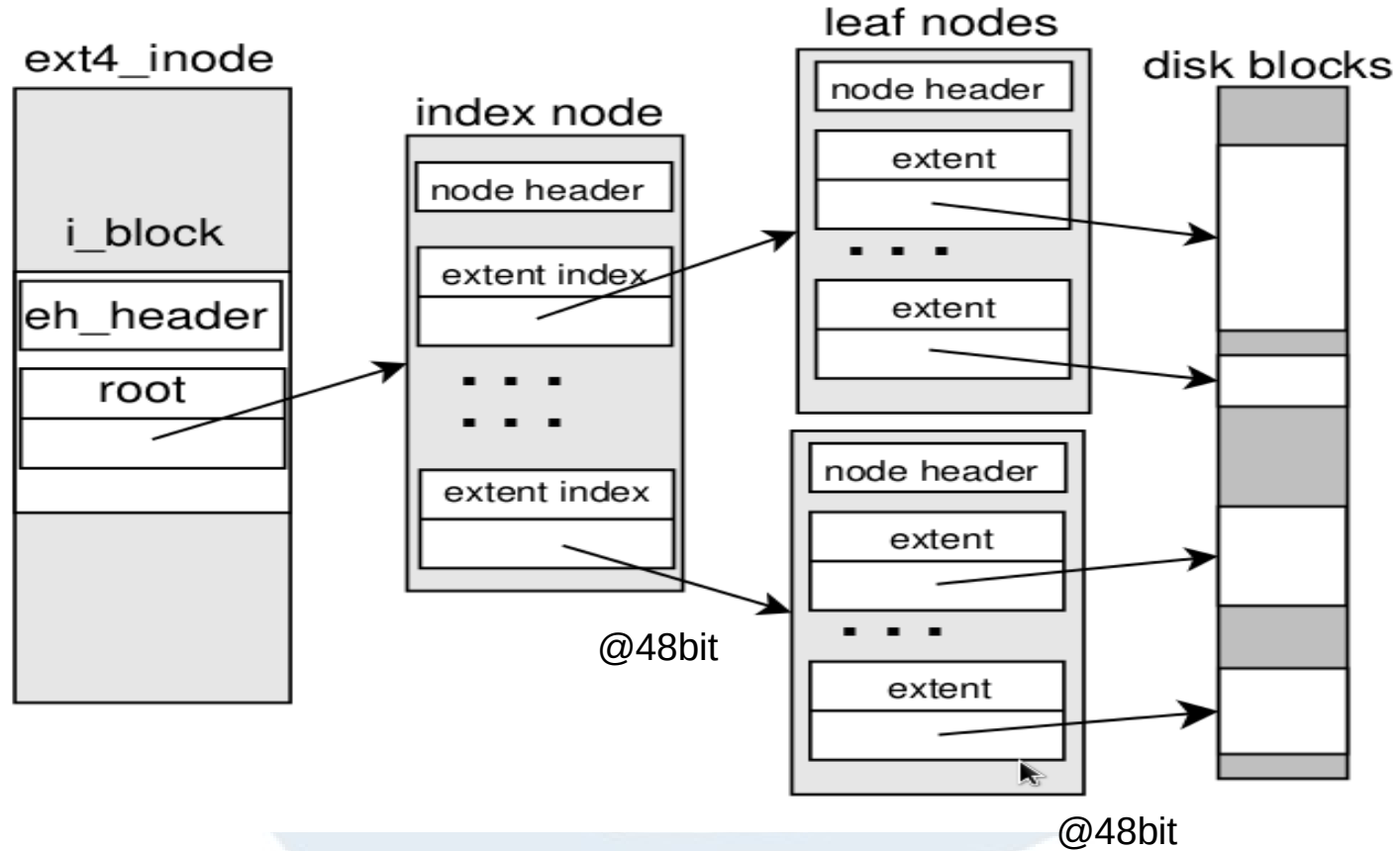
ext4

- 1 EiB (1 million TiB) max size
- Max file size: 16 TiB (with extents)
- Extents
 - Sets of continuous blocks
 - Easy management of big files (128 MiB per extent)
- Journal checksumming
 - Checks consistency of the journal
- Multiblocks allocation
 - Possibility to allocate several blocks at once when files are growing

ext4

- No limit to the number of files in a directory
- Delayed Allocation
 - Allocate blocks only when writing the data to the disk (less fragmentation)
- Fast fsck (does not check unused inodes)
- Barrier to ensure write order integrity
 - Disk controllers often re-arrange the order of the write instruction to optimize the speed of writing → force the order in some cases with barrier

Extents (1)



from *The new ext4 filesystem: current status and future plans*, A. Mathur 2007

Extents (2)

```
/*
 * ext4_inode has i_block array (60 bytes total).
 * The first 12 bytes store ext4_extent_header;
 * the remainder stores an array of ext4_extent.
 */

/*
 * Each block (leaves and indexes), even inode-stored has header.
 */
struct ext4_extent_header {
    __le16    eh_magic;        /* probably will support different formats */
    __le16    eh_entries;      /* number of valid entries */
    __le16    eh_max;          /* capacity of store in entries */
    __le16    eh_depth;        /* has tree real underlying blocks? */
    __le32    eh_generation;   /* generation of the tree */
};
```


Extents (3)

```
/*
 * This is the extent on-disk structure.
 * It's used at the bottom of the tree.
 */
struct ext4_extent {
    __le32ee_block;    /* first logical block extent covers */
    __le16ee_len;      /* number of blocks covered by extent */
    __le16ee_start_hi; /* high 16 bits of physical block */
    __le32ee_start_lo; /* low 32 bits of physical block */
};

/*
 * This is index on-disk structure.
 * It's used at all the levels except the bottom.
 */
struct ext4_extent_idx {
    __le32ei_block;    /* index covers logical blocks from 'block' */
    __le32ei_leaf_lo;  /* pointer to the physical block of the next *
                        * level. leaf or next index could be there */
    __le16ei_leaf_hi;  /* high 16 bits of physical block */
    __u16 ei_unused;
};
```

Btrfs

- Dynamic size
- Dynamic defragmentation
- Add/remove block devices
- Dynamic balancing between devices
- Subvolumes (mountable sub-trees)
- Snapshots
- RAID-like mirroring and striping
- ...

<https://btrfs.wiki.kernel.org/>

3IS – Operating Systems

–

Signals & Pipes (IPC 1)

2024 – Christophe Barès

christophe.bares@ensea.fr

Contributions: David Picard,
Arnaud Revel, Mickaël Maillard

Communication between processes

- Between parent and child
 - Data memory zone after `fork()` (but one way only)
 - Parameters of `exec()` (but one way only)
 - Anonymous pipe
- Between any 2 processes
 - Files
 - Named pipes

Problem: very slow solutions

Signals

- Easy method for communication between processes
- Raised by
 - Exception during execution
 - hardware: memory error, arithmetic exception
 - software: <CTRL>+C, SIGCHLD signal, on command (kill)
- When a process receives a signal
 - It's killed (by default)
 - Or a special handler is executed (if registred)

Signal types

/arch/x86/include/uapi/asm/signal.h

```
#define SIGHUP          1      /* Hangup (POSIX).  */
#define SIGINT          2      /* Interrupt (ANSI).  */
#define SIGQUIT         3      /* Quit (POSIX).  */
#define SIGILL          4      /* Illegal instruction (ANSI).  */
#define SIGTRAP         5      /* Trace trap (POSIX).  */
#define SIGABRT         6      /* Abort (ANSI).  */
#define SIGIOT          6      /* IOT trap (4.2 BSD).  */
#define SIGBUS          7      /* BUS error (4.2 BSD).  */
#define SIGFPE          8      /* Floating-point exception (ANSI).  */
#define SIGKILL         9      /* Kill, unblockable (POSIX).  */
#define SIGUSR1        10      /* User-defined signal 1 (POSIX).  */
#define SIGSEGV        11      /* Segmentation violation (ANSI).  */
#define SIGUSR2        12      /* User-defined signal 2 (POSIX).  */
#define SIGPIPE        13      /* Broken pipe (POSIX).  */
#define SIGALRM        14      /* Alarm clock (POSIX).  */
#define SIGTERM        15      /* Termination (ANSI).  */
#define SIGSTKFLT      16      /* Stack fault.  */
#define SIGCLD          SIGCHLD /* Same as SIGCHLD (System V).  */
#define SIGCHLD        17      /* Child status has changed (POSIX).  */
#define SIGCONT        18      /* Continue (POSIX).  */
#define SIGSTOP        19      /* Stop, unblockable (POSIX).  */
#define SIGTSTP        20      /* Keyboard stop (POSIX).  */
#define SIGTTIN        21      /* Background read from tty (POSIX).  */
#define SIGTTOU        22      /* Background write to tty (POSIX).  */
#define SIGURG         23      /* Urgent condition on socket (4.2 BSD).  */
#define SIGXCPU        24      /* CPU limit exceeded (4.2 BSD).  */
#define SIGXFSZ        25      /* File size limit exceeded (4.2 BSD).  */
#define SIGVTALRM      26      /* Virtual alarm clock (4.2 BSD).  */
#define SIGPROF        27      /* Profiling alarm clock (4.2 BSD).  */
#define SIGWINCH       28      /* Window size change (4.3 BSD, Sun).  */
#define SIGPOLL        SIGIO   /* Pollable event occurred (System V).  */
#define SIGIO          29      /* I/O now possible (4.2 BSD).  */
#define SIGPWR         30      /* Power failure restart (System V).  */
#define SIGSYS         31      /* Bad system call.  */
```

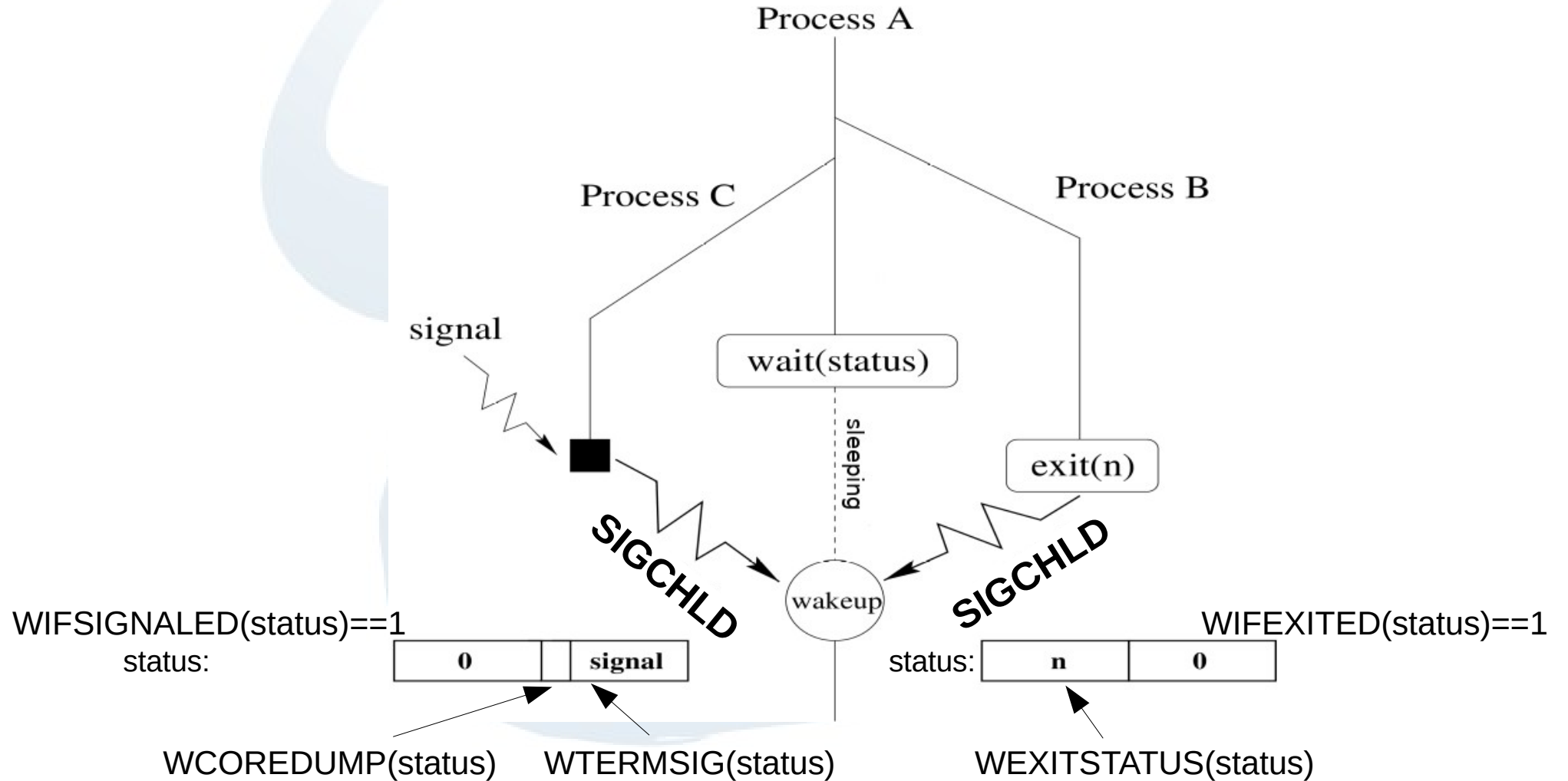
+ 32 "Real Time" signals == user defined

(see kill -l)

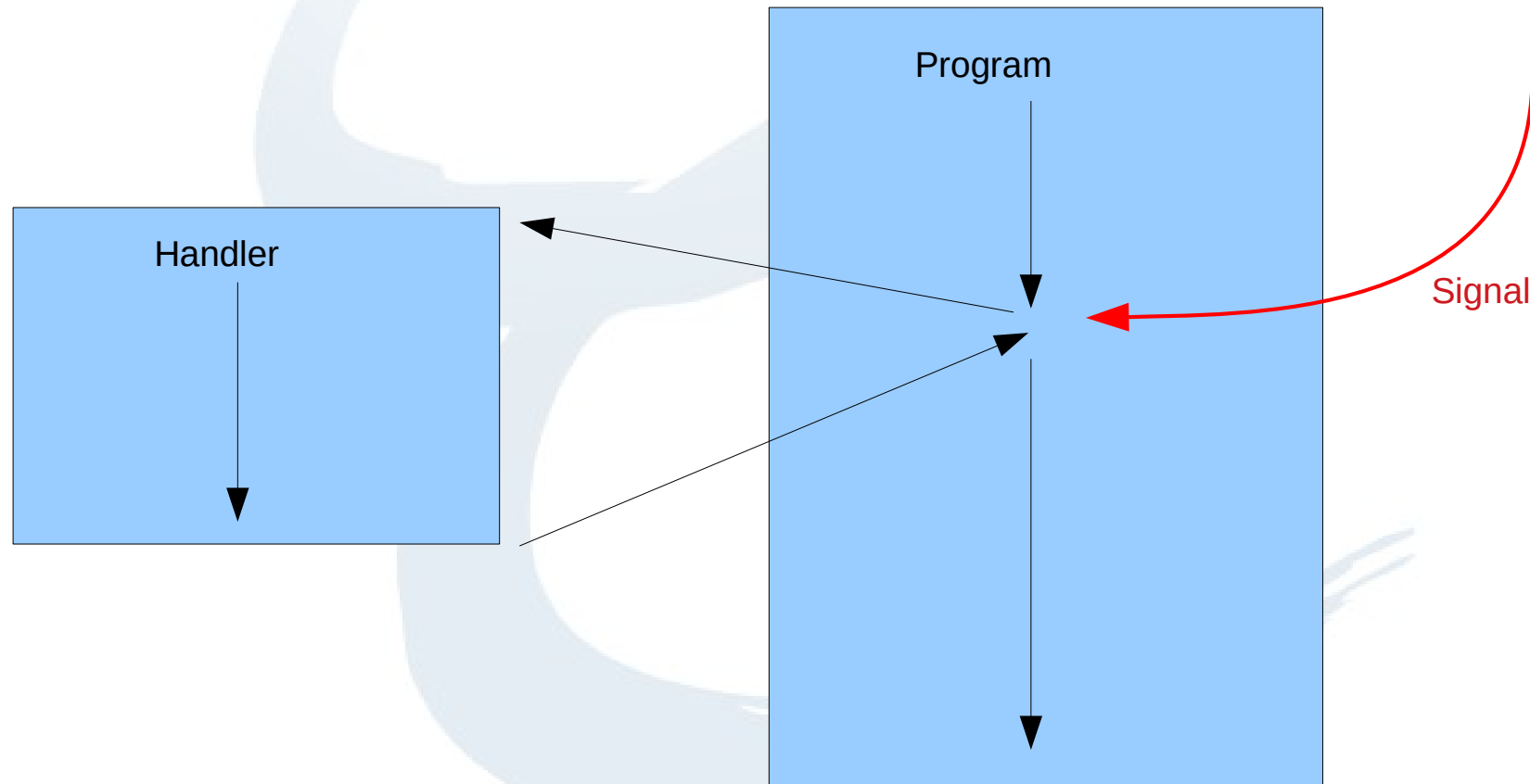
Signals are not interrupts!

- During an interrupt, the associated handler is run **immediately**
- When a signal is received, the associated handler will be run when the process get **scheduled** (next time it has access to the CPU)
- Signals are not **real time!** (not even those called that)

Default behavior



Signal handler



signal()

```
#include <signal.h>

void spsig(int sig)
{
    printf("Received the following signal: %d\n", sig);
}

int main()
{
    signal(SIGINT, spsig);
    pause();
    exit(0);
}
```

kill()

```
#include <stdio.h>

void spsig(int sig)
{
    printf("Signal %d received\n", sig);
}

int main()
{
    int idchild, status;

    if (idchild=fork()) { /* parent */
        sleep(5);
        kill(idchild, SIGUSR1);
        wait(&status);
        exit(0);
    }
    else { /* child */
        signal(SIGUSR1, spsig);
        pause();
        exit(1);
    }
}
```

Real time signals

- At least 8 “real-time” signals in norm POSIX 1.b
- Linux has 31 such signals from 34 (SIGRTMIN) to 64 (SIGRTMAX)
- Real time signals have no predetermined signification (up to the user to give a meaning to the signals)
- Default behavior is to kill the process
- If standard signals and real-time signals are received by a waiting process, at next schedule, standard signals will be handled first (on Linux)

Not exactly real time!!!

- The term "real-time" is **misleading** !
- **No temporal constraint** on the scheduler
- As for standard signals, the handler will be executed **only when the process is scheduled** (next time it access the CPU)

Properties

- Many instances of a real time signal can be stacked
- A real time signal can be raised by `sigqueue( )` with a parameter
- A handler using `sigaction( )` can access this parameter
- When several real time signals are raised, they are delivered starting from the lowest value (i.e. signals with the highest number have the lowest priority).
- Real time signals of same type are delivered respecting the emission order.

Handler example

```
#include <stdio.h> <stdlib.h> <sys/types.h> <unistd.h> <signal.h> <sys/wait.h>
```

```
void handler(int sig, siginfo_t *info, void *unused)
{
    printf("Receiving signal %d ", sig);
    switch ( info->si_code ){
    case SI_USER:
        printf("raised using kill() or raise() by process %d\n", info->si_pid);
        break;
    case SI_QUEUE:
        printf("raised using sigqueue() by process %d with parameter value %d\n",
            info->si_pid, info->si_value.sival_int);
        break;
    default:
        printf("\n");
        break;
    }
}
```

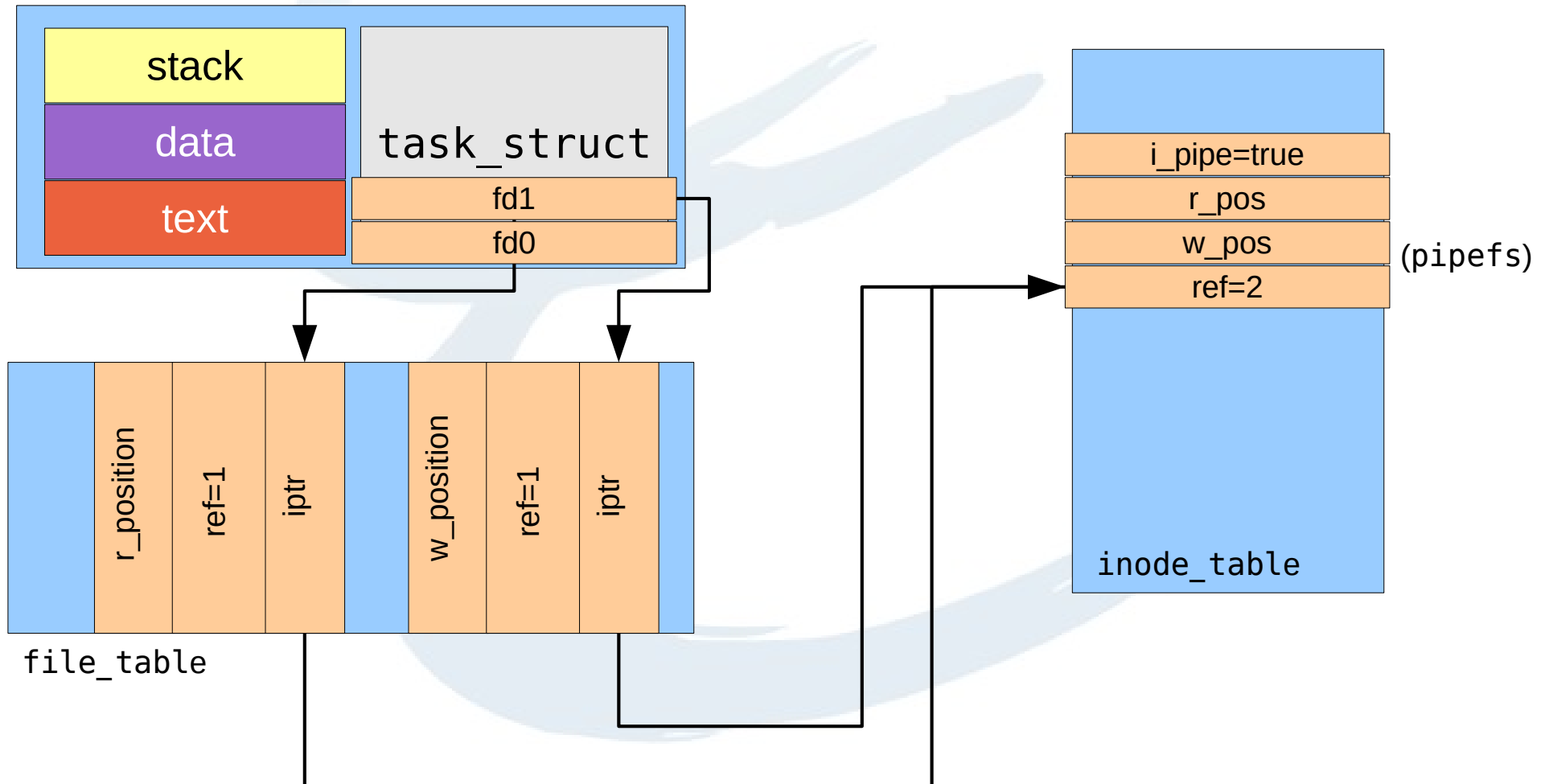
Calling example

```
int main()
{
    pid_t pid_child;
    switch ( pid_child = fork() ){
    case -1: perror("fork() error"); return EXIT_FAILURE;
    case 0:
        { sigset_t mask; struct sigaction action;
          sigemptyset(&mask); // clear signal mask
          sigaddset(&mask, SIGUSR1); // set a sig in mask
          sigprocmask(SIG_BLOCK, &mask, NULL); // add mask to current
          action.sa_sigaction = handler;
          action.sa_mask = mask;
          action.sa_flags = SA_SIGINFO;
          sigaction(SIGUSR1, &action, NULL);
          sigprocmask(SIG_UNBLOCK, &mask, NULL);
          while(1);
        }
    default:
        { sleep(1); // wait for the son to configure his handler
          union sigval val;
          val.sival_int = 123;
          sigqueue(pid_child, SIGUSR1, val);
          wait(NULL);
        }
    }
    return EXIT_SUCCESS;
}
```


Pipes

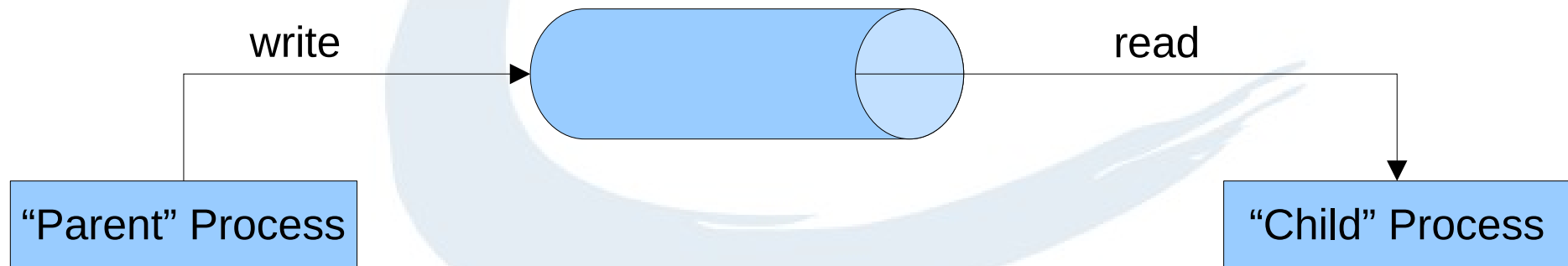
- A pipe is a virtual file (not stored on hdd)
- It has 2 file descriptors: one for reading, one for writing
- As file descriptors are shared between parent and child, pipe allows communication
- Backed by a circular buffer

pipe()



pipe()

- Parent process writes to the pipe
- Child reads from the pipe
- Or reverse...



Pipe() usage

- 1) Call pipe() => fd[0] et fd[1]
- 2) Call fork() => 2 process with same fd
 - Parent close one fd: close(fd[0])
 - Child close the other fd: close(fd[1])
- 3) Parent => Child communication:
 - Parent writes on its fd: write(fd[1], ...)
 - Child reads on its fd: read(fd[0], ...)

For Parent <= Child communication :

- Swap descriptor fd[0] and fd[1]

For Parent <=> Child (bidir.) communication:

- 2 calls to pipe() and crossed close()

Example with 1 pipe

```
int main()
{
    int status, pipedes[2];
    char buf[100];
    int len = 0;
    char *msg = "Hello, child !";
    Struct stat stat pipestat;

    if (pipe(pipedes)) exit(1);
    if (fork()) { /* parent */
        write(pipedes[1], msg, strlen(msg));
        wait(&status);
        return;
    }
    else { /* child */
        While( len==0 ) /* wait for data */
        {
            if (fstat(pipedes[0], &pipestat)) exit(3);
            len = (int)pipestat.st_size;
        }
        read(pipedes[0], buf, len);
        printf("My father told me : %s\n", buf);
        return;
    }
}
```

Example with 2 pipes

```
int pipdes1[2],pipdes2[2];

void child()
{
    char buff[21];

    /* close pipe 1 for writing
       and 0 for reading */
    close(pipdes1[1]);
    close(pipdes2[0]);

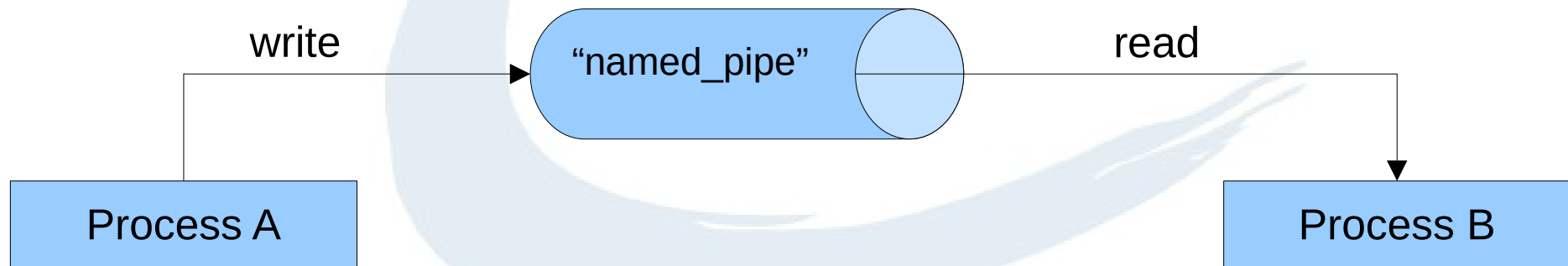
    read(pipdes1[0],buff,21);
    printf("child : %s\n",buff);
    write(pipdes2[1],"Message received",13);
    return;
}
```

```
int main()
{
    int status;
    char bufp[13];
    if (pipe(pipdes1))    exit(1);
    if (pipe(pipdes2))    exit(1);

    if (fork())
    {
        /* close pipe 0 for writing
           and 1 for reading */
        close(pipdes1[0]);
        close(pipdes2[1]);
        write(pipdes1[1],"Hello child!",21);
        read(pipdes2[0],bufp,13);
        printf("Parent: %s\n",bufp);
        wait(&status);
        exit(0);
    }
    else
    {
        fils();
    }
}
```

Named pipes

- Special file on the filesystem (p) created by `mkfifo()`
- Reserve inode in the filesystem
- Not used as a regular file but as a pipe (no underlying data saved on disk)



Example: create and write

```
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#define NAME_PIPE "./my_pipe"

int main(int ac, char **av)
{
    char DATA[32];
    int fd;

    mkfifo(NAME_PIPE, 0666);
    fd = open(NAME_PIPE, O_WRONLY);
    if (fd == -1)
        exit(1);

    sprintf(DATA, "%i say hello", getpid());
    write(fd, DATA, strlen(DATA) + 1);
    sleep(5);
    return 0;
}
```


Example: open and read

```
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>

#define NAME_PIPE "./my_pipe"
#define BUFFSIZE 1024

int main(int ac, char **av)
{
    char sbBuf[BUFFSIZE];
    int fd;

    fd = open(NAME_PIPE, "r");
    read(fd, sbBuf, BUFFSIZE);
    printf("[%i] %s\n", getpid(), sbBuf);
    return 0;
}
```

Sockets

- Extension of named pipes to **distant computers**
- Appeared 1983 in 4.2BSD
- Work like files but,
- Initialization to specify network protocols

Socket creation

```
int socket(int domain, int type, int protocol)
```

- Communication domain (layer 3) (PF_X or AF_X)
 - PF_INET (IPv4), PF_INET6 (IPv6)
 - PF_LOCAL==PF_UNIX (local to host)
 - PF_PACKET (raw): access to layer 2
- Socket type
 - SOCK_STREAM (connected) => TCP
 - SOCK_DGRAM (not connected) => UDP
 - SOCK_RAW
- Protocol (layer 4):
 - IPPROTO_TCP, IPPROTO_UDP... – can be zero (guessed by the OS)

Example

```
#include <sys/socket.h>

int main() {

    int sock;

    if((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        printf("error socket\n");
        return -1;
    }

    return 0;
}
```

Server side

- Attach socket to (address, port):

```
bind(int sock, struct sockaddr *a, socklen_t len)
```

- Listening for connection (listen):

```
listen(int sock, int queue_size)
```

- Wait for a client connection (blocking):

```
accept(int sock, struct sockaddr *a, socklen_t *len)
```

- And after:

- Usually, you fork()
- Parent process returns to “accept”
- Child process does the processing

Example

```
int main(int argc, char *argv[])
{
    int sfd, cfd; struct sockaddr_un my_addr, peer_addr; socklen_t peer_addr_size;

    sfd = socket(AF_UNIX, SOCK_STREAM, 0); if (sfd == -1) handle_error("socket");

    memset(&my_addr, 0, sizeof(struct sockaddr_un));
    my_addr.sun_family = AF_UNIX;
    strncpy(my_addr.sun_path, MY_SOCKET_PATH, sizeof(my_addr.sun_path) - 1);

    if (bind(sfd, (struct sockaddr *) &my_addr, sizeof(struct sockaddr_un)) == -1)
        handle_error("bind");
    if (listen(sfd, LISTEN_BACKLOG) == -1)
        handle_error("listen");

    while(1) {
        peer_addr_size = sizeof(struct sockaddr_un);
        cfd = accept(sfd, (struct sockaddr *) &peer_addr, &peer_addr_size);
        if (cfd == -1) handle_error("accept");
        if(fork()) { /* parent */
            close(cfd); continue;
        } else { /* child */
            close(sfd); /* processing */
        }
    }
}
```

Creation

```
int main(int argc, char *argv[])
{
    int sfd, cfd; struct sockaddr_un my_addr, peer_addr; socklen_t peer_addr_size;

    sfd = socket(AF_UNIX, SOCK_STREAM, 0); if (sfd == -1) handle_error("socket");

    memset(&my_addr, 0, sizeof(struct sockaddr_un));
    my_addr.sun_family = AF_UNIX;
    strncpy(my_addr.sun_path, MY_SOCKET_PATH, sizeof(my_addr.sun_path) - 1);

    if (bind(sfd, (struct sockaddr *) &my_addr, sizeof(struct sockaddr_un)) == -1)
        handle_error("bind");
    if (listen(sfd, LISTEN_BACKLOG) == -1)
        handle_error("listen");

    while(1) {
        peer_addr_size = sizeof(struct sockaddr_un);
        cfd = accept(sfd, (struct sockaddr *) &peer_addr, &peer_addr_size);
        if (cfd == -1) handle_error("accept");
        if(fork()) { /* parent */
            close(cfd); continue;
        } else { /* child */
            close(sfd); /* processing */
        }
    }
}
```

struct addr initialization

```
int main(int argc, char *argv[])
{
    int sfd, cfd; struct sockaddr_un my_addr, peer_addr; socklen_t peer_addr_size;

    sfd = socket(AF_UNIX, SOCK_STREAM, 0); if (sfd == -1) handle_error("socket");

    memset(&my_addr, 0, sizeof(struct sockaddr_un));
    my_addr.sun_family = AF_UNIX;
    strncpy(my_addr.sun_path, MY_SOCKET_PATH, sizeof(my_addr.sun_path) - 1);

    if (bind(sfd, (struct sockaddr *) &my_addr, sizeof(struct sockaddr_un)) == -1)
        handle_error("bind");
    if (listen(sfd, LISTEN_BACKLOG) == -1)
        handle_error("listen");

    while(1) {
        peer_addr_size = sizeof(struct sockaddr_un);
        cfd = accept(sfd, (struct sockaddr *) &peer_addr, &peer_addr_size);
        if (cfd == -1) handle_error("accept");
        if(fork()) { /* parent */
            close(cfd); continue;
        } else { /* child */
            close(sfd); /* processing */
        }
    }
}
```


Attachment

```
int main(int argc, char *argv[])
{
    int sfd, cfd; struct sockaddr_un my_addr, peer_addr; socklen_t peer_addr_size;

    sfd = socket(AF_UNIX, SOCK_STREAM, 0); if (sfd == -1) handle_error("socket");

    memset(&my_addr, 0, sizeof(struct sockaddr_un));
    my_addr.sun_family = AF_UNIX;
    strncpy(my_addr.sun_path, MY_SOCKET_PATH, sizeof(my_addr.sun_path) - 1);

    if (bind(sfd, (struct sockaddr *) &my_addr, sizeof(struct sockaddr_un)) == -1)
        handle_error("bind");
    if (listen(sfd, LISTEN_BACKLOG) == -1)
        handle_error("listen");

    while(1) {
        peer_addr_size = sizeof(struct sockaddr_un);
        cfd = accept(sfd, (struct sockaddr *) &peer_addr, &peer_addr_size);
        if (cfd == -1) handle_error("accept");
        if(fork()) { /* parent */
            close(cfd); continue;
        } else { /* child */
            close(sfd); /* processing */
        }
    }
}
```

Listen

```
int main(int argc, char *argv[])
{
    int sfd, cfd; struct sockaddr_un my_addr, peer_addr; socklen_t peer_addr_size;

    sfd = socket(AF_UNIX, SOCK_STREAM, 0); if (sfd == -1) handle_error("socket");

    memset(&my_addr, 0, sizeof(struct sockaddr_un));
    my_addr.sun_family = AF_UNIX;
    strncpy(my_addr.sun_path, MY_SOCKET_PATH, sizeof(my_addr.sun_path) - 1);

    if (bind(sfd, (struct sockaddr *) &my_addr, sizeof(struct sockaddr_un)) == -1)
        handle_error("bind");
    if (listen(sfd, LISTEN_BACKLOG) == -1)
        handle_error("listen");

    while(1) {
        peer_addr_size = sizeof(struct sockaddr_un);
        cfd = accept(sfd, (struct sockaddr *) &peer_addr, &peer_addr_size);
        if (cfd == -1) handle_error("accept");
        if(fork()) { /* parent */
            close(cfd); continue;
        } else { /* child */
            close(sfd); /* processing */
        }
    }
}
```

New connection

```
int main(int argc, char *argv[])
{
    int sfd, cfd; struct sockaddr_un my_addr, peer_addr; socklen_t peer_addr_size;

    sfd = socket(AF_UNIX, SOCK_STREAM, 0); if (sfd == -1) handle_error("socket");

    memset(&my_addr, 0, sizeof(struct sockaddr_un));
    my_addr.sun_family = AF_UNIX;
    strncpy(my_addr.sun_path, MY_SOCKET_PATH, sizeof(my_addr.sun_path) - 1);

    if (bind(sfd, (struct sockaddr *) &my_addr, sizeof(struct sockaddr_un)) == -1)
        handle_error("bind");
    if (listen(sfd, LISTEN_BACKLOG) == -1)
        handle_error("listen");

    while(1) {
        peer_addr_size = sizeof(struct sockaddr_un);
        cfd = accept(sfd, (struct sockaddr *) &peer_addr, &peer_addr_size);
        if (cfd == -1) handle_error("accept");
        if(fork()) { /* parent */
            close(cfd); continue;
        } else { /* child */
            close(sfd); /* processing */
        }
    }
}
```

Back to listen

```
int main(int argc, char *argv[])
{
    int sfd, cfd; struct sockaddr_un my_addr, peer_addr; socklen_t peer_addr_size;

    sfd = socket(AF_UNIX, SOCK_STREAM, 0); if (sfd == -1) handle_error("socket");

    memset(&my_addr, 0, sizeof(struct sockaddr_un));
    my_addr.sun_family = AF_UNIX;
    strncpy(my_addr.sun_path, MY_SOCKET_PATH, sizeof(my_addr.sun_path) - 1);

    if (bind(sfd, (struct sockaddr *) &my_addr, sizeof(struct sockaddr_un)) == -1)
        handle_error("bind");
    if (listen(sfd, LISTEN_BACKLOG) == -1)
        handle_error("listen");

    while(1) {
        peer_addr_size = sizeof(struct sockaddr_un);
        cfd = accept(sfd, (struct sockaddr *) &peer_addr, &peer_addr_size);
        if (cfd == -1) handle_error("accept");
        if(fork()) { /* parent */
            close(cfd); continue;
        } else { /* child */
            close(sfd); /* processing */
        }
    }
}
```

processing

```
int main(int argc, char *argv[])
{
    int sfd, cfd; struct sockaddr_un my_addr, peer_addr; socklen_t peer_addr_size;

    sfd = socket(AF_UNIX, SOCK_STREAM, 0); if (sfd == -1) handle_error("socket");

    memset(&my_addr, 0, sizeof(struct sockaddr_un));
    my_addr.sun_family = AF_UNIX;
    strncpy(my_addr.sun_path, MY_SOCKET_PATH, sizeof(my_addr.sun_path) - 1);

    if (bind(sfd, (struct sockaddr *) &my_addr, sizeof(struct sockaddr_un)) == -1)
        handle_error("bind");
    if (listen(sfd, LISTEN_BACKLOG) == -1)
        handle_error("listen");

    while(1) {
        peer_addr_size = sizeof(struct sockaddr_un);
        cfd = accept(sfd, (struct sockaddr *) &peer_addr, &peer_addr_size);
        if (cfd == -1) handle_error("accept");
        if(fork()) { /* parent */
            close(cfd); continue;
        } else { /* child */
            close(sfd); /* processing */
        }
    }
}
```

Client side

```
int connect(int sock, struct sockaddr *a, int len)
```

- Socket creation using `socket()`
- Open a connection to (address, port)
- No need to `bind()`, source port is dynamically attributed

Example

```
#define MY_ADDR ""

int main(int argc, char *argv[])
{
    int cfd; struct sockaddr_in my_addr;
    char buffer[64];

    cfd = socket(AF_INET, SOCK_STREAM, 0);
    if (cfd == -1) handle_error("socket");

    memset(&my_addr, 0, sizeof(struct sockaddr_in));
    my_addr.sin_family = AF_INET;
    strncpy(my_addr.sin_addr, MY_ADDR, sizeof(my_addr.sin_addr) - 1);

    if(connect(cfd, &my_addr, sizeof(struct sockaddr_in)) < 0) {
        handle_error("connect");
    }

    read(cfd, buffer, 64);
    printf("read : %s\n", buffer);

    return 0;
}
```

3IS – Operating Systems

–

System V IPC (IPC 2)

2024 – Christophe Barès

christophe.bares@ensea.fr

Contributions: David Picard,
Arnaud Revel, Mickaël Maillard

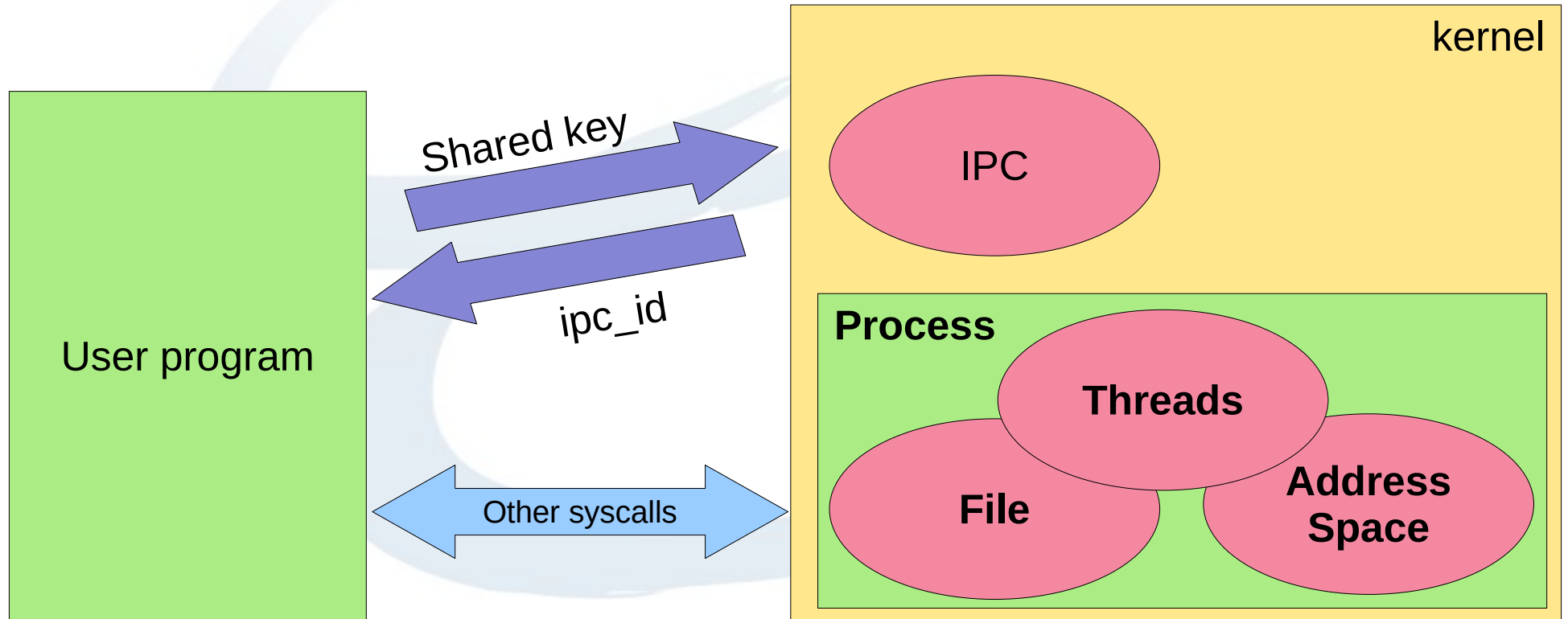
Inter-Process Communication

- 3 types of inter-process communications:
 - Message queues
 - “mail boxes” where processes can send or receive message
 - Shared memory
 - Memory zone where processes can write and read data
 - Semaphores
 - Data structures used for managing the access to shared resources

General Principles

- A shared key allows the access to an IPC
- The process that creates the IPC defines its access permissions
- IPC management is an independent kernel mechanism, which allows for external access or modification of IPC properties.
- IPC management is an extremely important attribute of the OS (present even in micro-kernel architecture)

General Principles



Shared Key Creation

```
key_t ftok(const char *pathname, int proj_id);
```

- pathname: path to a accessible file
- proj_id: 8 bit integer constant

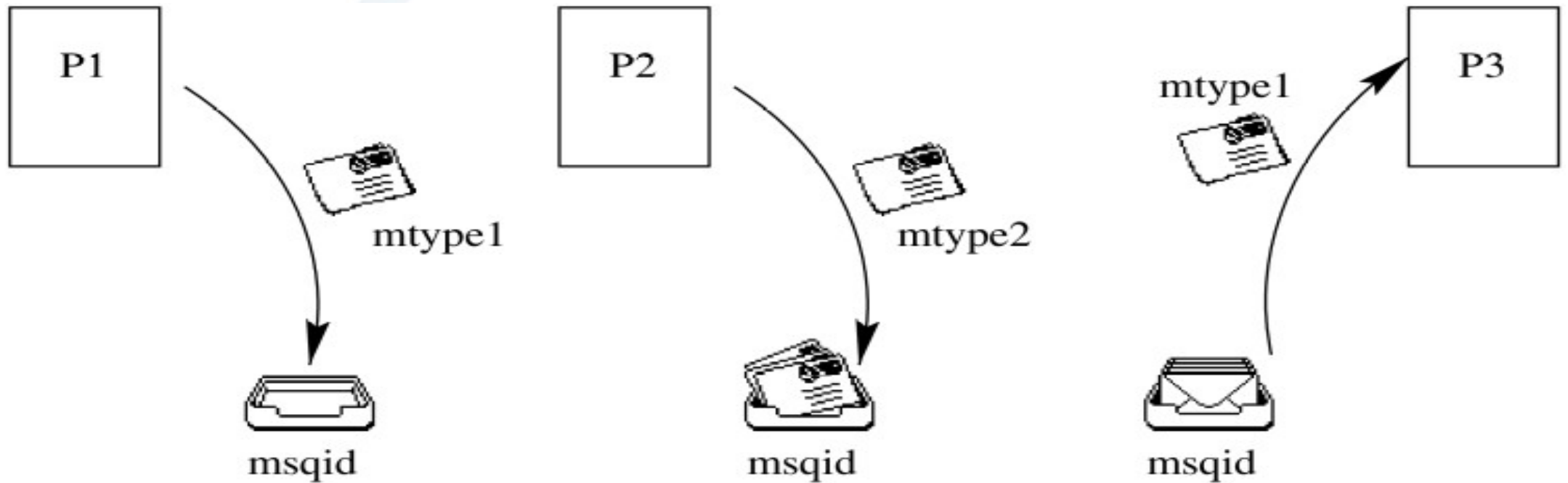
The return value is a key that can be used with every System V IPC

IPC_PRIVATE is a special non shareable key
(except in a parent-child context)

Message queues

- A kind of mail box.
- A process can put or get a message if it has the corresponding key.
- Each message can be associated with a type which allows processes to select only the interesting messages.
- Each process has a number linked to the message types which is used to identify the message's recipient.

Message queues



msqid_ds

- The message queue is defined by:
 - An id msqid
 - A structure msqid_ds containing information on the queue

```
/* one msqid structure for each queue on the system */
struct msqid_ds {
    struct ipc_perm msg_perm;
    struct msg *msg_first; /* first message on queue */
    struct msg *msg_last; /* last message in queue */
    time_t msg_stime;      /* last msgsnd time */
    time_t msg_rtime;      /* last msgrcv time */
    time_t msg_ctime;      /* last change time */
    struct wait_queue *wwait;
    struct wait_queue *rwait;
    ushort msg_cbytes;
    ushort msg_qnum;
    ushort msg_qbytes;     /* max number of bytes on queue */
    ushort msg_lspid;      /* pid of last msgsnd */
    ushort msg_lrpid;      /* last receive pid */
};
```

Creation example

```
#include <stdlib.h>
#include <stdio.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <fcntl.h>

int main()
{
    key_t key;
    int msqid;

    key = ftok("Makefile", 0x21);
    if ((msqid = msgget(key, S_IRWXU | IPC_CREAT)) == -1) {
        perror("msgget");
        exit(1);
    }
    printf("key : %x msqid : %d\n", key, msqid);
    exit(0);
}
```


Getting information

```
#include <sys/types.h> #include <sys/ipc.h> #include <sys/msg.h>
#include <stdio.h> #include <stdlib.h>
```

```
int main()
{
    key_t key;
    int msqid;
    struct msqid_ds buf;

    if ((key = ftok("Makefile", 0x21)) == -1) perror("ftok");
    if ((msqid = msgget(key, 0)) == -1) perror("msgget");
    printf("key : %x msqid : %d\n", key, msqid);

    if (msgctl(msqid, IPC_STAT, &buf) == -1) perror("msgctl");
    printf("User ID: %d\n", buf.msg_perm.uid);
    printf("Groupe ID: %d\n", buf.msg_perm.gid);
    printf("Permissions: %d\n", buf.msg_perm.mode);
    printf("Length: %ld\n", buf.msg_qbytes);
    printf("Msg. number: %ld\n", buf.msg_qnum);
    exit(0);
}
```

Sending messages

```
#include <sys/types.h> #include <sys/ipc.h> #include <sys/msg.h>
#include <stdio.h> #include <stdlib.h> #include <string.h>

#define BSIZE 80

int main()
{
    key_t key;
    int msqid;
    struct msgb {
        long mtype;
        char mtext[BSIZE];
    } msgp;
    char *msg = "Very Important Message!";

    if ((key = ftok("Makefile", 0x21)) == -1) perror("ftok");
    if ((msqid = msgget(key, 0)) == -1) perror("msgget");

    msgp.mtype = 42L;
    strncpy(msgp.mtext, msg, BSIZE);
    if (msgsnd(msqid, &msgp, strlen(msg), IPC_NOWAIT) == -1) {
        perror("msgsnd");
        exit(1);
    }
    exit(0);
}
```

Receiving messages

```
#include <sys/types.h> #include <sys/ipc.h> #include <sys/msg.h>
#include <stdio.h> #include <stdlib.h> #include <string.h>

#define BSIZE 80

int main()
{
    key_t key;
    int msqid, rtn, flags;
    long mtype;
    struct msgb {
        long mtype;
        char mtext[BSIZE];
    } msgp;

    if ((key = ftok("Makefile", 0x21)) == -1) perror("ftok");
    if ((msqid = msgget(key, 0)) == -1) perror("msgget");

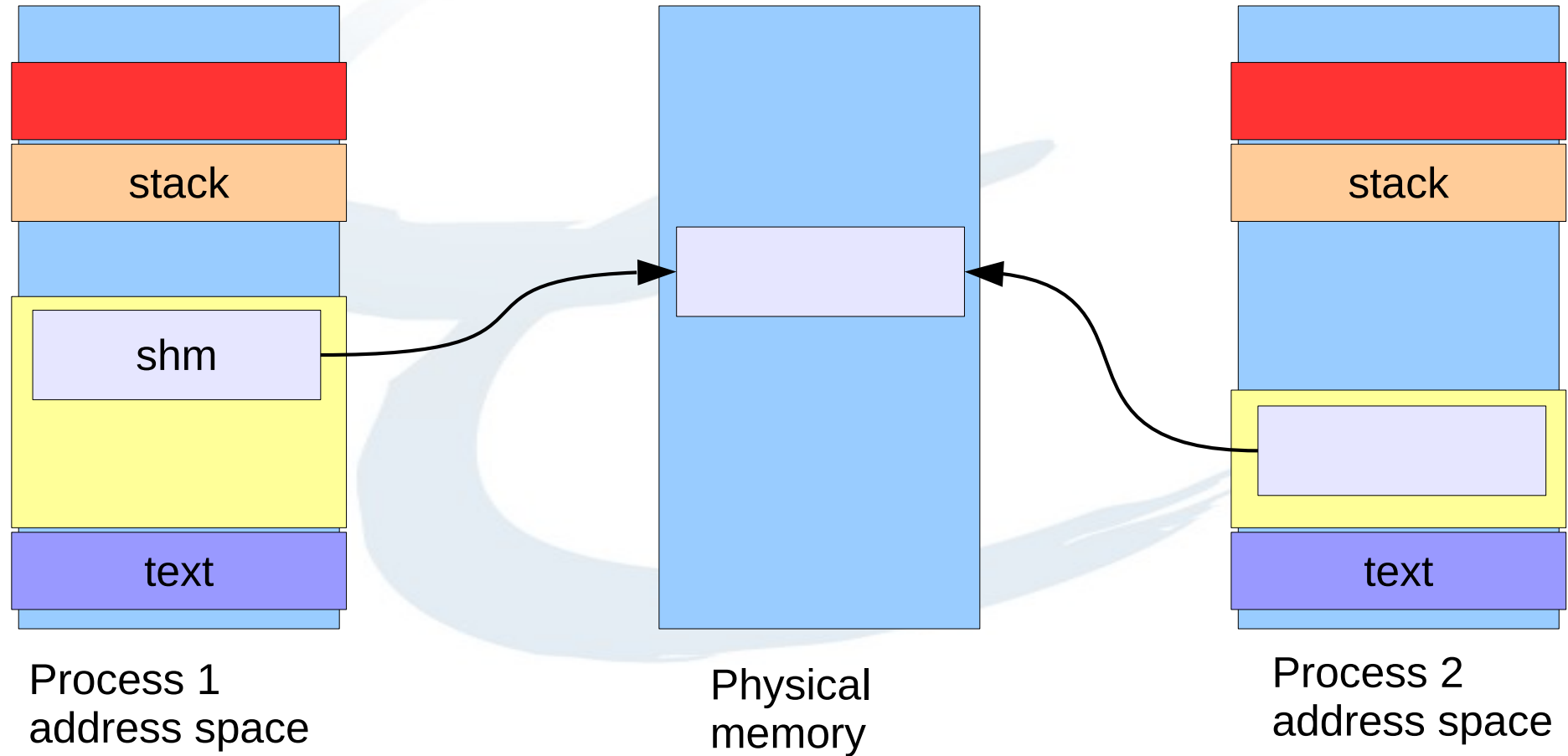
    mtype = 42L;
    flags = IPC_NOWAIT | MSG_NOERROR;
    if ((rtn = msgrcv(msqid, &msgp, sizeof(msgp) - 1, mtype, flags)) == -1) perror("msgrcv");

    msgp.mtext[rtn] = '\0';    /* end of string */
    printf("%d characters: %s\n", rtn, msgp.mtext);
    exit(0);
}
```

Shared memory

- Allows to map a memory block to several process address space
- Once the block is attached, it can be used like every normally allocated memory block

Shared memory



Attachment

- 2 steps:
 - Creation of the shared memory segment using the id
 - `shmget()`
 - Doing the mapping into the process address space (get the starting address of the segment)
 - `shmat()`

Creation example

```
#include <stdlib.h> #include <stdio.h> #include <sys/types.h>
#include <sys/ipc.h> #include <sys/shm.h> #include <fcntl.h>

#define SHM_SIZE 100

int main()
{
    key_t key;
    int shmid;

    key = ftok("Makefile", 0x25);
    if ((shmid = shmget(key, SHM_SIZE, S_IRWXU | IPC_CREAT)) == -1) {
        perror("shmget");
        exit(1);
    }
    printf("%s%d\n", "shared memory id is ", shmid);
}
```

Write example

```
#include <sys/types.h> #include <sys/ipc.h> #include <sys/shm.h>
#include <stdio.h> #include <stdlib.h> #include <string.h>

#define BSIZE 100

int main()
{
    key_t key;
    int shmid;
    char *buf;
    char *msg = "shared memory random message";

    if ((key = ftok("Makefile", 0x25)) == -1) perror("ftok");
    if ((shmid = shmget(key, 0, 0)) == -1) perror("shmget");
    if ((buf = shmat(shmid, NULL, 0)) == (char *)-1) perror("shmat");

    *buf = 0;                                /* segment is free */
    strncpy(buf + 1, msg, BSIZE - 1); /* writing msg */
    *buf = 1;                                /* message is written */
    while (*buf) ;                          /* wait for message read */
    shmdt(buf);                             /* detach segment */
}
```


Read example

```
#include <stdlib.h> #include <stdio.h> #include <sys/types.h>
#include <sys/ipc.h> #include <sys/shm.h> #include <string.h>

#define BSIZE 100
int main()
{
    key_t key;
    int shmid;
    char *buf;
    char msg[100];

    if ((key = ftok("Makefile", 0x25)) == -1) perror("ftok");
    if ((shmid = shmget(key, 0, 0)) == -1) perror("shmget");
    if ((buf = shmat(shmid, 0, 0)) == (char *)-1) perror("shmat");

    while (*buf != 1) ;           /* waiting for a message */
    strncpy(msg, buf + 1, BSIZE - 1); /* read message */
    *buf = 0;                     /* say message is read */
    buf[BSIZE] = '\0';            /* end of string */
    printf("%s\n", msg);          /* print message */
    shmdt(buf);                   /* detach */
}
```

Semaphores

- Some resources are accessed by several processes
- However, they cannot be used by these processes at the same time
 - For hardware reasons (printers, files, ...)
 - For data integrity reasons (shared memory, ...)
- Semaphores are used for managing access permissions the shared resources

Example: bank transaction

- Let's imagine the following process:
 - READ X (bank account value)
 - ADD X, C (put some money on the account)
 - WRITE X (save new bank account value)
- When several processes are occurring at the same time, we have many scenarios

Scenario 1

- READ X
- ADD X, 1
- WRITE X
- —
- —
- —

- —
- —
- —
- READ X
- ADD X, 1
- WRITE X

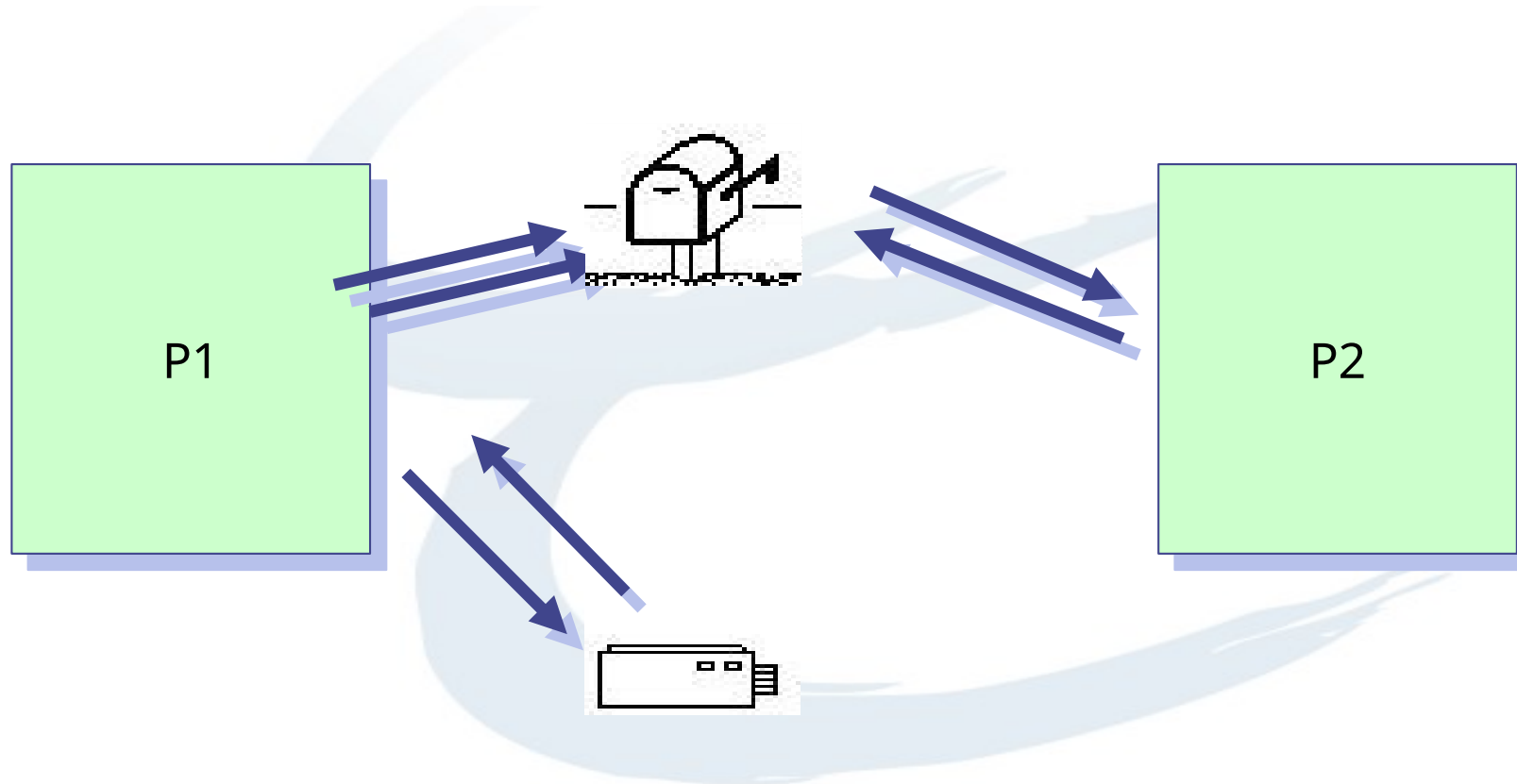
Result: $X = X + 2$

Scenario 2 (early preemption)

- READ X
 - ADD X, 1
 - —
 - —
 - WRITE X
 - —
- —
 - —
 - READ X
 - ADD X, 1
 - —
 - WRITE X

Result: $X = X + 1$!

mechanism



Creation example

```
#include <stdlib.h> #include <stdio.h> #include <sys/types.h>
#include <sys/ipc.h> #include <sys/sem.h> #include <fcntl.h>

#define SEM_SET 1
#define SEM0 0

int main()
{
    key_t key;
    int semid;

    key = ftok("Makefile", 0x27);
    if ((semid = semget(key, SEM_SET, S_IRWXU | IPC_CREAT)) == -1) perror("semget");
    printf("%s%d\n", "Semaphore set id is ", semid);

    union semun {
        int val;
    } arg;
    arg.val = 1;

    if (semctl(semid, SEM0, SETVAL, arg) == -1) perror("semctl");
    printf("Semaphore initialized\n");
}
```

Operations example

```
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <fcntl.h>
```

```
#define SEM_SET 1
#define SEM0 0
```

```
int semid;
```

```
void semcall(int op)
{
```

```
    struct sembuf sb;
```

```
    sb.sem_num = SEM0;
```

```
    sb.sem_op = op;
```

```
    sb.sem_flg = SEM_UNDO;
```

```
    if (semop(semid, &sb, 1) == -1) {
        perror("semop");
        exit(1);
    }
```

```
}
```

```
int main()
```

```
{
```

```
    key_t key;
```

```
    pid_t pid = getpid();
```

```
    key = ftok("Makefile", 0x27);
```

```
    if ((semid = semget(key, SEM_SET,
                        S_IRWXU | IPC_CREAT)) == -1) {
        perror("semget");
        exit(1);
    }
```

```
    printf("Semaphore %d\n", semid);
```

```
    printf("Access demand by %d\n", pid);
```

```
    semcall(-1);          /* get the sem */
```

```
    printf("Access ok %d\n", pid);
```

```
    sleep(10);
```

```
    semcall(1);           /* release the sem */
```

```
    printf("Resource freed par %d\n", pid);
```

```
}
```


Memory mapping

```
#include <sys/mman.h>

void *mmap(void *addr, size_t length,
           int prot, int flags,
           int fd, off_t offset);
int munmap(void *addr, size_t length);
```

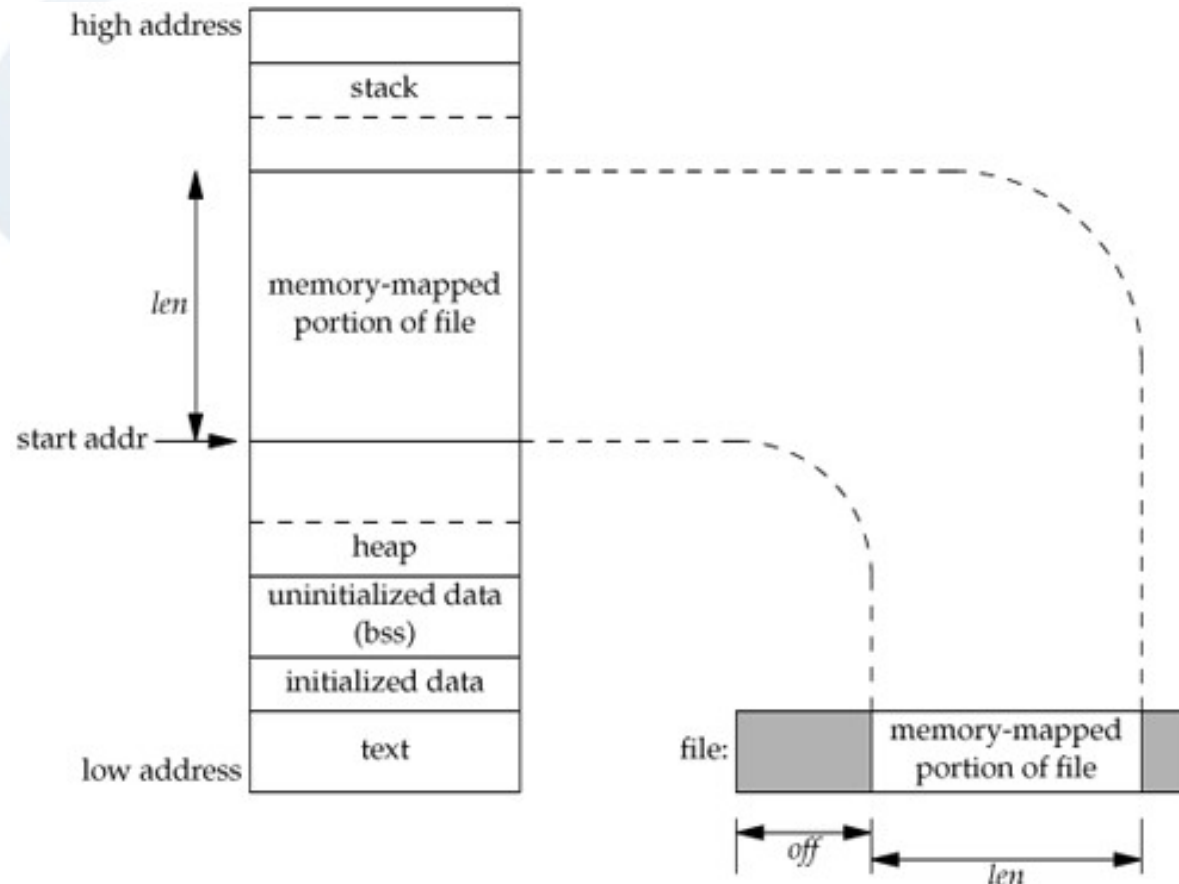
Creates a new mapping in the virtual address space

This mapping :

- lies in a segment that is neither in the heap nor in the stack
- can be linked to a file in a filesystems
- can be share between process.

Memory mapping

```
start_addr = mmap(NULL, len, PROT_READ, MAP_SHARED, fd, off);
```



3IS – System on Chip - Modules

2022 – Christophe Barès

Contributions de : David Picard,
Arnaud Revel, Mickaël Maillard

christophe.bares@ensea.fr

Introduction

- Le noyau Linux est un noyau monolithique **modulaire**
- Ajout ou enlèvement dynamiquement de code
 - Évite d'avoir un noyau énorme
 - Pas nécessaire de recompiler le noyau a chaque fois que l'on désire ajouter une fonctionnalité
 - Pas nécessaire de relancer la machine a chaque modification
- Les modules implémentent :
 - les **pilotes** (drivers)
 - des services dont le code doit être exécuté avec des **droits privilégiés**

Sources du noyaux

- Pour compiler un module, il faut
 - les sources (généralement dans /usr/src/linux)
 - ou les entêtes noyau (includes)
installable via le gestionnaire de paquets :

`apt install linux-headers-XXXXX`

```
.  
|-- 3rdparty  
|-- Documentation  
|-- arch  
|-- block  
|-- crypto  
|-- drivers  
|-- fs  
|-- include  
|-- init  
|-- ipc  
|-- kernel  
|-- lib  
|-- mm  
|-- net  
|-- scripts  
|-- security  
|-- sound  
'-- usr
```

Compilation

<https://www.kernel.org/doc/html/v5.8/kbuild/modules.html>

- Utilisation d'un Makefile spécifique :

```
obj-m := mon_mondule_a_moi.o
```

```
mon_mondule_a_moi-y := code_pour_mon_mondule_a_moi.o
```

```
KERNEL_SOURCE ?= /usr/src/linux-$(shell uname -r)/build
```

```
all:
```

```
    make -C $(KERNEL_SOURCE) M=$(PWD) modules
```

```
clean:
```

```
    make -C $(KERNEL_SOURCE) M=$(PWD) clean
```

```
install:
```

```
    make -C $(KERNEL_SOURCE) M=$(PWD) modules_install
```

Gestion des modules

Commandes en espace utilisateur :

- `insmod` : insérer le module sans vérification des dépendances
- `modprobe` : charger le module avec vérification des dépendances
- `rmmod` : décharger le module
- `lsmod` : lister les modules chargés
- `modinfo` : afficher des infos sur le module

Exemple minimaliste

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>

#define DRIVER_AUTHOR "Christophe Barès"
#define DRIVER_DESC "Hello world Module"
#define DRIVER_LICENSE "GPL"

int __init hello_init(void){
    printk(KERN_INFO "Hello world!\n");
    return 0;
}
void __exit hello_exit(void) {
    printk(KERN_ALERT "Bye bye...\n");
}

module_init(hello_init);
module_exit(hello_exit);

MODULE_LICENSE(DRIVER_LICENSE);
MODULE_AUTHOR(DRIVER_AUTHOR);
MODULE_DESCRIPTION(DRIVER_DESC);
```


Macros `__exit` et `__init`

Annotations : `#include <linux/init.h>`

Un module peut aussi être totalement intégré au noyau au moment de sa compilation: `build in`

Dans ce cas:

- `__init` indique au noyau d'effacer cette fonction après l'initialisation du driver
- `__exit` indique au compilateur de ne pas compiler cette fonction dans le noyau (aucune utilité de le cas "`build in`")

```
$ sudo dmesg |grep Freeing
[    0.172689] Freeing SMP alternatives memory: 40K
[    0.774623] Freeing initrd memory: 55752K
[    1.338595] Freeing unused decrypted memory: 2040K
[    1.338966] Freeing unused kernel image (initmem) memory: 2636K
[    1.369737] Freeing unused kernel image (text/rodata gap) memory: 2044K
[    1.370094] Freeing unused kernel image (rodata/data gap) memory: 1448K
```

Paramètres d'un module

- Différentes macros pour déclarer les arguments à passer au module:

```
module_param(var, type, droits)
```

```
module_param_array(var, type, addr, droits)
```

```
module_param_string(nom_dans_modinfo, var, taille, droits)
```

- Utilisation dans le code du module:

```
int myint = 3;
```

```
module_param(myint, int, 0);
```

- Chargement :

```
modprobe monmodule myint=2
```

procfs

- `procfs` (process file system) est un pseudo-système de fichiers monté sur `/proc` utilisé pour accéder aux informations du noyau en cours d'exécution
- Ne correspond à aucun fichier sur les block devices
- Infos sur les processus (`/proc/[0-9]+/`)
- Debugging, tweaking, information, etc

Création : `proc_create`

```
static inline struct proc_dir_entry *proc_create(  
    const char *name,  
    umode_t mode,  
    struct proc_dir_entry *parent,  
    const struct proc_ops *proc_fops)
```

- *name* : nom du fichier à créer
- *mode* : droits d'accès au fichier
- *parent* : entrée parente dans l'arborescence procfs (NULL si à la racine de /proc/)
- *proc_ops*: structure des pointeurs des fonctions utilisables (*file_operations* "classique" avant le noyau 5.6)

Chemin complexe

Support des chemins complets (“/proc/un/chemin/complexe”)

```
struct proc_dir_entry *proc_mkdir(  
    const char *name,  
    struct proc_dir_entry *parent)
```

- *name* : nom du répertoire
- *parent* : entrée parente dans l'arborescence procfs (NULL si à la racine de /proc/)

Suppression

```
void remove_proc_entry(  
    const char* name,  
    struct proc_dir_entry* parent);
```

- *name* : nom du répertoire
- *parent* : entrée parente dans l'arborescence

Communication

```
struct proc_ops {
    int (*proc_open)(struct inode *, struct file *);
    ssize_t (*proc_read)(struct file *, char __user *, size_t, loff_t *);
    ssize_t (*proc_write)(struct file *, const char __user *, size_t, loff_t *);
    loff_t (*proc_lseek)(struct file *, loff_t, int);
    int (*proc_release)(struct inode *, struct file *);
    __poll_t (*proc_poll)(struct file *, struct poll_table_struct *);
    long (*proc_ioctl)(struct file *, unsigned int, unsigned long);
#ifdef CONFIG_COMPAT
    long (*proc_compat_ioctl)(struct file *, unsigned int, unsigned long);
#endif
    int (*proc_mmap)(struct file *, struct vm_area_struct *);
    unsigned long (*proc_get_unmapped_area)(struct file *, unsigned long,
        unsigned long, unsigned long, unsigned long);
};
```

https://elixir.bootlin.com/linux/v5.6.19/source/include/linux/proc_fs.h#L15

Read function

```
ssize_t (*proc_read)(struct file *file, char __user *buffer,  
    size_t count, loff_t *offset);
```

- *file* : décrit le fichier à lire
- *buffer* : zone mémoire où écrire les données lues
- *count* : max octets à lire dans buffer
- *offset* : position dans le fichier

Read exemple

```
#define message ...
#define TAILLE ...

ssize_t fops_read(struct file *file, char __user * buffer,
                  size_t count, loff_t * ppos)
{
    int copy;
    if (count > TAILLE) {count = TAILLE;}
    if (copy = copy_to_user(buffer, message, count)){
        return -EFAULT;
    }
    printk(KERN_INFO "message read, %d, %p\n", copy, buffer);
    return count - copy;
}
```

Write

```
ssize_t (*proc_write) (struct file* file, const char __user * buffer,  
                        size_t count, loff_t *offset);
```

- *file* : décrit le fichier à écrire
- *buffer* : zone mémoire où lire les données à écrire
- *count* : max octets à écrire dans buffer
- *offset* : position dans le fichier

Write exemple

```
ssize_t fops_write(struct file * file, const char __user * buffer,
                  size_t count, loff_t * ppos)
{
    int len = count;
    if (len > TAILLE) {len = TAILLE;}

    printk(KERN_INFO "Recieving new message\n");
    if (copy_from_user(message, buffer, count)) {
        return -EFAULT;
    }
    message[count] = '\0';
    size_of_message = strlen(message);
    printk(KERN_INFO "New message : %s\n", message);
    return count;
}
```

Fichiers virtuels = Sequence Files (seq_file)

Permet de générer facilement des données répétitives en lecture (tableaux, structures, listes chaînées..)

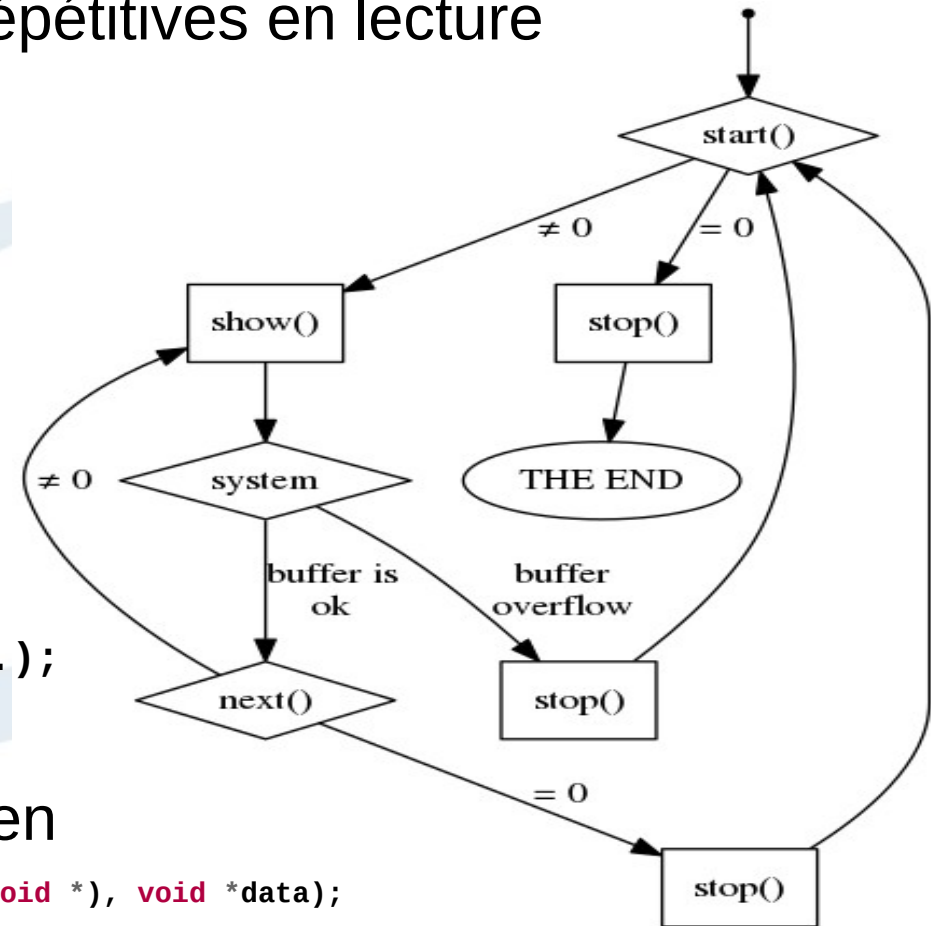
```
struct seq_operations {  
    void * (*start) (struct seq_file *m, loff_t *pos);  
    void (*stop) (struct seq_file *m, void *v);  
    void * (*next) (struct seq_file *m, void *v, loff_t *pos);  
    int (*show) (struct seq_file *m, void *v);  
};
```

Dans la fonction show, ce “printf” envoie les données au read :

```
int seq_printf(struct seq_file *m, const char *f, ...);
```

Quand il n’y a qu’une seule donnée,
C’est à dire quand next est inutile: single_open

```
int single_open(struct file *file, int (*show)(struct seq_file *, void *), void *data);  
int single_release(struct inode *inode, struct file *file);
```



Sequence Files (seq_file)

```
#include <linux/module.h>
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/fs.h>           // for basic filesystem
#include <linux/proc_fs.h>      // for the proc filesystem
#include <linux/seq_file.h>     // for sequence files
#include <linux/jiffies.h>      // for jiffies

static struct proc_dir_entry *jif_file;

static int jif_show(struct seq_file *m, void *v)
{
    seq_printf(m, "%llu\n", get_jiffies_64());
    return 0;
}

static int jif_open(struct inode *inode, struct file *file)
{
    return single_open(file, jif_show, NULL);
}

static const struct proc_ops jif_pops = {
    .open = jif_open,
    .read = seq_read,           // from seq_file.h
    .llseek = seq_lseek,        // from seq_file.h
    .release = single_release,   // Instead of seq_release
};
```

```
static int __init jif_init(void)
{
    jif_file = proc_create("jif", 0, NULL, &jif_pops);

    if (!jif_file) {
        return -ENOMEM;
    }

    return 0;
}

static void __exit jif_exit(void)
{
    remove_proc_entry("jif", NULL);
}

module_init(jif_init);
module_exit(jif_exit);

MODULE_LICENSE("GPL");
```

Sequence Files 2 (seq_file)

```
#include <linux/module.h>
#include <linux/init.h> // for __init & __exit macros
#include <linux/kernel.h>
#include <linux/fs.h>      // for basic filesystem
#include <linux/proc_fs.h> // for the proc filesystem
#include <linux/seq_file.h> // for sequence files
#include <linux/jiffies.h> // for jiffies
```

```
static struct proc_dir_entry *jif_file;
```

```
static int jif_show(struct seq_file *m, void *v)
{
    seq_printf(m, "%llu\n", get_jiffies_64());
    return 0;
}
```

```
// This macro defines "jif_proc_ops" and use "jif_show"
DEFINE_PROC_SHOW_ATTRIBUTE(jif);
```

```
static int __init jif_init(void)
{
    jif_file = proc_create("jif", 0,
                          NULL, &jif_proc_ops);

    if (!jif_file) {
        return -ENOMEM;
    }

    return 0;
}

static void __exit jif_exit(void)
{
    remove_proc_entry("jif", NULL);
}

module_init(jif_init);
module_exit(jif_exit);

MODULE_LICENSE("GPL");
```

Entrée périphérique

- Entrée périphérique : fichier pour d'interagir avec un module
- Création :

```
# mknod /dev/nom_du_peripherique c majeure mineure
```
- Périphérique de type char (c) ou block (b)
- majeure : numéro définissant une classe de périphériques
 - disques durs, usb, pci...
- mineure : instance particulière pour ce type de composant
 - les constructeurs peuvent être différents
 - le pilote est le même

Types (extrait de documentation/devices.txt)

3 block First MFM, RLL and IDE hard disk/CD-ROM interface
 0 = /dev/hda Master: whole disk (or CD-ROM)
 64 = /dev/hdb Slave: whole disk (or CD-ROM)

For partitions, add to the whole disk device number:

 0 = /dev/hd? Whole disk
 1 = /dev/hd?1 First partition
 2 = /dev/hd?2 Second partition
 ...
 63 = /dev/hd?63 63rd partition

For Linux/i386, partitions 1-4 are the primary partitions, and 5 and above are logical partitions. Other versions of Linux use partitioning schemes appropriate to their respective architectures.

4 char TTY devices
 0 = /dev/tty0 Current virtual console
 1 = /dev/tty1 First virtual console
 ...
 63 = /dev/tty63 63rd virtual console
 64 = /dev/ttyS0 First UART serial port
 ...
 255 = /dev/ttyS191 192nd UART serial port

UART serial ports refer to 8250/16450/16550 series devices.

Pilotes : implementer les file_ops associées

```
struct file_operations {
    struct module *owner;
    loff_t (*llseek) (struct file *, loff_t, int);
    ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
    ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);
    ssize_t (*aio_read) (struct kiocb *, const struct iovec *, ulong, loff_t);
    ssize_t (*aio_write) (struct kiocb *, const struct iovec *, ulong, loff_t);
    int (*readdir) (struct file *, void *, filldir_t);
    uint (*poll) (struct file *, struct poll_table_struct *);
    int (*ioctl) (struct inode *, struct file *, uint, ulong);
    long (*unlocked_ioctl) (struct file *, uint, ulong);
    long (*compat_ioctl) (struct file *, uint, ulong);
    int (*mmap) (struct file *, struct vm_area_struct *);
    int (*open) (struct inode *, struct file *);
    int (*flush) (struct file *, fl_owner_t id);
    int (*release) (struct inode *, struct file *);
    int (*fsync) (struct file *, struct dentry *, int datasync);
    int (*aio_fsync) (struct kiocb *, int datasync);
    int (*fasync) (int, struct file *, int);
    int (*lock) (struct file *, int, struct file_lock *);
    ssize_t (*sendfile) (struct file *, loff_t *, size_t, read_actor_t, void *);
    ssize_t (*sendpage) (struct file *, struct page *, int, size_t, loff_t *, int);
    ulong (*get_unmapped_area) (struct file *, ulong, ulong, ulong, ulong);
    int (*check_flags) (int);
    int (*dir_notify) (struct file *filp, ulong arg);
    int (*flock) (struct file *, int, struct file_lock *);
    ssize_t (*splice_write) (struct pipe_inode_info *, struct file *, loff_t *, size_t, uint);
    ssize_t (*splice_read) (struct file *, loff_t *, struct pipe_inode_info *, size_t, uint);
};
```

Quelles opérations ?

- Il n'est pas obligatoire de définir toutes les opérations
 - Open : pour initialiser les ressources liées au périphériques.
 - Release : pour libérer ces mêmes ressources.
 - read/write : permet d'échanger des données avec le périphérique.

```
static struct file_operations mon_driver_fops = {  
    .owner =    THIS_MODULE,  
    .read =    mon_driver_read,  
    .write =    mon_driver_write,  
    .open =    mon_driver_open,  
    .release = mon_driver_release,  
};
```

```

#include <linux/kernel.h>    /* printk() */
#include <linux/module.h>    /* modules */
#include <linux/fs.h>        /* file_operations */

MODULE_DESCRIPTION("mydriver1");
MODULE_AUTHOR("Stelian Pop/Pierre Fichoux, Open Wide");
MODULE_LICENSE("GPL");

/* Arguments */
static int major = 0; /* Major number */
module_param(major, int, 0644);
MODULE_PARM_DESC(major, "Static major number (none = dynamic)");

/* File operations */
static ssize_t mydriver1_read(struct file *file, char *buf, size_t count, loff_t *ppos)
{
    printk(KERN_INFO "mydriver1: read()\n");
    return count;
}
static ssize_t mydriver1_write(struct file *file, const char *buf, size_t count, loff_t *ppos)
{
    printk(KERN_INFO "mydriver1: write()\n");
    return count;
}
static int mydriver1_open(struct inode *inode, struct file *file)
{
    printk(KERN_INFO "mydriver1: open()\n");
    return 0;
}
static int mydriver1_release(struct inode *inode, struct file *file)
{
    printk(KERN_INFO "mydriver1: release()\n");
    return 0;
}

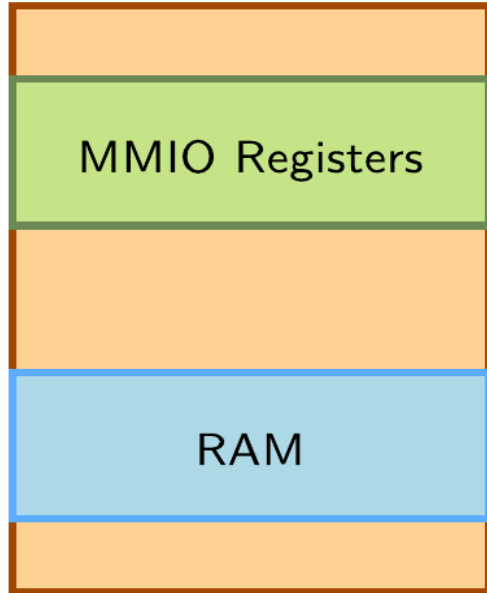
```

```

static struct file_operations mydriver1_fops = {
    .owner =    THIS_MODULE,
    .read =    mydriver1_read,
    .write =    mydriver1_write,
    .open =    mydriver1_open,
    .release =  mydriver1_release,
};
/* Init and Exit */
static int __init mydriver1_init(void)
{
    int ret;
    ret = register_chrdev(major, "mydriver1", &mydriver1_fops);
    if (ret < 0) {
        printk(KERN_WARNING "mydriver1: unable to get a major\n");
        return ret;
    }
    if (major == 0) major = ret; /* dynamic value */
    printk(KERN_INFO "mydriver1: successfully loaded with major %d\n", major);
    return 0;
}
static void __exit mydriver1_exit(void)
{
    unregister_chrdev(major, "mydriver1");
    printk(KERN_INFO "mydriver1: successfully unloaded\n");
}
/* Module entry points */
module_init(mydriver1_init);
module_exit(mydriver1_exit);

```

Port I/O vs. Memory-Mapped I/O



Physical Memory
address space, accessed with
normal load/store instructions



Separate I/O address space,
accessed with specific instructions

I/O Ports

- Chaque périphérique est connecté à un bus d'I/O et dispose de ces propres adresses
- On appelle ces adresses les “ports d'E/S” (I/O ports)
- Le noyau permet de lire et écrire sur le bus d'I/O avec des routines spécifiques
- On peut réserver des ports afin qu'un module ait l'exclusivité de leur utilisation

Utilisation des ports d'E/S

```
int inb(int read_addr)
```

- Récupère un octet depuis le port défini par l'adresse read_addr

```
void outb(int write_addr, char value)
```

- Envoie l'octet value sur le port défini par l'adresse write_addr.
- Une routine par type (b pour byte, w pour 16bits, l pour 32bits)

Exemple (tiré du LDD3)

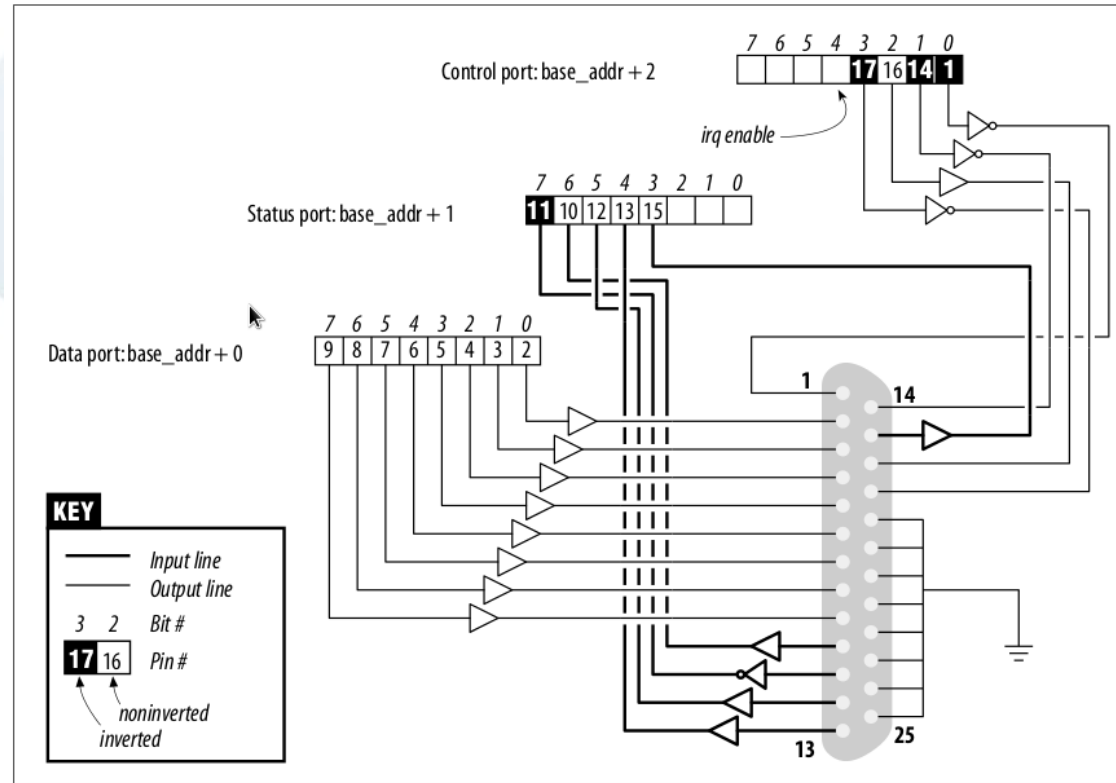


Figure 9-1. The pinout of the parallel port

Exemple

```
/* default is the first printer port on PC's */
static unsigned long base = 0x378;

/* Version-specific methods for the fops structure. */
ssize_t short_write(struct file *filp, const char __user *buf, size_t count, loff_t *f_pos)
{
    int retval = count;
    unsigned char *kbuf = kmalloc(count, GFP_KERNEL), *ptr;

    if (!kbuf) return -ENOMEM;
    if (copy_from_user(kbuf, buf, count)) return -EFAULT;
    ptr = kbuf;

    while (count--) {
        outb(*(ptr++), base);
        wmb(); // write memory barrier: ensure successive outb order
    }
    return retval;
}

struct file_operations short_fops = {
    .owner    = THIS_MODULE,
    .write    = short_write,
};
```

Kernel malloc (1)

```
#include <linux/slab.h>
```

```
void * kmalloc(size_t size, int flags)
```

```
void kfree(const void *ptr)
```

- Size < PAGE_SIZE (4k) => contiguous
- Return NULL on error
- Flags:
 - GFP_KERNEL: process context, can sleep
 - GFP_ATOMIC: process context, cannot sleep interrupt handler, tasklet,
 - | GFP_DMA: DMA-able memory.

Kernel malloc (2)

```
#include <linux/vmalloc.h>
void * vmalloc(size_t size)
void vfree(const void *ptr)
```

- Here, you can choose $\text{Size} > \text{PAGE_SIZE}$ (4k)
=> physically not contiguous
- Return NULL on error

I/O memory

- Les périphériques modernes contiennent un petit espace de mémoire adressable.
 - Des registres de contrôle (similaires aux ports d'E/S)
 - Du stockage de données (textures video, paquets réseau, ...)
 - ...
- Le noyau permet de mapper cet espace dans l'espace mémoire
- On appelle ce mapping la mémoire d'E/S (I/O memory)

Utilisation de la mémoire d'E/S

- Mapping :
 - `struct resource *request_mem_region(unsigned long start, unsigned long len, char *name);`
- Démapping :
 - `void release_mem_region(unsigned long start, unsigned long len);`
- Test de disponibilité :
 - `int check_mem_region(unsigned long start, unsigned long len);`
- Remapping :
 - `void *ioremap(unsigned long phys_addr, unsigned long size);`
 - Le pointeur obtenu pointe vers la mémoire du périphérique
 - On ne doit pas utiliser ce pointeur comme n'importe quel pointeur, à cause de problèmes d'optimisation matériel (reordering des instructions) !

Communication avec la mémoire d'E/S

- Lire :
 - `unsigned int ioread8(void *addr);`
 - `unsigned int ioread16(void *addr);`
 - `unsigned int ioread32(void *addr);`
- Écrire :
 - `void iowrite8(u8 value, void *addr);`
 - `void iowrite16(u16 value, void *addr);`
 - `void iowrite32(u32 value, void *addr);`
- En boucle :
 - `void ioread8_rep(void *addr, void *buf, unsigned long count);`
 - ...

Timer

- `void fastcall init_timer(struct timer_list * timer)`
création d'un timer logiciel programmé
 - `timer_list` structure définissant un champ `function` appelée lorsque le temps défini par le timer est dépassé et `data` qui définit un paramètre à passer à la fonction
- `int mod_timer(struct timer_list * timer, unsigned long expires)`
active le timer et donne le moment en nombre de ticks depuis le démarrage du système où doit être déclenché le timer
 - Si l'on veut déclencher le timer dans 100 ticks, `expires` vaut `jiffies+100`
 - Il faut réarmer le timer dans la fonction de gestion si l'on veut un timer périodique

Exemple

```
#include <linux/timer.h>
#include <linux/asm/param.h>    // pour HZ
#define INTERVALLE 1*HZ        // 1 seconde

static struct timer_list timer;

static void montimer(unsigned long data)
{
    ...
    /* Il faut réarmer le timer si l'on veut un appel périodique */
    mod_timer(&timer, jiffies + INTERVALLE);
    ...
}

static int __init module_init(void)
{
    ...
    init_timer(&timer);
    timer.function = montimer;
    timer.data = 0;
    mod_timer(&timer, jiffies + INTERVALLE);
}
```


Time in Kernel

- La constante HZ définit les “ticks”
- Il y a HZ ticks par seconde.
- Les ticks sont importants pour l'ordonnanceur
- Les ticks sont générés par un timer spécifique
- Les “jiffies” représentent le nombre de ticks depuis le boot

```
#include <linux/jiffies.h>
```

```
u64 get_jiffies_64(void);
```

```
extern unsigned int jiffies_to_msecs(const unsigned long j);
```

```
extern unsigned int jiffies_to_usecs(const unsigned long j);
```

```
...
```

```
time_after(a,b)
```

```
time_before(a,b)
```

```
...
```

– Actuellement: HZ = 100

Mutex

- Ressources critiques du noyau : au module de gérer lui-même les accès
- `DECLARE_MUTEX(sem)`
 - Permet de déclarer un sémaphore
- `mutex_lock(&sem)`
 - Sert à l'acquisition
- `mutex_unlock(&sem)`
 - Effectue la libération

Interruptions

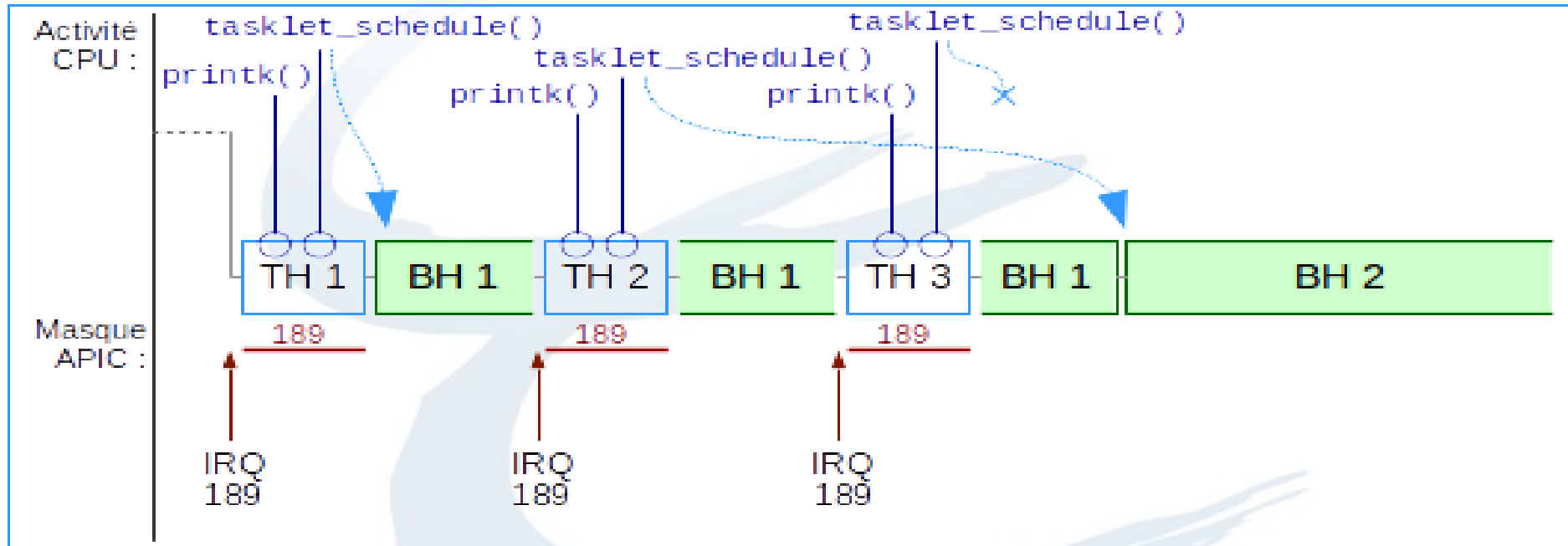
```
int request_irq(unsigned int irq,  
               void (*gest)(int, void *, struct pt_regs *),  
               unsigned long drap_interrupt,  
               const char *devname,  
               void *dev_id)
```

- Permet d'associer un numéro d'interruption (`irq`) à une fonction `gest` qui traite cette interruption pour le compte du driver du périphérique `devname` repéré par son identifiant `dev_id`
- La fonction de gestion de l'interruption retourne:
 - `IRQ_HANDLED` si l'interruption a été traitée correctement
 - `IRQ_NONE` sinon
- La libération d'une interruption s'effectue via l'appel à `void free_irq(unsigned int irq, void *dev_id)`

Gestion des interruptions longues

- Parfois le traitement à effectuer lors d'une interruption peut être très long
- Le noyau doit rester le moins longtemps possible dans une routine d'interruption (blocage du système)
- Traitement en deux temps :
 - Partie top-half : acquittement rapide de l'interruption et ajout du traitement à effectuer dans une liste de tasklet (faible latence) ou une file de traitements (workqueue)
 - Partie bottom-half : gestion différée (asynchrone) du traitement de l'interruption par la tasklet ou la workqueue

Gestion des interruptions longues



Les tasklets sont exécutées dès que possible: bypass l'ordonnanceur

Les Workqueues sont exécutées dans un contexte de processus, après ordonnancement.

Tasklets

- Les tasklets sont des appels effectués uniquement dans un contexte d'interruption
- Créer une tasklet revient à faire une demande au noyau pour exécuter une tâche atomique de manière différée
 - En fonction de sa disponibilité
 - Jamais au delà d'un tick
 - Limite le temps pendant lequel l'interruption est masquée (seulement pendant le top-half)

Tasklets

- **DECLARE_TASKLET(name, func, data)**
 - Permet de déclarer la tasklet `name`, de l'associer à la fonction `func` en lui passant les données `data`
- **tasklet_schedule(&name)**
 - Demande l'exécution de la tasklet
- **tasklet_hi_schedule(&name)**
 - Demande d'exécution haute priorité à n'utiliser que dans le cadre de pilote faible latence tels que les buffer audio.
- **DECLARE_TASKLET_DISABLED(), tasklet_disable(), tasklet_disable_nosync()**
 - Inhiber une tasklet
- **tasklet_enable()**
 - Permet de la réactiver
- **tasklet_kill()**
 - Permet de la supprimer

Exemple

```
#include <linux/interrupt.h>
```

```
void tasklet_function(unsigned long);
```

```
char tasklet_data[64];
```

```
DECLARE_TASKLET(test_tasklet, tasklet_function, (unsigned long) &tasklet_data);
```

```
void tasklet_function(unsigned long data)
```

```
{
```

```
    struct timeval now;
```

```
    do_gettimeofday(&now);
```

```
    printk("%s at %ld,%ld\n", (char *) data, now.tv_sec, now.tv_usec);
```

```
}
```

```
int init_module(void) {
```

```
    sprintf(tasklet_data, "%s\n", "Linux tasklet called in init_module");
```

```
    tasklet_schedule(&test_tasklet);
```

```
}
```


Workqueue

linux/workqueue.h

- Fonctionnent dans le contexte d'un processus noyau
 - plus de souplesse quant à la possibilité d'être interrompues/reprises
- Peuvent s'exécuter de manière différée
- Création :
 - **create_workqueue(name)**
 - Crée la file et renvoie un pointeur sur une structure `struct workqueue_struct`
- Destruction :
 - **void destroy_workqueue(struct workqueue_struct *queue)**
 - Permet de détruire une file

Utilisation des workqueues

- `DECLARE_WORK(nom, fonction, donnees)`
 - Déclare la workqueue en associant une fonction à exécuter à un nom de workqueue
- `int queue_work(struct workqueue_struct *wq, struct work_struct *work)`
 - La tâche est mise dans la file immédiatement
- `int queue_delayed_work(struct workqueue_struct *wq, struct work_struct *work, unsigned long delay)`
 - La tâche est mise dans la file mais ne s'exécutera pas avant que `delay` ticks ne soient passés
- `int cancel_delayed_work(struct work_struct *work)`
 - permet de supprimer une tâche
- `void flush_workqueue(struct workqueue_struct *queue)`
 - Permet de vider une file de travail

VEEK-MT2S (DE-10)

- Cyclone V SE SoC—5CSXFC6D6F31C6N
 - Dual-core ARM Cortex-A9 (HPS)
 - 110K programmable logic elements
 - 5,140 Kbits embedded memory
 - 6 fractional PLLs
 - 2 hard memory controllers
- SDRAM 1 GB
- Peripherals:
 - Two USB 2.0 Host
 - 10/100/1000 Ethernet
 - PS/2 mouse/keyboard
 - IR emitter/receiver
 - I2C multiplexer



VEEK-MT2S (DE-10)

- Audio:
 - 24-bit CODEC, line-in, line-out, and microphone-in jacks
- Video Input:
 - TV decoder (NTSC/PAL/SECAM)
 - Video-in connector
- ADC:
 - Fast throughput rate: 1 MSPS
 - Channel number: 8
 - Resolution: 12-bit
- Switches, Buttons and LEDs
- G-sensor for the HPS



VEEK-MT2S (DE-10)

8-Megapixel Digital Camera

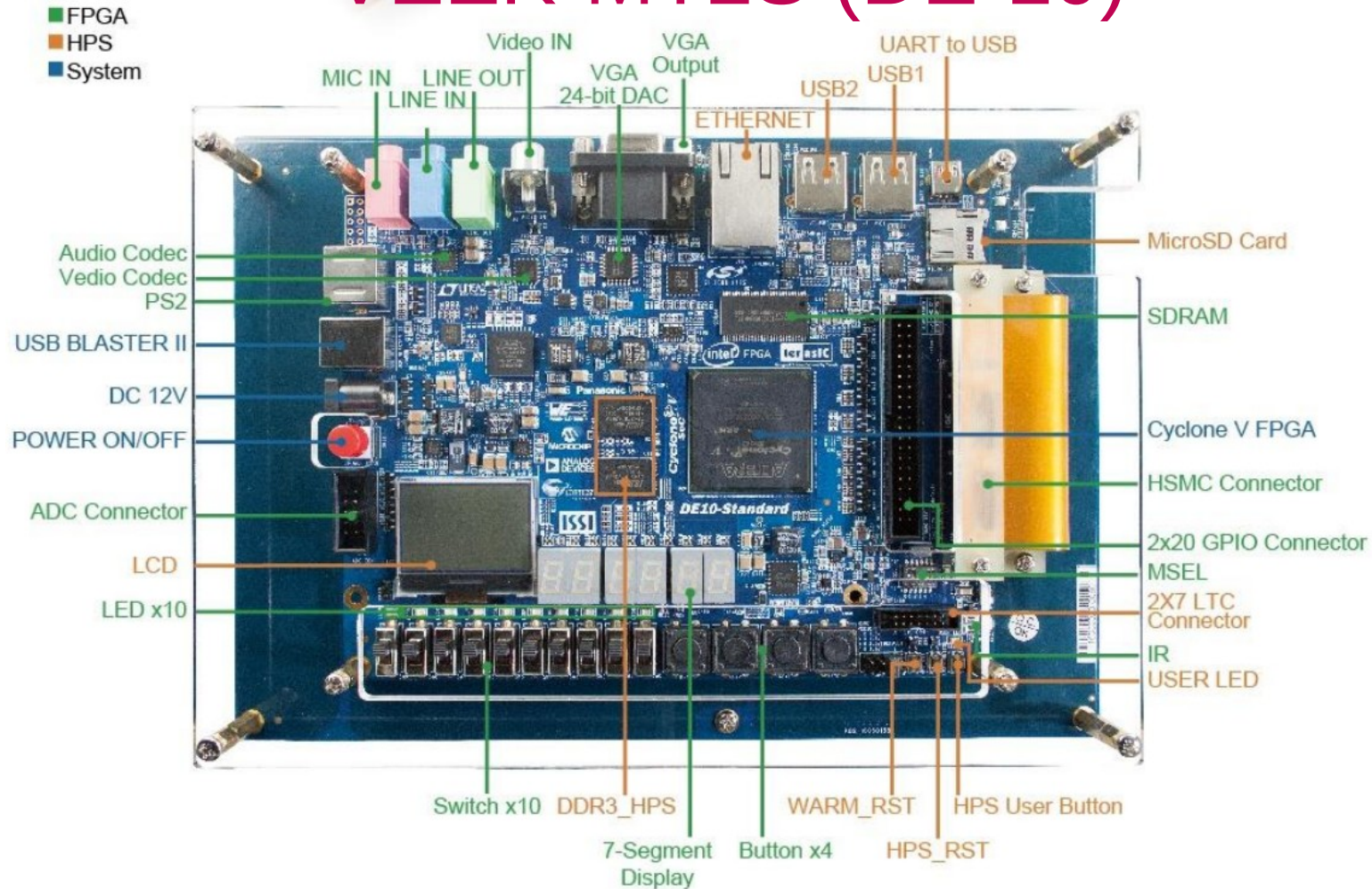
Ambient Light Sensor

7" 800x480 LCD
Touch Panel

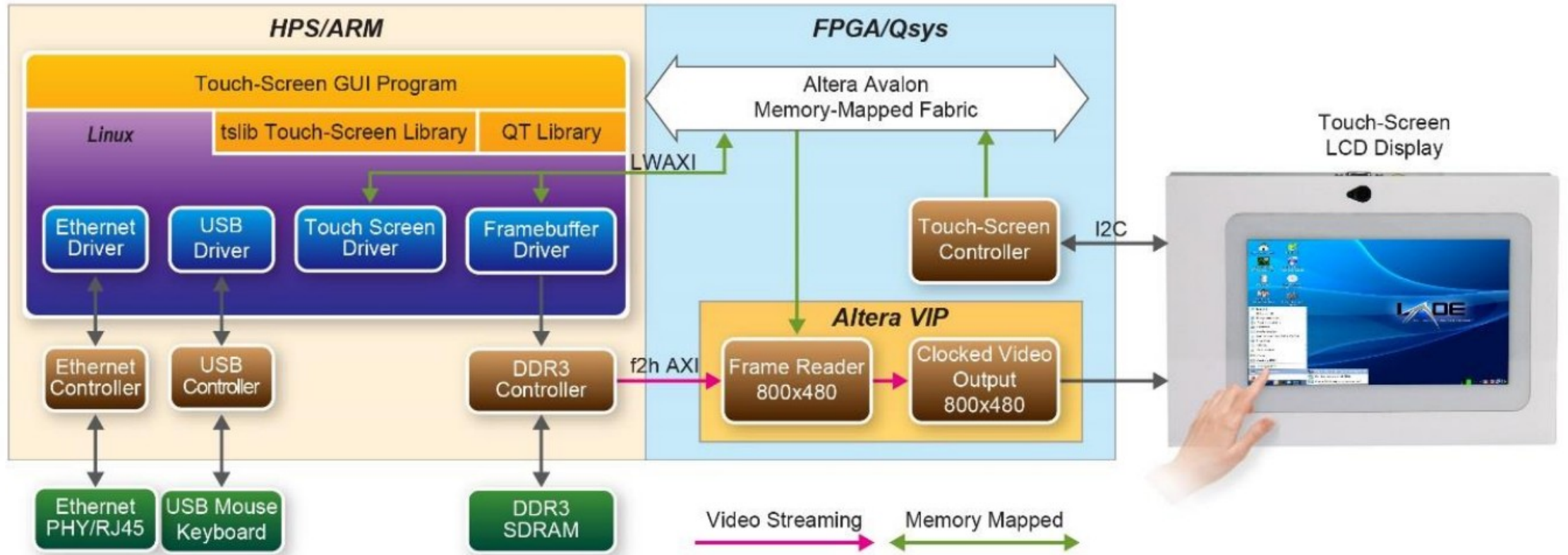
MPU9250 Accelerometer
Gyroscope
Magnetometer Sensor



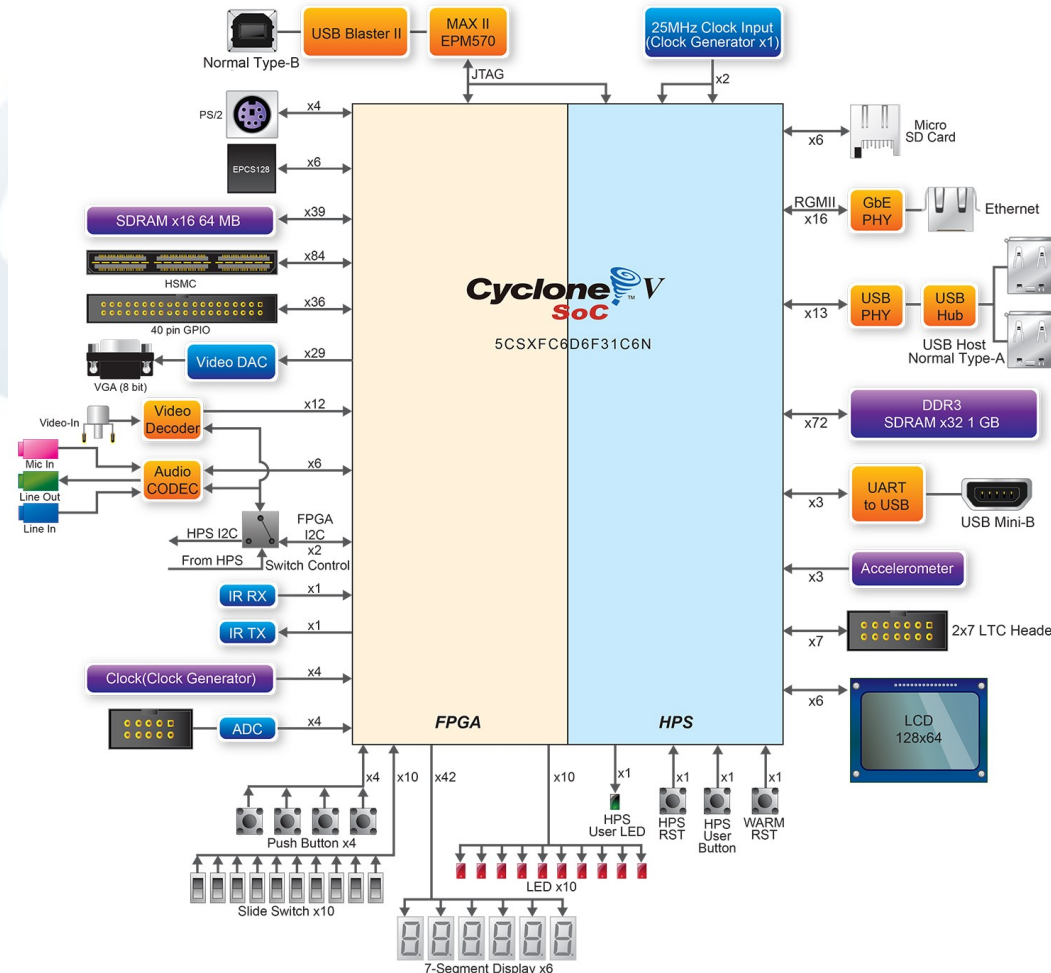
VEEK-MT2S (DE-10)



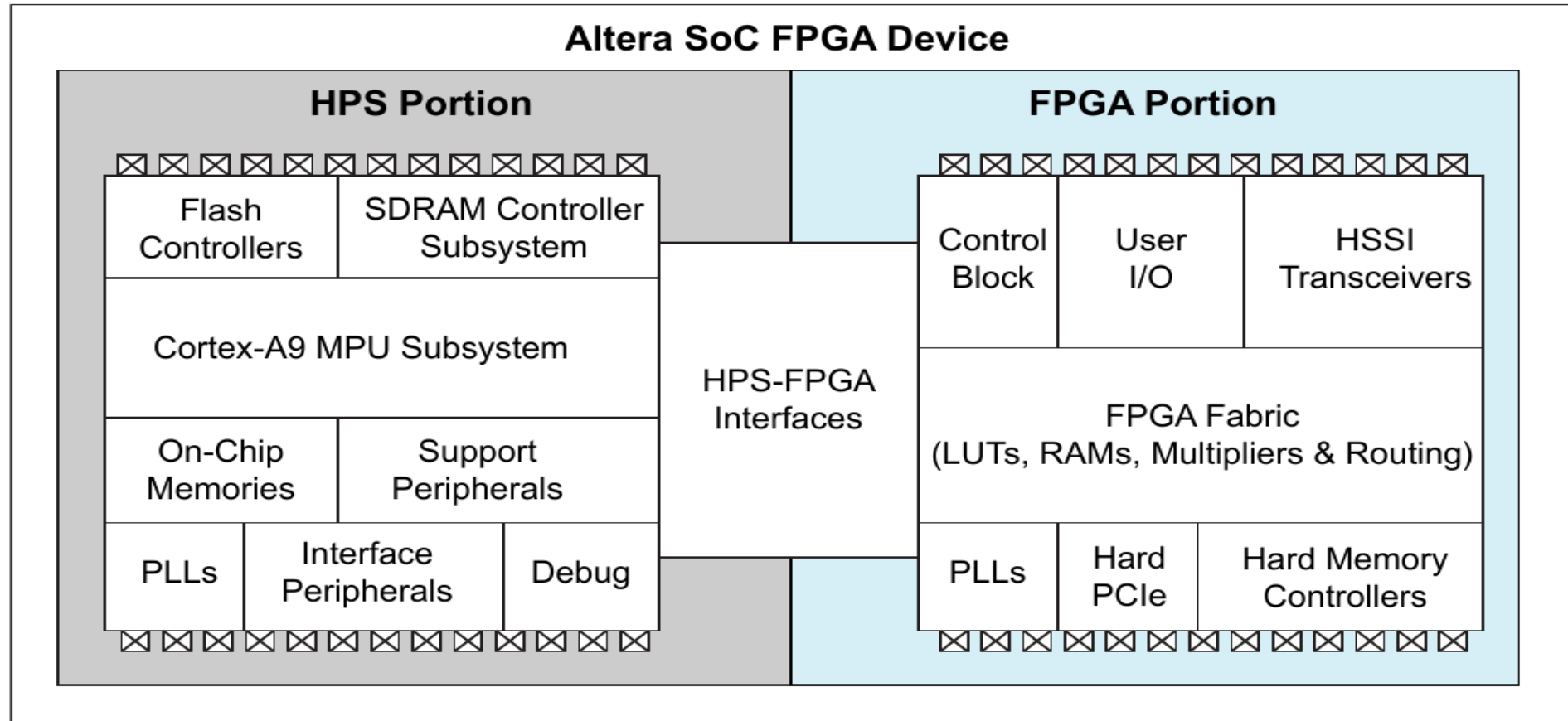
VEEK-MT2S (DE-10)

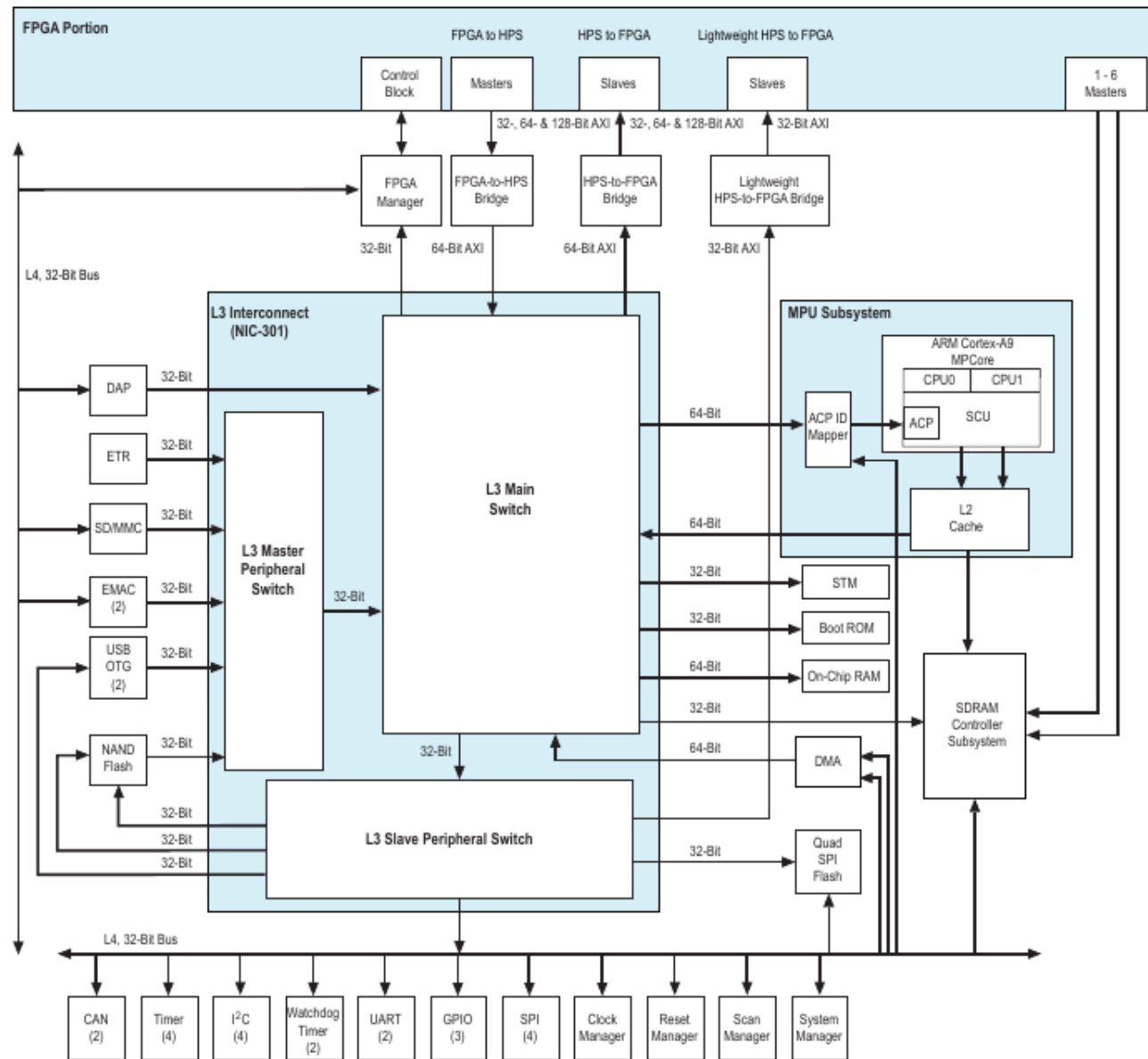


VEEK-MT2S (DE-10)

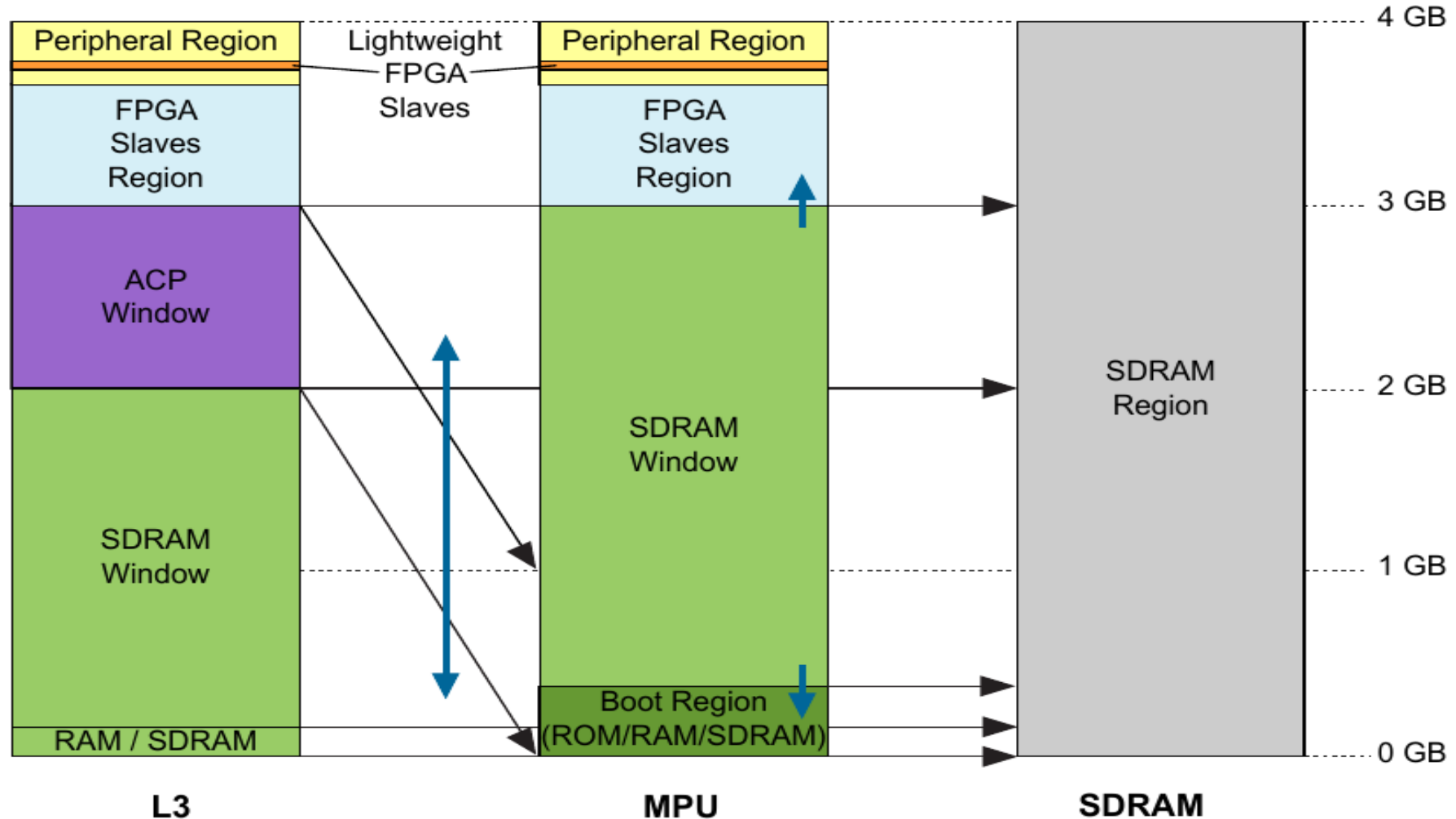


VEEK-MT2S (DE-10)





Espaces d'adressage du HPS



Adresses

Identifier	Region Name	Base Address	Size
FPGASLAVES	FPGA slaves	0xC0000000	960 MB
PERIPH	Peripheral	0xFC000000	64 MB
LWFPGASLAVES ⁽¹⁾	Lightweight FPGA slaves	0xFF200000	2 MB

Slave Identifier	Slave Title	Base Address	Size
STM	STM	0xFC000000	48 MB
DAP	DAP	0xFF000000	2 MB
LWFPGASLAVES	FPGA slaves accessed with lightweight FPGA-to-HPS AXI bridge	0xFF200000	2 MB
LWHPS2FPGAREGS	Lightweight FPGA-to-HPS AXI bridge GPV	0xFF400000	1 MB
HPS2FPGAREGS	HPS-to-FPGA AXI bridge GPV	0xFF500000	1 MB
FPGA2HPSREGS	FPGA-to-HPS AXI bridge GPV	0xFF600000	1 MB
EMAC0	EMAC0	0xFF700000	8 KB

Slave Identifier	Slave Title	Base Address	Size
STM	STM	0xFC000000	48 MB
DAP	DAP	0xFF000000	2 MB
LWFPGASLAVES	FPGA slaves accessed with lightweight FPGA-to-HPS AXI bridge	0xFF200000	2 MB
LWHPS2FPGAREGS	Lightweight FPGA-to-HPS AXI bridge GPV	0xFF400000	1 MB
HPS2FPGAREGS	HPS-to-FPGA AXI bridge GPV	0xFF500000	1 MB
FPGA2HPSREGS	FPGA-to-HPS AXI bridge GPV	0xFF600000	1 MB
EMAC0	EMAC0	0xFF700000	8 KB
EMAC1	EMAC1	0xFF702000	8 KB
SDMMC	SD/MMC	0xFF704000	4 KB
QSPIREGS	Quad SPI flash controller registers	0xFF705000	4 KB
FPGAMGRREGS	FPGA manager registers	0xFF706000	4 KB
ACPIDMAP	ACP ID mapper registers	0xFF707000	4 KB

Slave Identifier	Slave Title	Base Address	Size
GPIO0	GPIO0	0xFF708000	4 KB
GPIO1	GPIO1	0xFF709000	4 KB
GPIO2	GPIO2	0xFF70A000	4 KB
L3REGS	L3 interconnect GPV	0xFF800000	1 MB
NANDDATA	NAND controller data	0xFF900000	1 MB
QSPIDATA	Quad SPI flash data	0xFFA00000	1 MB
USB0	USB0 OTG controller registers	0xFFB00000	256 KB
USB1	USB1 OTG controller registers	0xFFB40000	256 KB
NANDREGS	NAND controller registers	0xFFB80000	64 KB
FPGAMGRDATA	FPGA manager configuration data	0xFFB90000	4 KB
CAN0	CAN0 controller registers	0xFFC00000	4 KB
CAN1	CAN1 controller registers	0xFFC01000	4 KB
UART0	UART0	0xFFC02000	4 KB
UART1	UART1	0xFFC03000	4 KB
I2C0	I2C0	0xFFC04000	4 KB
I2C1	I2C1	0xFFC05000	4 KB
I2C2	I2C2	0xFFC06000	4 KB
I2C3	I2C3	0xFFC07000	4 KB